



Bedside CGM Companion

Hackathon Project Specification

Contents

Terminology

What It Does

One System, Three Audiences (Sponsor Challenges)

A Day With Irin (Five Perspectives)

How It All Works

Complete Parts List (Exact Items)

Tech Stack

Team, Repository, and Workflow

- Repository Structure

Initial Setup, Per Person

- Common Steps (everyone)

- Chris, extra steps

- Justin, extra steps

- George, extra steps

- Slavik, extra steps

Device

- Hardware

- Hardware Setup, Step by Step

- Software on the Pi

- Functional Requirements

Machine Learning

- Forecast Model

- ML Method (George's checklist, specific enough to hand to a Claude)

- Rounds Classification and Metrics (ml/models/nights.py)

- Functional Requirements

- LLM Narratives

Web App

- Family View

- My Irin (dashboards)

- Irin Buddy (Find a Buddy)

- Clinician Inbox (mock Ascend)

- Watcher Page (Night Buddy)

Interface Contracts

Claude Code Workflow

Standard Operating Procedures

- Everyone: Session Rituals

- George: Claude Drives, George Steers

- Slavik: Hands on Hardware, Claude on Diagnosis

Justin: Edit, Refresh, Screenshot

Chris: Merge Gate and Deploy

Demo Plan

- Core Three (always shown)
- Live Interactions (no files, performed against whatever is playing)
- Held in Reserve (Q&A ammunition)
- Run of Show (3-minute slot)
- Irin Rounds Run of Show (Impiricus)
- Night Buddy and Family Story Beats (Meta)
- The Meta Video (2 to 3 minutes)
- Sponsor Demo Logistics and Fallbacks

Deployment and Hosting

Hackathon Packing Checklist

- The Device
- Power
- Cables and Spares
- Tools and Consumables
- Computing and Demo
- Personal

Beyond the Hackathon

A Raspberry Pi bedside device for people with type 1 diabetes on injections. It displays live glucose, predicts lows 30 minutes before they happen, wakes the sleeper with light and sound, and delivers a plain-language morning report. Logging makes the system smarter over time.

The problem. CGM phone alarms fire when you are already low, they are easy to sleep through, and the surrounding apps give you graphs but no answers. Irin alarms before the low, wakes the whole room, and explains the night in plain language.

Terminology

Quick definitions for readers who do not live with type 1 diabetes.

Term	Meaning
Type 1 diabetes (T1D)	An autoimmune condition where the body makes no insulin. The person doses insulin manually, every day, to keep blood glucose in a safe range.
Blood glucose (mg/dL)	The amount of sugar in the blood. A common target range is roughly 70 to 180 mg/dL.
Low (hypoglycemia)	Blood glucose below about 70 mg/dL. Dangerous in the short term, especially during sleep, which is why overnight alarms matter.
High (hyperglycemia)	Blood glucose above the target range. Harmful over time and treated with insulin.
CGM	Continuous glucose monitor, a wearable sensor (here, a Dexcom) that reports a glucose reading about every 5 minutes.

Term	Meaning
Trend arrow	The CGM's indicator of which direction glucose is heading and how fast.
Time in range (TIR)	The percentage of readings inside the target range. The main day-to-day scorecard for glucose management.
MDI	Multiple daily injections. Managing T1D with insulin pens instead of a pump, which means no device automatically tracks doses.
Basal insulin	Long-acting insulin taken once or twice a day at a set time as a background dose. Forgetting it is a common and costly mistake.
Bolus (rapid-acting insulin)	Fast-acting insulin taken at meals to cover carbohydrates, dosed in units.
Carbs	Grams of carbohydrate in a meal, the number insulin doses are calculated from.
IOB (insulin on board)	An estimate of how much rapid-acting insulin is still active in the body from recent doses. Prevents stacking doses on top of each other.
Nightscout	An open-source, self-hosted service the diabetes community uses to store and serve CGM data. This project's system of record.
Treatment	Nightscout's term for any logged event: an insulin dose, a meal, exercise, or a note.
Rounds, Night Buddy, Family Story	The terms below belong to the sponsor entries; the two companion specifications define them in full.
Clinical Signal Card	The published card format Rounds sends to a doctor: a headline, the metrics behind it with a confidence label on each, the nights it drew from, a narrative written from those numbers, and the actions the doctor can take. One schema, two programs, versioned as v0.
Standing Cards	The always-on program for a patient who is not mid-titration: Basal Check (detect), Hypo Response (safety), Follow-up (verify). Fires on patterns, at most once per type per 14 days.
Step Watch	The therapy-start program: a scheduled watch from a new GLP-1 or weekly-basal prescription to a settled maintenance dose, with an early check, a check after each step, a gate before each step-up, and graduation.
Clean night	A night with no late meal, no late correction, no exercise, basal on time, no treated low, adequate sensor coverage, and the patient home. The only nights a basal conclusion can be drawn from; every other night carries a reason code.
Measured / reported / inferred	The confidence label every card metric carries: recorded by the device, answered by the patient, or derived from the shape of the glucose curve. A card never shows a blank row.
Inbox (mock Ascend)	The doctor-facing web page where cards arrive, styled as a screen inside Impiricus Ascend, served at doctor.irin-out-of-sleep-at-hackgt.tech; it talks only to the relay.
Relay / blind courier	The small server between the Pi and the inbox that plays the role of Impiricus. It stores sealed envelopes it cannot open and sees only sender, recipient, time, and size.
Pairing	The one-time handshake linking the device to one doctor or one buddy: QR, keys exchanged, the same four digits on both screens, PIN confirm on the

Term	Meaning
	device. Revocable by either side.
Near-miss	A predicted-low warning that was acknowledged and the actual low never happened, usually because the patient treated it. Invisible to every CGM report; Irin knows because it fired the warning.
Unfelt low / IAH	A low the patient did not notice, measured by asking the next morning about a specific event. Impaired awareness of hypoglycemia affects about 17.5% of adults with T1D, with roughly 4x the severe lows; the two standard questionnaires disagree most of the time.
Level 2 low / nocturnal burden	Glucose under 54 mg/dL, clinically significant regardless of symptoms; a red card on its own. Nocturnal burden is minutes under 70 in the night window plus the area under 70.
NightRecord / ledger	One summary row per night (coverage, reason codes, overnight rise, low point, burden, near-misses, level 2 lows, alarm events), built at the night-window end. The ledger is the table of these rows.
Morning recall	The next-morning question about one specific nocturnal low, with the curve attached: felt and treated, woke but felt nothing, don't remember, awake anyway. Unanswered by noon is no answer, never fine.
Narrative + validator	The two-to-three-sentence text on a card, message, or story, written by narrative.py's chain (Backboard → Meta → Claude → template); the validator requires every number in it to exist in the computed metrics, else a deterministic template renders instead.
Irin Brain / Irin Bedside / brain_only	Brain is the software (night classification, metrics, forecaster, both programs, narrative, delivery) and runs on any CGM feed; Bedside is this device, the confirmatory test. brain_only is the flag that makes Brain ignore presence, alarm hardware, and logged context, so a no-device patient's card can sit next to a full one.
Titration / GLP-1 / maintenance dose	Stepping a drug dose up until it settles, for GLP-1s about every 4 weeks, while insulin often has to be stepped down at the same time. The maintenance dose is where the steps stop.
Spark / Ascend / Wallet	Impiricus's products the cards ride on: Spark triggers engagement from real-world events such as a first prescription; Ascend is the two-way AI inbox; Wallet sends copay cards, hub forms, and education to a patient's phone.
Twin / mirror buddy	A buddy in your own time zone (shared rhythm, room-scale waking both ways with two devices) or 10 to 14 hours away, awake during your night.
Watcher / watchable	The two sides of the network: a watchable person can generate alerts, a watcher responds to them. Everyone with a CGM feed is watchable; watching is opt-in; only the watchable can be watchers.
The ladder / a rung	The ordered chain of responses to a low: warning, full alarm, escalation, re-arm. Night Buddy adds one final rung, another human, and the emergency script after it.
Hub / listing / claim / treating	The opt-in worldwide list of currently unacknowledged lows (a listing: first name, alarm state, elapsed minutes, confidence); a claim is a 3-minute exclusive lease so nobody piles on; treating is the patient's one-press "I'm handling it," which expires in 20 minutes.
Brokered contact / emergency script	Calls placed by the relay so no phone number crosses the network; the script is the patient's own pre-written instructions, shown only to the live claim-holder.
Family Story, Level 1 / Level 2	The plain-language morning story for consented family: Level 1 is the story only, with zero glucose values; Level 2 is the story plus the family view.

Term	Meaning
Irin Cloud	The patient-owned cloud service beside the relay: ingest, time series, dashboards, the family rollup, audio. It sees glucose; the relay never does.
My Irin	The app's second tab: eleven dashboards over the patient's history, drawn from Tiger Cloud continuous aggregates. Pictures, never card decisions.
Hypertable / continuous aggregate	Tiger Data's time-partitioned table and its self-refreshing rollup (a materialized query over time buckets), the two things every dashboard is built from.

What It Does

- **Live bedside display.** Glucose, trend arrow, 3-hour graph with target band, insulin on board, and time in range. Dim clock mode overnight.
- **Predictive low alarm.** An ML model forecasts glucose 30 minutes ahead. When the forecast crosses the low threshold, a WARNING wakes you gently: amber light, a softer distinct tone, patient ramp. An actual low is the FULL alarm: red, loud, escalating, re-arming if you stay low. The two are deliberately different so the sound itself tells you whether you have 30 minutes or none, and acknowledging the warning never silences the real alarm. Also alarms on highs and on stale data from a sensor or connection failure.
- **Morning report.** Overnight high and low with times, time in range, and alarm events, plus a two-paragraph plain-language summary written by an LLM from your stats and logs. Delivered by email and kept in a report history on the phone.
- **The app.** One hosted app, four tabs, one code to get in. Irin Device: live view, alarm acknowledge, all settings, logging (basal taken, meals as carbs plus units, exercise, notes), voice logging through the browser ("log 45 carbs and 5 units") with every insulin entry echoed back, in an ElevenLabs voice, and confirmed before saving, and the demo panel. My Irin: your history as dashboards. Irin Buddy: find a buddy and the watch. Irin Rounds: share with your doctor. The kiosk on the nightstand stays a glance-and-alarm screen with no forms.
- **My Irin.** Eleven dashboards over every night the device has seen, including 22 months of history: a nights strip colored by reason, time-in-range trend, the overnight profile endocrinologists read (median and percentile bands by time of day), where your lows live (hour by weekday), alarm response, near-misses, basal timing, sensor health, the Step Watch view during a watch, buddy nights covered, and an under-the-hood tile. Pictures, never decisions: cards are computed elsewhere.
- **Context-aware alarms.** Logged events adjust alarm behavior through transparent rules. A recent meal softens a predicted low, insulin without carbs raises sensitivity, evening exercise tightens overnight vigilance, and a missed basal gets flagged.

One System, Three Audiences (Sponsor Challenges)

Irin enters main judging and two sponsor challenges as ONE project. The bedside device is the spine; the two sponsor entries are doors into the same house, never separate products. Everything hangs off the same machinery: the forecast, the alarm ladder, presence, the relay, and the narrative with its no-invented-numbers validator. The companion specifications (docs/Irin_Rounds.pdf and

docs/Irin_Night_Buddy.pdf) are the source of truth for each entry; this section is the map, the Rounds, Night Buddy, and Family Story blocks in every later section (How It All Works, Device, Machine Learning, Web App, Interface Contracts, Demo Plan) carry the mechanics, and the per-owner plans in docs/plans/ carry the build order, interfaces, exact numbers, and named checks for every tier.

- **Irin (main judging).** The device and The Save: everything else in this document.
- **Irin Rounds (Impiricus challenge, “Invent the Next Way We Engage HCPs”).** What happens at home becomes short, decision-ready Clinical Signal Cards for the patient's own doctor, two programs on one engine. Standing Cards watch an ordinary patient month after month: Basal Check (the overnight rise on clean nights only, with every confounded night set aside and its reason listed), Hypo Response (alarms that had to escalate, re-arms, response times confirmed by the radar, lows the patient did not remember, glucagon status), and Follow-up (before and after a confirmed dose change). Step Watch watches the weeks after a GLP-1 or weekly-basal start step by step: an early check, a check after each step, a gate before each step-up with one decision, proceed or hold, tolerance as a first-class signal, and graduation. Every number is computed by code and carries a confidence label, measured, reported, or inferred; cards never carry a dose. They are encrypted on the device to the paired doctor's key, carried by a blind relay (relay/, “mock Impiricus”, a second small FastAPI service on Vultr with MongoDB Atlas behind it that stores ciphertext only), and read in an Ascend-styled clinician inbox (doctor.irin-out-of-sleep-at-hackgt.tech). A doctor's reply travels back authenticated and never applies until the patient echo-and-confirms on the device. Irin Brain (the software) runs on any CGM feed without hardware and is the screen; Irin Bedside (this device) is the confirmatory test. The 2026-09-25 Rounds spec supersedes the earlier two-card brief.
- **Night Buddy + Family Story (Meta challenge, “Bringing People Closer Together with AI”).** The guardian connects the people around the patient. Night Buddy pairs T1D adults as mutual overnight backstops, a twin in your own time zone or a mirror 10 to 14 hours away who is awake during your night, and adds one final HUMAN rung to the alarm ladder: if the full alarm goes unacknowledged with the radar confirming someone is in the room, the buddy is alerted to make a loud noise happen; at scale, an opt-in community hub lets verified T1D volunteers claim a listing and respond. It rides the Rounds relay, pairing, crypto, and narrative machinery, and, decision 29, Meta's own pieces: Muse Spark (the Meta Model API) writes the Buddy narratives and the Family Story behind the same validator, and the WhatsApp Cloud API is the second alert channel beside the watcher page; the matching score stays deterministic. Family Story, the entry's BUILT half, sends consented family members a plain-language story of the night instead of raw data, at a disclosure level the patient controls (Level 1 story only, Level 2 story plus the family view), over the morning report's existing email path, with approve-each-morning or automatic sending, one-tap pause and revoke, and a validator that bans glucose values from story-only emails; full definition in docs/FAMILY_STORY.md. The Meta deliverables are a working prototype, a 2-3 minute video (Slavik), a public repository (a fresh mirror built on submission day; the working repo stays private), and a short write-up (docs/META_WRITEUP.md, a team effort).

Build sequencing (binding). 1) Core Irin end to end, the freeze rule: nothing else starts until The Save plays with both alarm tiers, escalation, and acknowledge; then Family Story (about half a day of Chris, two hours of Justin), because it rides the morning report and is the Meta entry's built half. 2) Rounds, the platform and Standing Cards (R1-R9): contracts, the night ledger with reason codes, the recorder, low events; delivery (crypto, pairing, relay, inbox shell); Basal Check, Hypo Response, Follow-up with the noise budget; the doctor-message path proven once on a basal change. Standing Cards run on Chris's real history around real basal increases, said exactly this way and never unqualified: “glucose real; reason codes inferred and labeled; acknowledge, presence, and recall data are a labeled overlay”. 3)

Rounds, Step Watch and recall (R10-R13): the plan and its windows, the status rules, gate and check cards, the Spark offer with the plan setup form, the stomach check-in and weekly injection logs, the morning recall questions and the unfelt-low rate, the replay seek with idempotent catch-up and the synthetic titration scenario, the resources handoff, the “what Impiricus sees” panel, and the narrative validator; the minimum credible Rounds demo is one Standing Card and one Step Watch card on the same patient with a real doctor action and a real patient confirm. 4) The Night Buddy demo tier, only if R1-R11 landed with a full day remaining before judging; otherwise Night Buddy is video-only, no debate at 3 AM. 5) Rounds polish (R14): step-week vigilance, the night replay, Brain versus Bedside side by side. Cut order if time runs short, first cut first: Night Buddy code, then the Rounds cut order, hold the line: night replay, step-week vigilance, Brain versus Bedside side by side, the Follow-up card, real end-to-end crypto (plain HTTPS, honestly labeled, panel kept), QR pairing (pre-pair), Basal Check’s “possibly too high” direction. Never cut: the confirm step, the DEMO badges, the “no patient data shared with any manufacturer” banner, and the noise budget. Where the integrations sit: the domain, Vultr, Atlas, and the compose file are day-one infrastructure (Chris, hour one, they replace work); the forwarder and Irin Cloud’s ingest land with the core (C1); the aggregates and the My Irin tab (C2-C4) run in George’s and Justin’s lanes in parallel with Rounds; ElevenLabs lands with the buddy alert (B4) and the spoken echoes (core step 11, an hour); Backboard lands at R13 behind the validator. Sponsor cut order, first cut first: the My Irin dashboards (C3-C5, the aggregates still stand for metrics.md), Backboard (the direct Claude call remains), the device’s spoken echoes (the buddy alert voice stays). Never cut: the hosting and Atlas swaps, because they replace work already planned.

What it adds to the repo. relay/ (Chris), frontend/clinician/ and frontend/watch/ (Justin), backend/app/rounds/ (ledger, alarm_events, nights_adapter, low_events, recall, cards, standing, step_watch, narrative, noise, crypto, pairing, relay_client, messages, vigilance), backend/app/buddy/, and backend/app/family_story.py (Chris), ml/models/nights.py (George: the one shared night-classification and metrics module the backend imports) and ml/evaluate_rounds.py (the three measured numbers), a companion JSON file next to each demo scenario, a seek in the replay engine, sixteen more safety invariants in CLAUDE.md (twenty-two in all), and three more run-of-shows in demo/DEMO_SCRIPT.md. Decisions the plans now assume, all recorded in docs/OPEN_ITEMS.md: the AlarmEvent recorder samples the RAW radar every 30 s for presence_during and aggregates it by one rule (a majority of samples, or any detection within 2 minutes of alarm start, means home; zero detections means away; anything else is unknown), because the night rule pins the Home/Away state to Home overnight and makes it useless as 3 AM evidence; the phone app carries all input and the kiosk carries no forms, with one cached-PIN verb, the big on-screen acknowledge during an alarm, while pairing confirm and doctor-message confirm re-prompt the keypad and the phone app keeps its PIN in sessionStorage; alarm.py exposes a single on_transition observer hook from day one so no later tier ever edits it; every card metric carries a confidence label and no row is ever blank; and every sponsor model is in contracts.py from the day-one skeleton. The six sponsor integrations replace planned pieces rather than add products: Vultr hosts everything Irin owns in the cloud (in place of Railway for the relay and Vercel for the pages), MongoDB Atlas holds the relay’s documents and Nightscout’s database (in place of SQLite on a volume and Railway’s Mongo), Tiger Cloud is the time series behind the new My Irin tab and the family rollup (in place of the family page reading Nightscout directly), ElevenLabs is the voice (in place of browser speech and a wav chime for the buddy alert), Backboard is the narrative backend with memory and routing (in place of the direct Claude call, which stays as the fallback), and the .tech domain gives every name real HTTPS, which the fallback chain needed anyway.

Safety Posture

Irin is a companion to CGM alarms, not a replacement and not a medical device.

Stale data is always shown as stale and never forecast on. Demo data is always badged DEMO and never rendered as if live. Insulin is never stored without echo and confirm. Low alarms override quiet settings. Rounds and Night Buddy extend this posture and never weaken it: cards never contain dose recommendations, every number on them is computed by code and carries a confidence label, and no conclusion is drawn from stale nights or windows under 70% coverage; Irin never changes a prescribed plan or dose by itself, silence means the plan stands, and a doctor's message never applies without patient confirm; patient data leaves the device only encrypted to the paired doctor; the buddy and hub rungs are additive only and never delay or quiet a local alarm; demo cards, alerts, and listings are always badged DEMO and never reach a real inbox, hub, or phone; a story-only Family Story email never contains a glucose value, and a demo-mode story is never emailed to anyone. Irin Cloud and My Irin extend it once more: the relay never holds a glucose value and the time series lives in a separate, patient-owned service; dashboards never decide a card; a model's words always pass the validator, whichever link of the chain wrote them; neither a WhatsApp message nor a Muse prompt ever carries a glucose value; and the voice alert and the WhatsApp message are additive rungs that can never delay or quiet a local alarm.

A Day With Irin (Five Perspectives)

What the device is like to live with, start to finish, no technical terms, told from every chair at the table: the person who sleeps next to it, the doctor who never sees the bedroom, the stranger who is awake when you are not, the mother two states away, and the company that carries the envelopes. This is also the story the pitch tells.

The sleeper (you)

Evening. The device sits on your nightstand looking like a small smart display. The frame around the screen glows a soft, steady color: green-teal means you're in range, and most of the time that's all you ever need from it. Walk past, see calm light, keep living. If you want detail, the screen already shows it: your number, the trend arrow, the last three hours as a graph, how much insulin is still working in you, and today's time in range.

Basal time. Around your usual long-acting insulin time, a quiet nudge appears on the screen and on your phone: basal not logged yet. Take your pen, then tap "basal taken" on your phone, or just tell it. If you forget, the nudge gets more insistent after an hour, and after ninety minutes it emails you. It never assumes; it only reminds.

Bedtime. At your configured hour the screen dims to a clock with your number small beneath it. No graphs glowing at your face, no light show. The device is at its quietest exactly when you are.

2:47 AM. Your glucose has been drifting down for an hour. You don't know that, because you're asleep, and your phone's alarm won't fire until you're actually low. But the device has been forecasting 30 minutes ahead all night, and the forecast just crossed your low line. The frame ramps from nothing to warm amber, a soft distinct tone grows patiently, and you wake before the low arrives, to a warning, not an emergency, because you still have time. The screen says PREDICTED LOW, 74 and falling. You grab

the juice from your nightstand, tap Acknowledge on your phone, and the room goes quiet, done: a warning acknowledged stays acknowledged. But the device hasn't lowered its guard; if the low arrives anyway, the real alarm, bright red and loud, fires fresh, and acknowledging that one only buys quiet, not safety: still low fifteen minutes later, it refuses to stay silent and alarms again. And if you'd slept through the amber entirely, it would have climbed to the full red alarm on its own within five minutes, strobing and getting louder until someone dealt with it.

Morning. When the night window ends, the screen greets you with the night in one view: your overnight low and high with times, how long you spent below range, and your time-in-range rings starting fresh for today. Your inbox has the same story written in plain language: two short paragraphs, like a friend who watched over the night, noting the 2:47 dip, that you treated it, and that the rest of the night was steady. Every report is also saved in the phone app, so your last few weeks of nights are always a tap away.

Breakfast. You're dosing for eggs and toast. Instead of opening an app and tapping through menus, you say: "log 30 carbs and 4 units." It answers back, "30 carbs and 4 units, save?", because insulin is never recorded on a maybe. You say yes. The display's insulin-on-board figure updates, and for the next four hours the device knows there's insulin working in you, which makes its predictions and alarms smarter.

Afternoon. Lunch runs high and the frame shifts to amber with a gentler alert; highs matter, but they're not a 2 AM emergency and the device treats them with proportion. Later you log a run. The device quietly tightens its overnight vigilance, because it knows evening exercise is how 2 AM lows happen. And if you eat dinner and it sees you trending down afterward, it doesn't cry wolf: carbs are on board, so the would-be alarm softens instead of waking the house.

Away from the nightstand. A tiny radar built into the device senses whether anyone is in the room, and it's sensitive enough to detect a sleeper lying perfectly still by the rise and fall of their breathing. When it's confident the room is empty for a sustained stretch, the device goes politely dormant: no lights, no sounds, just a dim clock, because alarming an empty bedroom helps nobody. Coming back is the opposite of cautious: walk in and it's on duty again instantly. A Home/Away toggle in the phone app is the manual fallback, and it always wins over the radar; the radar alone can never silence the device during the night window. Nothing stops being tracked while you're out, and the phone app works from anywhere: your number, the graph, logging, and the acknowledge button ride along in your pocket. And the people who worry about you have their own window: a simple page your mom can open from another state showing your current number, the trend, and how last night went. She checks it with her morning coffee instead of texting "you up?"

When things break. If the sensor dies or the connection drops, the device says so instead of pretending: the display marks the data as stale and raises its own alert, because a glucose display showing a frozen number is more dangerous than no display at all. And through all of it, your phone's own CGM alarms keep running untouched; Irin is a second guardian, never a replacement for the first.

Your doctor (Irin Rounds)

Pairing, once. At a visit, Dr. Patel asks whether Irin can tell her how your nights actually go. You tap Share with my doctor in the phone app, the device shows a square code, she scans it with her own phone, both screens show the same four digits, you type your PIN on the device, and that is the whole ceremony. In production the invitation reaches you from Ascend, the way Impiricus already sends things to a patient's phone; the square code is the hackathon's version of that handshake. From now on, what happens at home can travel to her, sealed so that only she can open it, and you can stop it with one tap.

The app shows you every card before and after it goes, so nothing leaves the nightstand that you have not seen.

Ordinary nights. You log your basal when you take it, carbs when you eat, and the device does the rest. Some nights you eat late. Some nights you correct at bedtime. Some nights you go for a run at seven. Irin notices all of it and quietly sorts your nights into the ones that can say something about your basal dose and the ones that cannot: a late snack, a bedtime correction, a rebound after a treated low, a basal taken an hour late, a night the sensor was patchy. Those nights are set aside, with the reason written down. What is left is a clean night, and only clean nights are allowed to say anything about your dose.

2:47 AM, a save, and the morning question. The forecast crosses your low line while you are asleep. Amber light, a soft tone, and you wake before the low instead of during it. Juice, acknowledge, back to sleep. In the morning the phone app shows one card for it, with the curve attached: at 2:47 AM you were 58 for about 25 minutes. Do you remember that? Four answers: I felt it and treated it; I woke up but felt nothing; I don't remember it; I was awake anyway. If you logged juice within half an hour of the bottom, it already knows you treated it and asks only whether you felt it. Unanswered by noon is recorded as no answer, never as fine. The nightstand screen stays glanceable and shows only a small chip, two questions waiting in the app. This is the clinical standard for awareness of lows, self-report, except that today it is asked once, in a clinic, about a vague six-month impression, and the two standard questionnaires disagree about the same patient nearly 60% of the time. Irin asks about one night, the next morning, with the glucose curve already in hand.

Two weeks in, a card she can act on. You never see this part. Nothing arrives for most of those two weeks, and that is the design: quiet nights make no cards. Then an email says only that a card is waiting. She opens Ascend, the Impiricus inbox where she already requests samples and resources, and a Basal Check unlocks on her screen as a new kind of card among the ones she already gets: a strip of fourteen nights colored by reason, eight clean, two late snacks, one rebound, one late basal, two with poor sensor coverage; a chart of the overnight rise on the clean nights only; and the number, a median rise of 42 on your eight clean nights, six of eight in the same direction. She does not have to decide whether a dip was treated or a spike was dinner, because the card already excluded those nights and told her why. The same rising line can mean four different things, a basal that is too low, a basal taken late, a dinner under-covered, or a rebound from a low that means the basal is too high, and those need opposite prescriptions. Thirty seconds, two units, next patient.

The confirm. She types the new basal into the card and taps send. Your phone and your device both ask you to confirm it, the units spoken out loud on the device, the same rule as when you log insulin by voice, and you do. Nothing in Irin ever changes a dose by itself: had she done nothing, nothing would have changed, and had you declined, nothing would have changed either. Her inbox reads Patient confirmed 07:14.

Day 7 and day 14. A Follow-up card arrives, comparing your clean nights before and after: the rise went from 42 to 8. She does nothing, which is the correct action. On a patient with fewer clean nights it would have said not enough data yet instead of guessing.

The near-misses, and the safety card. On another card the inbox says something no CGM report can: three predicted lows were caught and treated before they happened. Your time below range looks perfect precisely because the device keeps working, so your basal may be too high, and she would never have seen it in Clarity. The better Irin works, the cleaner your chart looks; the card is the only place the near-misses exist. And when a fortnight holds two warnings that had to climb from amber to red to

wake you, or a low you did not remember, a Hypo Response card arrives with your response times, counted only on nights the radar knew you were in the room, and one line she can act on immediately: no glucagon on file. She taps Access and Resources, picks glucagon as a category, then a brand, and Ascend does what Ascend already does, with a banner that says no patient data was shared with any manufacturer. It is true; the courier only ever held ciphertext. The card never suggested a dose. It never will; cards show data, doctors type numbers.

Without the device. Nobody has to own the device for the cards to exist. With only the phone app and a CGM, Irin still sorts your nights (a late meal shows up as a fast rise in the first two hours, a treated low as a climb after the dip; what it cannot see, it marks as context unavailable rather than assuming clean), still finds your near-misses from the phone warnings, still asks the morning question, and still sends Dr. Patel a Basal Check and a number no questionnaire produces: four nocturnal lows in 14 nights, three answered, two unfelt, one no answer. Nothing on that card is blank. Every number carries how sure Irin is: measured by the device, reported by you, or inferred from the curve. The app tells her whether you knew. The device tells her whether you woke, when, and that you were there. Brain is the screen; Bedside is the confirmatory test, the way an ECG leads to a Holter monitor, and during a titration the device can be a clinic loaner you hand back at the end.

A second patient (Step Watch)

Maria. Maria is 58, has had type 2 diabetes for eleven years, and takes insulin. In October her primary care doctor starts her on a weekly GLP-1 shot that steps up in dose every four weeks. The guideline says her insulin needs will drop by a quarter to a third and should be reassessed within days of each step. Her next visit is in three months. Between visits nobody is watching, which is how people either go low or quit the drug.

The offer. When the prescription is written, Spark, the Impiricus engine that notices a first prescription in claims data, drops a small card into Dr. Alvarez's Ascend inbox: new GLP-1 start in your panel, offer Step Watch? Thirty seconds of setup: the drug and its schedule pre-filled from the label, the start date, whether Maria is on insulin, and three options, ketone prompts, a little extra caution around each step week, a slower climb. The plan reaches Maria's phone and she confirms it on the screen, and the watch is on. Her Basal Check goes quiet for the duration, because during a titration a clean night would be measuring the drug, not the dose. In production the invitation reaches her through Wallet, the same way Impiricus sends her a copay card.

The first weeks. Every morning the phone app asks how her stomach is: fine, rough, can't eat, one tap. Once a week she says I took my shot and confirms what it repeats back. On day 3 Dr. Alvarez gets a card saying nothing has moved yet, labeled early and limited, and does nothing. Three days before each planned increase a gate card asks one question, proceed or hold, and if she says nothing the plan stands; Irin never blocks a prescription.

Step two. At four weeks Maria goes to 5 mg. Food stops being interesting, she eats less, and the insulin dose that was right last month is now slightly too much. She is low at 3 AM twice that week, and both mornings she answers that she does not remember it. Days three through seven after the step produce an amber card: overnight low point down 22 from her baseline, two near-misses, four percent of the night below range, one low not remembered, rough stomach three days out of five, every number labeled measured, reported, or inferred. Dr. Alvarez sends back two things: a smaller basal dose, and the next increase pushed out four weeks. Both surfaces ask Maria to confirm, and she does; the inbox shows Patient confirmed, and the timeline on her screen slides, 7.5 mg moving from October to November. A

level 2 low, a re-armed alarm, or two hours above 200 on a day she could not eat would not have waited for a window at all; those go red immediately. On the resources page Dr. Alvarez pulls stomach-side-effect education and a copay card, category first, brand second, with the same banner about patient data. Samples of the next pen strength would come through the QPharma integration, bridge supply if her coverage lapses through Medvantx, and a prior-authorization fight goes through hub enrollment.

What did not happen, and graduation. Maria did not quit. She did not have a severe low at 3 AM. She did not spend three months deciding on her own that the drug was making her sick. She reaches the dose that works and holds it for four quiet weeks, and the watch ends with a summary her doctor reads before the next visit, baseline against now. Her Basal Check picks up again, with the post-graduation nights as its new baseline. Every hold is a patient who did not quit.

Your buddy (Night Buddy)

Tom. You live alone. The safety nets everyone assumes, a partner who hears the alarm, a parent down the hall, are not in the apartment. So you opt in, and Irin matches you with Tom in Melbourne, fourteen hours ahead: your 3 AM is his 5 PM. He has type 1 too. You never exchange numbers or addresses; you exchange first names, and most days the entire relationship is one line each morning: all quiet for both of you last night.

The night it matters. Months later the amber warning fires and you do not stir. The low arrives and the room goes red and loud, and still nothing. Five minutes on, the frame strobes at full volume. Ten minutes on, with the radar sure someone is in the bed, the device does the one thing it could not do before: it asks another human. Tom's phone lights up in daylight and speaks, in a calm voice, on a loop until he answers: your buddy is in trouble, alarm unacknowledged twelve minutes, presence confirmed. Your first name, no glucose number, no map. He taps I've got this, which gives him three minutes in which nobody else will pile on, and taps Call. The relay connects the call without showing either number; your phone rings, and your device adds a chime it uses for nothing else. You surface, treat, and tap acknowledge. His screen shows your line recovering. Had he not reached you by the twenty-minute mark, your own emergency script, the one you wrote when you joined, would have fired regardless: call my sister, she has a key. Nothing Tom does can delay that clock. A buddy is an extra chance, never a postponement.

The morning after. Tom's phone reads: he's okay. Your call at 3:12 woke him; he treated and recovered. Yours says the same from the other side. The next morning it is back to all quiet.

The hub. Say Tom's phone had been in a drawer. If you also opted into the community watch list, your listing appears to verified volunteers around the world, every one of them a person with type 1 who has fumbled for juice at 2 AM: first name, alarm unacknowledged, presence confirmed, ordered by urgency. A volunteer in Lisbon claims it and makes the same call. And if you were in fact awake and treating, one giant button on your phone, I'm treating, clears the listing so everyone can stand down; if it expires twenty minutes later and your glucose has not recovered, the listing comes back at the top of the list, because a person who said they were handling it and is still low is the strongest signal the hub can produce. A wasted call is thirty seconds of thanks for checking. A missed emergency is the thing everyone here is afraid of.

Without a device, on either side. Tom never needed one: a phone and the watcher page make him a full member, buddy or hub volunteer, as long as a CGM feed stands behind his account, since only the watchable can be watchers. And you would not need one to be watched: the same software that runs on the nightstand runs in the cloud with the hardware turned off, watching your CGM feed, warning

your phone, and posting your listing marked unconfirmed, because without the radar “not answering” and “not there” look the same. The device is never the ticket in. It is what makes your cry for help unmistakable.

Your mom (Family Story)

Same night, two ways. Your mom is two states away. Before Irin, the only way to reassure her was to hand her your entire glucose stream, forever, so she could watch a graph and worry. Now she is a recipient, story only, and each morning an email arrives with a twenty-second voice note and the same words in plain language: a long low around 3 AM, treated, back in range by four; the rest of the night was steady. No numbers, ever, on that setting; the device checks every story for a stray glucose value before it goes out and falls back to a plain template if one slips in. On a night the sensor was off, the story says Irin did not have data last night, never all fine.

Who is in control. The first story to any new recipient waits for your approval; after that they are automatic, unless you prefer to approve each one. Your morning screen shows a chip, Sent to Mom, with pause and remove one tap away, and she can unsubscribe from any email. Her page with the live number still exists if you choose to give her that level; Family Story sits in front of it, it does not replace it.

Impiricus (the blind courier)

What they carry. Every card to Dr. Patel, every hold from Dr. Alvarez, and every alert to Tom travels through Impiricus’s network, and Impiricus cannot read any of it. Each is sealed on the nightstand to one person’s key and opened only in that person’s browser. What the courier sees is a sender, a recipient, a time, and the size of the envelope. On the demo table a panel shows exactly that view.

The moment. Impiricus’s own founder says physicians should be able to reach critical resources for a patient the moment they are needed, without fragmented systems or delayed follow-up. Everything the platform ships today assumes the doctor is already in the conversation, because the moment is a visit, a rep interaction, or a claims trigger that arrives weeks later. Nothing on it can tell a physician when the moment has arrived for a patient at home. Rounds supplies the moment. A fortnight of clean nights rising 42 mg/dL is a moment. Night 4 after a dose increase, when Maria was low for 25 minutes and did not know it, is a moment. Ascend already handles everything that comes after it.

What it rides on. Nothing in the story above asked Impiricus to build a new product; every beat rode something they already run. Spark, the engine that notices a first prescription in claims data, raised the Step Watch offer in Dr. Alvarez’s inbox. The inbox itself is Ascend, their two-way platform where doctors already request samples, dosing tools, and resources; a Clinical Signal Card is a new kind of card inside it, not a new app. Dr. Patel’s identity, and the key her browser holds, are trusted because she is one of the million verified physicians on their network. The invitation that reached Maria’s phone, and the stomach-side-effect education she was handed, travel through Wallet, the tool Impiricus built for sending things to a patient’s phone. Samples of the next pen strength come through the QPharma integration, bridge supply if her coverage lapses through Medvantx, prior authorization through hub enrollment, and a clinical question to a field rep or MSL. Whether watched patients stay on therapy longer would be measured on the claims data pharma already buys through Impiricus, a hypothesis they would test, never a result we claim. What Rounds never does is deliberate too: no text messages, nothing that recreates Pulse or ION, no dose numbers that would look like their dosing calculators, no existing feature rebuilt. It is the first channel on the platform that starts from the doctor’s own patient

instead of from a drug company, and it takes Impiricus's own promise, sensitive data broken into pieces nobody can identify, to its limit: a courier that carries only envelopes it cannot open. Irin is the first device to publish into that channel, not the only one that could.

What they gain. A new kind of card inside the inbox their million doctors already use; the first patient-originated engagement channel in the industry; a recurring, non-promotional reason for a doctor to open the platform between rep visits, thirteen touchpoints across a 24-week tirzepatide watch and up to one Standing Card per type per fortnight after that; two new pharma moments, the glucagon prescription that only 8.7% of short-acting insulin users ever fill and the persistence of a drug more than half of patients quit within a year; and a second kind of customer, the device makers, whose data becomes decisions pushed into a channel doctors already use, including the digitally dark third no portal reaches. The drug company behind Maria's shot never sees a glucose value. It sees what it already buys: a doctor requested stomach-side-effect education and a copay card, at the moment she was deciding whether to hold the next step. Cards never carry a dose, Irin never changes a plan by itself, and a doctor's message never applies until the patient confirms it at the bedside.

One device, five chairs. The sleeper gets thirty minutes of warning and a room that wakes. The doctor gets the nights sorted, the near-misses nobody else can see, and the one instrument that measures whether a patient feels their lows. The buddy gets a promise with real stakes that costs five seconds a day. The mother gets the story instead of the data. The courier gets a channel it cannot read. It is one system, and these are its audiences.

How It All Works

Normal operation Dexcom sensor → Dexcom Share → Nightscout (Railway) → Pi poller (60 s) → SQLite cache → forecast model → alarm engine evaluates actual value, prediction, staleness, and context rules → WebSocket pushes state to the kiosk display and any open web apps → LED and sound controllers react to state changes.

Presence LD2410B-P radar on GPIO17 → presence reader with debounce → presence state (slow and skeptical to Away, instant to Home; the manual app toggle always wins; radar absence never sets Away during the night window) → output gates dim the display and silence LEDs and speaker. Alarm logic, logging, forecasting, and the phone path are never gated.

Logging Phone (touch or voice) → web app → backend parser and validator → SQLite, posted to Nightscout as a treatment → IOB recalculated → display updates.

Morning Night window ends → stats computed from SQLite → matplotlib graph PNG → narrative summary (narrative.py, the morning_report task) → email sent and report stored → morning screen shown on the device.

Rounds, the card pipeline (Impiricus) Alarm engine transitions → AlarmEvent recorder, an observer on alarm.py's on_transition hook, writes one event per episode (tier, start, acknowledge time and surface, escalated, re-arm count, crossed_actual, presence_during from the raw radar sampled every 30 s) → night window ends → the ledger writes the NightRecord: coverage, reason codes (late meal, late correction, basal late or missed, exercise, treated low, stale, away, or clean, each marked logged or inferred), overnight rise, low point, nocturnal burden, near-misses, level 2 lows; every number from ml/models/nights.py → each nocturnal low becomes a LowEvent with its nadir, duration, recovery slope, and inferred-unfelt flag → in the morning the phone app asks about it (at most two cards; carbs

within 30 minutes of the nadir pre-fill “treated”; unanswered by noon is no answer, never fine) → 07:05 the programs evaluate as pure functions: Standing Cards check their pattern rules over the last 14 nights, Step Watch checks whichever window just closed; most mornings nothing fires, by design → the noise budget decides what actually sends (one Standing Card per type per 14 days, one check and one gate per step, greens to a weekly digest, red bypasses everything, a watch suspends Basal Check and absorbs Hypo Response) → numbers from code, two to three sentences from narrative.py (the clinician_card task: Backboard → Meta → Claude → template), a validator that rejects any number not in the metrics and falls back to a template → sealed to the paired doctor’s public key (crypto_box) → POST to the relay, outbound only → the relay stores ciphertext and emails “a card is waiting” with zero patient data → the doctor’s browser fetches with its bearer and decrypts with the private key it holds.

Rounds, the reply Doctor taps adjust basal / hold / proceed / adjust insulin / message / schedule / end watch, or Accept on a Spark offer → the inbox seals a DoctorMessage to the device’s key → the Pi polls the relay (60 s live, 5 s demo) → the message opens only with the paired doctor’s key, which is what authenticates it → confirm takeover on the display and phone, an insulin change echoing its units out loud → Confirm (fresh PIN) applies it: plan_create activates the watch, hold_step shifts every later step by the hold, insulin_change updates the basal setting and logs a therapy change and starts a Follow-up; Decline or 24-hour expiry applies nothing; silence changes nothing → the resolution word goes back to the relay → the inbox shows Patient confirmed 07:14. Access and Resources ends in POST /v0/resources/request {doctor, category, brand}, a schema with no patient field, the only thing a pharma-side system would ever see.

Step Watch, the lifecycle Spark notices a first prescription (in the demo, a simulated trigger) → Ascend shows “New GLP-1 start. Offer Step Watch?” → 30 seconds of plan setup: drug and schedule from the label, editable; start date; on insulin; ketone prompts, step-week vigilance, slower escalation → pairing if not already paired → the plan arrives as an authenticated message and the patient confirms it → baseline = the 14 nights before the start (fewer than 5 usable is “baseline thin”) → early check on day 3 (nights 1 to 3, labeled limited) → step check on day 7 after each step (days 3 to 7) → step gate 3 days before each planned step-up: proceed or hold; silence means the plan stands → a level 2 low, a re-armed alarm, or a ketone-risk episode on a can’t-eat day goes red immediately → graduation after four green weeks at the maintenance dose, with a final card that doubles as visit prep → Standing Cards resume with the post-graduation nights as their baseline. Tirzepatide: 2.5 → 15 mg in five steps, 15 mg at week 20, graduation at week 24; Ozempic graduates around week 16.

Pairing (shared by Rounds and Night Buddy) “Share with my doctor” (PIN) → the device mints a single-use 128-bit token with a 10-minute expiry → the QR encodes the inbox URL with the token and device public key in the URL fragment, which browsers never send to a server → the doctor’s browser generates its keypair with tweetnacl-js, keeps the private key, posts its public key → both screens show the same four digits, computed on each side from device key || doctor key || token, so nobody, including the relay, can swap a key → the patient confirms on the device with a fresh PIN → the relay issues the doctor’s bearer. Revoke from either side deletes keys on both sides. If the QR will not scan at the table, the demo phone is pre-paired; in production the Impiricus-signed doctor directory is the trust anchor, invites come from Ascend including telehealth short codes, and passkeys protect private keys. A buddy pairs the same way, the watcher page being the browser peer.

Pairing your Irin Tap Pair your Irin in the app → the kiosk shows a QR and a 6-digit code (single use, 10 minutes, shown only while the radar sees someone in the room) and the Pi registers it with the relay: POST /v0/device/pairings (X-Source-Key) {code, device_id, device_url, token, expires_at}, device_url

being its DEVICE_URL (<https://device.irin-out-of-sleep-at-hackgt.tech> on the Pi, <http://localhost:8000> on a laptop) → the app scans the QR (it encodes the same code plus the app URL, so scanning equals typing) or you type the code → the app redeems it: POST /v0/device/pair {code, username} → {device_id, device_url, token}; the relay writes a DevicePairing document (kind owner) and marks the code used → the app stores device_url and token → the Pi learns the completion (your username) on its next relay poll (relay_client.py, the same poll as doctor messages) and stores the DevicePairing locally → every Device-tab call goes to the Pi with that token and the PIN → the device can post listings and alerts under your account. Unpair: DELETE /v0/device/pair with the token from the app, or from the kiosk settings on the Pi; both sides revoke on the next poll. DevicePairing is its own model; Pairing.peer_kind stays doctor | buddy. No Bluetooth: Safari on iPhone has no Web Bluetooth, and the phone needs the network path anyway.

My Irin, the dashboards Pi poller → SQLite (local, unchanged) → the forwarder POSTs the last five minutes of readings, alarm events, low events, and treatments to Irin Cloud, outbound only → Tiger Cloud hypertables → continuous aggregates (1-day stats, 30-minute overnight profile, nightly rollup with a 22:00 origin, hourly heatmap) refresh on their own → GET /v1/dash/* with the owner bearer → the My Irin tab draws. Family Story Level 2 and the family page read one rollup (last night: low, high, minutes under 70) through a family bearer that can be revoked. Cards never read from here.

Find a Buddy Opt in to a personal buddy → pick someone you know by username, or be matched → the globe: spin it, tap the political timezones you want your buddy in (each shows its live clock), as many as you like → the languages you speak → your availability, with 22:00 to 08:00 locked as asleep → the emergency contact (a number nobody will ever see; the relay places the call) and the instructions → the profile lands in the relay's directory → one aggregation pipeline scores candidates: language overlap, timezone in your set, how much of your sleep window their availability covers, a bonus for a mirror 10 to 14 hours away → the top three, either side can decline (declined never return) → pairing (keys exchanged, four digits) → the introduction line and the why-this-match line, written by Muse Spark through narrative.py from the score parts, first names only; the score itself is deterministic and an LLM never picks the buddy.

Night Buddy, the ladder (Meta) T-30: forecast crosses, amber warning (a near-miss if acknowledged; this rung is why the human rungs rarely fire) → T: actual low, full alarm → T+5: strobe and maximum volume → T+10: still unacknowledged AND the recorder's presence verdict is home (in the room but unresponsive; an empty room means the bathroom) → the buddy rung, another observer on the same hook, seals a BuddyAlert to the buddy's key and posts it, and opens a hub listing if opted in → the watcher page shows first name, elapsed minutes, "presence confirmed"; no glucose, no location → I've got this = a 3-minute exclusive lease → Call = brokered by the relay, no numbers exposed; while the alert is open the watcher page loops the ElevenLabs voice, "Your buddy Chris is in trouble, his alarm has been unacknowledged for twelve minutes," until Call is tapped, and beside the loop the relay (relay/notify.py) sends the buddy the same text and the clip as a WhatsApp message through the Cloud API, the second channel, never a glucose value; the device's next poll plays the buddy chime → resolution on two readings back above the threshold or an acknowledge → the close-out is written by the narrative pipeline and sealed to both sides. T+20: the patient's own emergency script fires on an independent clock.py timer that no claim, call, or status can delay. A Brain-only patient has no radar, so its condition is stricter: confirmed level 2 low sustained 10 minutes with zero phone interaction, listed as unconfirmed.

Night Buddy, the hub Listing (first name, alarm state, elapsed minutes, confidence label; the relay rejects any glucose, location, or phone field) → the verified volunteers' list, ordered by urgency then confidence → claim (lease; a second simultaneous tap gets a clean conflict) → the emergency script is visible only to the live claim-holder → brokered call → the patient's one-press "I'm treating" clears the listing for 20 minutes; unrecovered at expiry, it returns at top urgency → resolution by glucose, never by a volunteer's word → every claim in the audit log.

Family Story, the morning (Meta) Morning-report job → one FamilyStory per active recipient → the family prompt (two to three sentences, the patient's agency, never alarm, never advice) → validator inverted for Level 1 (zero mg/dL values, clock times allowed), template on any violation; a no-data night says Irin had no data, never "all fine" → the first story to anyone, and every story for approve-each recipients, waits for the patient's tap → the existing SMTP path with an unsubscribe line, Level 2 adding the family-view link → "Sent to Mom" chip with one-tap pause; paused and revoked recipients are skipped.

Demo mode A replay engine replaces the Nightscout poller behind the same interface and replays real historical data at any speed. Every arrow downstream is identical. A 24-week watch cannot be played ($4,032 \text{ hours} \div 60 = 67 \text{ hours at } 60x$), so the replay engine gains a seek: it bulk-loads readings to a timestamp, advances clock.py, and the engine's idempotent catch-up generates every card that should exist by then; the demo panel's buttons read "jump to step N day D". Demo cards, alerts, listings, and stories carry is_demo, land only in demo pairings and the demo hub, and never reach a real inbox, phone, or mailbox.

Complete Parts List (Exact Items)

Every item in the build. One ribbon cable option is needed, not both.

Component	Exact item
Computer	Raspberry Pi 5 with Raspberry Pi Active Cooler
Storage	256GB microSDXC card, flashed with Raspberry Pi OS via Raspberry Pi Imager
Display	Raspberry Pi Touch Display 2, 7 inch (SC1635)
Display ribbon cable	Included in the Touch Display 2 box (22-pin to 15-pin FFC for the Pi 5). GDIYMODULES 2-pack kept as spares
Display power cable	Included in the Touch Display 2 box (GPIO power cable, 5V pin 2 and ground pin 6)
Display mounting screws	Included in the Touch Display 2 box (M2.5); the SmartiPi case includes its own hardware
Case	SmartiPi Touch Pro 3, Large, by Smarticase
Pi wall power	RasTech 27W GaN PD USB-C power supply, 5.1V 5A, plugged directly into the Pi
LED strip + corners	BTF-LIGHTING WS2812B RGB ECO LED strip bundle: 16.4FT 60LED/m 300 pixels IP30 DC5V, with 10 sets of 3PIN 10mm right-angle L-shape solderless

Component	Exact item
	corner connectors
LED channels	Muzata 6-pack 3.3FT silver LED strip channel with milky diffuser, U1SW WW, 17x7mm, fits strips up to 12mm
LED power	ALITOVE 5V 5A AC-to-DC power supply adapter, 5.5x2.5mm plug, with green screw-terminal barrel adapter
Level shifter	Bridgold SN74AHCT125N quad bus buffer, DIP-14, 10-pack
Resistor	330 ohm from the ELEGOO UNO R3 Super Starter Kit resistor assortment
Capacitor	Rubycon 1000uF 16V 10x16mm electrolytic capacitor, 15-pack (EEEEEE)
Presence radar	EC Buying HLK-LD2410B-P 24GHz mmWave human presence sensor module with Bluetooth, pre-soldered 1.27mm-pitch pin header
Radar cable	EC Buying 1.25mm to 2.54mm female Dupont adapter cable, 5-pin, 20cm, 10-pack
Speaker	LIELONGREN USB computer soundbar speaker (USB plug-n-play audio)
Breadboard	Mini breadboard from the ELEGOO UNO R3 Super Starter Kit (full-size breadboard as spare)
Jumper wires	Inland 30cm 40-pin female-to-male set, plus ELEGOO 120pcs Dupont wire M-F / M-M / F-F
Screwdriver	Small Phillips #0 / #1 precision screwdriver (confirm owned)
Mounting tape	3M Double Sided Tape Heavy Duty 5925, 0.5 inch x 15.4FT, black
Saw	KATA 10 inch mini hacksaw for metal cutting, with 3 blades, 24 TPI
Miter box	Hanpex Metal Small Aluminum Miter Box, 5.3 inch, 45 and 90 degree slots
Files	ValueMax 7PCS interchangeable needle file set
Battery backup	UGreen 100W 20000mAh Power Bank
Battery-to-LED cable	Meliya USB-A to DC 5.5x2.1mm barrel jack power cable, 2-pack, 3.3ft
USB cables	Anker 322 USB-A to USB-C 6ft braided (x2), Anker 60W USB-C to USB-C, Anker 60W USB-C to USB-A
Backup display	52Pi 7 inch capacitive touch HDMI screen, with 2x Inland HDMI to micro-HDMI cables
Power strip	Any surge-protected power strip with a 10ft+ cord
Card reader	USB microSD card reader (only if the laptop has no SD slot)
Consumables	Electrical tape, zip ties, velcro or Command strips, scissors, sharpie, sandwich bags or parts organizer, ruler and pencil
Required tools	Multimeter, borrowed is fine (the polarity check at the screw terminal before first power, and every debugging step after); the no-meter fallback

Component	Exact item
	for polarity is the kit's LED + 330 ohm resistor. A few Wago 221 lever connectors (\$5) join the strip's data lead to a jumper without solder if its end is not factory-tinned.
Optional / borrow	Drill with 6 to 8mm bit (only if drilling the back cover); hot glue gun; soldering iron (not needed: corners clip, power screws in, the one breadboard lead is tinned or joined with a Wago 221)

Tech Stack

- **Pi OS.** Raspberry Pi OS 64-bit, Trixie (Debian 13) (aarch64; XGBoost, PyNaCl, and pydantic ship prebuilt wheels only for it), with Chromium in kiosk mode for the display UI, enabled by hand after the final audit. Python 3.14 everywhere: laptops natively, the Pi through uv (Trixie ships 3.13 and never 3.14 from apt; the 3.13 escape hatch is why the release matters), with the code kept 3.13-compatible.
- **Backend on the Pi.** Python and FastAPI with WebSocket for live updates, SQLite as the local-first cache and ledger (the device works with zero internet), Pi5Neo or Adafruit Blinka SPI NeoPixel for LEDs (rpi_ws281x does not work on the Pi 5), gpiozero on lgpio for the radar, aplay or pw-play for sound inside the kiosk user's PipeWire session (no pygame), smtplib over the Gmail app password for the morning email and Family Story, matplotlib for the email graph, ffmpeg to turn ElevenLabs mp3 into wav, and an outbound forwarder that streams readings and events to Irin Cloud every five minutes.
- **Data.** Dexcom G7 → Dexcom Share → Nightscout on Railway (its database on MongoDB Atlas) through Nightscout's connect plugin (CONNECT_SOURCE=dexcomshare; the old bridge plugin is deprecated) → the Pi's 60-second poller; the replay engine behind the same interface with a seek for demos; 8 Dexcom Clarity export CSVs covering about 22 months for training, evaluation, and the My Irin history.
- **Display UI.** HTML, CSS, and JavaScript served from FastAPI on localhost: the kiosk glances and alarms and carries no forms.
- **The app.** One hosted web app at irin-out-of-sleep-at-hackgt.tech with four tabs, Irin Device, My Irin, Irin Buddy, Irin Rounds, gated by the PIN (the "1234" code, entered once, kept in sessionStorage, sent as a header). Vite + React + TypeScript + Tailwind (decision 28), built with npm run build into app/dist/, which is committed and served by Caddy, so the server never runs Node; message types generated from the Pi's and the relay's /openapi.json (npm run types), so a field that does not exist is a compile error; d3-geo inside a React component for the timezone globe (orthographic projection, drag to spin, geoContains for the tap) over the bundled timezone-boundary-builder GeoJSON; Web Speech API for voice logging. The kiosk display and the three role pages stay plain HTML, CSS, and JavaScript with no build step; Web Audio and the Wake Lock API on the watcher page. The Device tab talks to the paired Pi at device.irin-out-of-sleep-at-hackgt.tech; the other tabs talk to the relay and Irin Cloud. The role pages live beside it: doctor.irin-out-of-sleep-at-hackgt.tech (the inbox), watch.irin-out-of-sleep-at-hackgt.tech (buddies and the hub), family.irin-out-of-sleep-at-hackgt.tech (the family view).
- **Irin Cloud on Vultr.** One Vultr VPS running Docker Compose: Caddy (automatic HTTPS for every .tech name), the relay (the blind courier plus the hub and the buddy directory; MongoDB Atlas for its documents; ciphertext and no glucose, ever), and the cloud service (ingest, time series

on Tiger Cloud, the dashboards API, the family rollup, matching, audio rendering). The same compose file with local mongo and timescaledb containers is the laptop fallback.

- **Databases.** MongoDB Atlas (free M0): Nightscout's own database, and the relay's collections (profiles, buddy links, hub listings with TTL indexes as a janitor, pairings, sealed cards and messages, audit). Tiger Cloud (TimescaleDB): hypertables for readings, alarm events, low events, and treatments, continuous aggregates for the My Irin dashboards and the family rollup, compression on the history. The Pi keeps SQLite so it never depends on either.
- **Voice.** ElevenLabs text to speech, rendered once and cached: the buddy alert the watcher page loops, the device's spoken echoes (doctor-message confirm, voice-logging echo, the predicted-low voice after the tone), the morning report's play button, the Family Story voice note attached to the email, and the emergency script read to a volunteer. Alarm tones stay tones.
- **Narratives.** narrative.py with a chain of four links and the no-invented-numbers validator after every one: Backboard (one API key, X-API-Key against app.backboard.io/api, the backboard-sdk client; assistants as memory scopes, one per NarrativeScope, memory Auto for patient, doctor, and buddy scopes and Readonly for family; llm_provider and model_name per message from the routing table; a different provider gives a second opinion on every card) → the Meta Model API direct (<https://api.meta.ai/v1>, OpenAI-SDK compatible through the openai package with base_url; used when the routing table names provider meta, or as the fallback for openrouter rows when Backboard is down) → the direct Claude API → the deterministic templates. NARRATIVE_BACKEND (backboard | anthropic | template) names the first link to try; NARRATIVE_ROUTING (JSON, task → [provider, model]) has eight tasks, final: clinician_card and morning_report → anthropic, a Sonnet-class Claude; buddy_line and family_story → openrouter with the Muse Spark name if Backboard's OpenRouter provider carries it, else meta / muse-spark-1.3; match_explanation, emergency_script, and availability_parse → meta / muse-spark-1.3, always direct, structured output; second_opinion → openai, a small model, a different provider from the writer.
- **Meta.** The Meta challenge entry is built on Meta's own pieces (decision 29). The Meta Model API: base URL <https://api.meta.ai/v1>, OpenAI-SDK compatible (the openai package with base_url; openai in backend/requirements.txt and relay/requirements.txt), models muse-spark-1.3 (text, structured output), muse-voice-transcribe-1.0 (speech to text, optional later for voice logging), muse-image-1.0; env META_MODEL_API_KEY on the Pi and on the server. Muse Spark writes the Irin Buddy narratives: the introduction line and the why-this-match line from the score parts (first names only, never a glucose value), the emergency script (rough notes → a calm, ordered script, JSON fields), free-text availability parsed into the week-grid rows the user then confirms, and the Family Story; the matching score stays deterministic (the Atlas aggregation) and an LLM never picks the buddy. The Graph API: the WhatsApp Cloud API is the second alert channel. At the buddy rung the relay (relay/notify.py) sends the registered buddy a text, "Irin: your buddy <first name> is in trouble. The alarm has been unacknowledged for N minutes. Open <watch URL>", and the ElevenLabs clip as an audio message (type audio, link on cloud.irin-out-of-sleep-at-hackgt.tech) through POST https://graph.facebook.com/<version>/{PHONE_NUMBER_ID}/messages, beside the watcher-page loop; never a glucose value in the message (relay invariant); free-form messages only inside WhatsApp's 24-hour window after the buddy last messaged the number (each buddy sends "hi" during setup and before the demo), an approved template outside it; env on the server only: WHATSAPP_TOKEN, WHATSAPP_PHONE_NUMBER_ID. Optional Facebook Login for "a buddy I already know" (VITE_FB_APP_ID in frontend/app/.env.production, public; /me/friends returns friends who also use Irin), only if Justin has time, cut first. Locked-phone honesty: a web page cannot ring through silent; the demo is the wake-locked watcher page plus the WhatsApp message; production is native Critical Alerts or a call through the WhatsApp/Twilio calling API.

- **ML.** pandas and NumPy for cleaning and features, XGBoost for the forecaster (scikit-learn for metrics), matplotlib for evaluation plots, saved as XGBoost JSON and loaded on the Pi with one shared feature module for training and inference; ml/models/nights.py, numpy only, for every card number; SQL over Tiger Cloud for the dashboards, with a test that the two agree wherever they overlap.
- **Tools.** GitHub repo (main, dev, and feature branches), Claude Code and Cursor under one constitution (CLAUDE.md mirrored into .cursor/rules and AGENTS.md), VS Code with Remote SSH into the Pi, Raspberry Pi Imager, Docker Desktop (Chris's laptop only, for the compose fallback), Cloudflare (DNS for the domain and the Pi's named tunnel), Railway (Nightscout), Vultr, MongoDB Atlas, Tiger Cloud, ElevenLabs, Backboard, the Meta developer app (Model API, WhatsApp Cloud API), and the HLK Bluetooth app for radar setup.
- **Rounds (Impiricus).** PyNaCl on the Pi and tweetnacl-js in the browser (X25519 crypto_box authenticated encryption), a qrcode generator for pairing, the relay on Vultr with MongoDB Atlas behind it (runnable on the laptop from the same compose file), the clinician inbox at doctor.irin-out-of-sleep-at-hackgt.tech, and ml/models/nights.py (numpy only). Card narratives come through Backboard with a clinician prompt behind the validator and a second-model check.
- **Night Buddy (Meta).** Hub tables and the buddy directory on the same relay (Atlas), the matching pipeline as one aggregation, the watcher page at watch.irin-out-of-sleep-at-hackgt.tech with the ElevenLabs voice loop, a wake lock, and the brokered call as a relay event the Pi's poll turns into the chime, matching profiles from the Find a Buddy flow, and the WhatsApp Cloud API as the second alert channel (relay/notify.py: the alert text and the clip as an audio message). The demo keeps the watcher page open (wake lock) and sends the WhatsApp message; production is native Critical Alerts or a call (the WhatsApp/Twilio calling API), named in the video narration, not built.
- **Family Story (Meta).** Zero new infrastructure: the narrative module with a family prompt (the family_story task, Muse Spark through narrative.py, behind the same validator) and the validator inverted per level, Settings for recipients and consent, the morning report's SMTP path with an unsubscribe line, and, new, a 20-second ElevenLabs voice note attached to the email. The family page reads a rollup from Irin Cloud with a revocable family bearer instead of a Nightscout token in page source.

Team, Repository, and Workflow

- **Chris (backend):** the Python service under backend/app/, plus sounds/ and tests/ — datasources, alarm engine, IOB, forecast wrapper, voice parsing, reports, WebSocket hub, auth, scheduler.
- **Justin (frontend):** everything under frontend/ — the kiosk display page (served by FastAPI at /), the hosted app with its four tabs (a Vite + React + TypeScript + Tailwind project in frontend/app/, built into dist/ and served from Vultr), and the family, doctor, and watcher pages (plain HTML, served from Vultr beside it). The display and role pages have no build step: edit, refresh, see it. The app has one: npm run dev for hot reload while working, npm run build plus a committed dist/ before every merge, and npm run types to regenerate the message types from the backend's and the relay's OpenAPI. Chris provides the WebSocket client scaffolding (the useDeviceSocket hook) and keeps the voice flow's confirm logic; everything visual is his. Suggested ramp: the gate and tab shell, then the Device tab's live view and acknowledge, then settings, then My Irin and Find a Buddy, with the family view as the stretch goal.
- **George (ML and AI):** everything under ml/ — data cleaning, dataset, training, evaluation, the model artifact.

- **Slavik (hardware):** everything under hardware/ — LED and sound drivers, the presence radar reader, mocks, wiring scripts and docs, plus the physical build.
- **Shared files, owner of record: Chris.** The repo-root files (README, LICENSE, .gitignore, .env.example, CLAUDE.md), .claude/, contracts.py, demo/, docs/, and deploy/ are all created by Chris in the day-one skeleton commit, so every teammate's Claude Code boots into the same rules from the first clone. Afterward anyone drafts changes to shared files on a branch, but the merge goes through Chris. Content still comes from everyone (George writes the metrics claims, Slavik the wiring notes); Chris is the merge gate and editorial voice, not the sole author.
- **Sponsor-challenge additions, same lanes.** relay/ is Chris's (owner of record like the other shared services); frontend/clinician/ and frontend/watch/ are Justin's; ml/models/nights.py is George's second boundary file; hardware asks nothing mandatory, and Slavik owns the Meta demo video. Two more seams join the four meeting points: relay/README.md (the card schema and relay API the inbox and watcher page render from) and demo/scenarios/README.md (the companion JSON George writes and Chris's batch ledger build reads).

Meeting points. Four boundaries, each with one file or contract: backend/app/contracts.py (the shared data models, reviewed by a second person before merging), the WebSocket message types and REST endpoints defined there (the seam between Chris's backend and the frontend; changes to what the frontend sends get Chris's sign-off before merging), ml/models/predict.py (George maintains, Chris calls), and hardware/hal.py (Slavik makes real on the Pi, Chris imports). Everything else is edited by exactly one owner.

Three rules. 1) contracts.py is the treaty: any change gets a second pair of eyes. 2) Everything runs on a laptop: the replay engine substitutes for Nightscout and hal.py substitutes mocks for LEDs, speaker, and the presence radar (IRIN_HW=mock), so nobody needs the Pi to develop. 3) Real medical data never enters git: the raw Clarity export lives only in the gitignored ml/data/, and only cleaned, date-shifted demo scenarios and the trained model are committed.

Workflow. Three tiers of branches: main, dev, and feature branches. main is what the Pi pulls, so it must always boot, and it is HUMAN-ONLY: GitHub branch protection blocks direct pushes, and only Chris merges dev into main, by hand, once dev is verified working — never a Claude. dev is the integration branch where finished features land and where everything-works gets proven. Feature branches (chris/alarm-engine, george/train-v1) cut from dev and merge back to dev after the human reviews the diff: short-lived, small fast merges, no branch older than a day. The Pi deploys by git pull of main and a service restart.

Repository Structure

```

irin/
|-- README.md           (overview, how to run)
|-- CLAUDE.md           (rules every Claude Code session auto-loads)
|-- .cursor/
|   |-- rules/
|       |-- irin.mdc     (CLAUDE.md mirrored for Cursor; regenerated by deploy/sync_rules.sh)
|-- AGENTS.md           (the same constitution for any other agent)
|-- journal/            (cross-Claude bulletin)
|   |-- chris.md         (overwritten each session; git keeps history)
|   |-- justin.md
|   |-- george.md
|   |-- slavik.md
|-- LICENSE              (MIT)

```

```

|-- .gitignore
|-- .env.example      (template for secrets)
|-- .env              (real secrets, never committed)
|-- .claude/
|   |-- commands/
|       |-- audit.md      (the /audit final-audit command)
|       |-- review.md     (the /review pre-merge command)
|       |-- skeleton.md   (the /skeleton day-one bootstrap, run once)
|
|-- backend/ -- OWNER: CHRIS --
|   |-- requirements.txt
|   |-- app/
|       |-- __init__.py
|       |-- main.py        (FastAPI entry point)
|       |-- contracts.py   (shared data models)
|       |-- config.py      (settings)
|       |-- auth.py        (PIN gate for tunnel access)
|       |-- clock.py       (real or accelerated time source)
|       |-- scheduler.py   (night window, basal nudge, report trigger)
|       |-- presence.py    (Home/Away state machine: debounce, night rule, toggle override)
|       |-- store.py       (SQLite layer)
|       |-- alarm.py       (alarm state machine + context rules)
|       |-- iob.py         (insulin on board)
|       |-- forecast.py    (runs the ML model)
|       |-- voice.py       (voice text parser)
|       |-- reports.py     (stats, graph PNG, LLM summary, email)
|       |-- ws.py          (WebSocket hub)
|       |-- forward.py     (outbound forwarder: readings and events to Irin Cloud every five minutes)
|       |-- voice_out.py   (ElevenLabs spoken echoes after the tone; mp3 → ffmpeg → wav in
sounds/generated/; VOICE_BACKEND none | elevenlabs)
|       |-- family_story.py (Family Story: recipients, family prompt, send hook)
|       |-- rounds/        (Irin Rounds: the platform, Standing Cards, Step Watch)
|           |-- __init__.py
|           |-- ledger.py   (NightRecord at night-window end; idempotent catch-up)
|           |-- alarm_events.py (AlarmEvent recorder, observes alarm.py)
|           |-- nights_adapter.py (reads the Pi's store for nights.py; brain_only mode)
|           |-- low_events.py  (LowEvent per nocturnal low: nadir, slope, inferred unfelt)
|           |-- recall.py     (morning recall: one card per low, unfelt-low rate)
|           |-- cards.py      (SignalCard assembly, headline from code, confidence map)
|           |-- standing.py   (Basal Check, Hypo Response, Follow-up rules)
|           |-- step_watch.py (TitrationPlan, windows, status rules, gates, graduation)
|           |-- narrative.py  (Backboard → Meta → Claude → template + validator)
|           |-- noise.py     (the noise budget: rate limits + program interaction)
|           |-- crypto.py    (PyNaCl crypto_box seal / open, device keypair)
|           |-- pairing.py   (QR, 4-digit confirm, revoke; peer_kind doctor | buddy)
|           |-- relay_client.py (outbound POST cards; one poll for doctor messages and owner-pairing
completions)
|       |-- messages.py      (DoctorMessage queue, confirm / decline / expire)
|       |-- vigilance.py     (step-week vigilance, predicted-low threshold only)
|       |-- buddy/          (Night Buddy, demo tier)
|           |-- __init__.py
|           |-- rung.py      (buddy-rung observer + T+20 emergency-script clock)
|           |-- treating.py  (treating status, expiry, opt-ins)
|       |-- datasource/
|           |-- __init__.py
|           |-- base.py      (source interface)
|           |-- nightscout.py (live poller)
|           |-- replay.py    (demo engine)
|-- sounds/                (alarm audio)
|   |-- alarm_soft.wav

```

```

| | |-- alarm_urgent.wav
| | |-- chirp.wav
| | |-- buddy_chime.wav (Night Buddy brokered-call chime)
| | |-- make_tones.py (generates the four wavs; run once)
| | |-- generated/ (ElevenLabs wav cache from voice_out.py; created at runtime, gitignored)
| |-- tests/
| | |-- test_alarm.py
| | |-- test_iob.py
| | |-- test_voice.py
| | |-- test_replay.py
| | |-- test_presence.py
| | |-- test_alarm_events.py
| | |-- test_ledger.py
| | |-- test_low_events.py
| | |-- test_standing.py
| | |-- test_noise.py
| | |-- test_step_watch.py
| | |-- test_recall.py
| | |-- test_seek.py
| | |-- test_narrative.py
| | |-- test_crypto.py
| | |-- test_pairing.py
| | |-- test_messages.py
| | |-- test_vigilance.py
| | |-- test_buddy.py
| | |-- test_family_story.py
| |-- irin.db (created at runtime, gitignored)
| |-- keys/ (created on first boot, gitignored)
| | |-- device_private.key (never leaves the Pi)
| | |-- device_public.key (shared at pairing)
|
|-- frontend/ -- OWNER: JUSTIN --
| |-- display/ (kiosk page, served by FastAPI at /)
| | |-- index.html
| | |-- style.css
| | |-- display.js
| |-- app/ (the hosted four-tab app: Vite + React + TypeScript + Tailwind; dist/ served at irin-out-
of-sleep-at-hackgt.tech)
| | |-- package.json (react, react-dom, d3; vite, typescript, tailwindcss, openapi-typescript)
| | |-- vite.config.ts (react + tailwindcss plugins)
| | |-- index.html
| | |-- .env.production (VITE_RELAY_URL, VITE_CLOUD_URL, VITE_FB_APP_ID; public values, committed)
| | |-- .env.development.local (VITE_* dev targets, VITE_DEVICE_URL for the mock backend;
gitignored)
| | |-- src/ (main.tsx, App.tsx, config.ts, tabs/{device,myirin,buddy,rounds}/, lib/ hooks, types/
generated, assets/timezones.geojson)
| | |-- public/ (manifest.json, favicon.svg)
| | |-- dist/ (npm run build output; COMMITTED, what Caddy serves)
| |-- family/ (read-only view, family.irin-out-of-sleep-at-hackgt.tech)
| | |-- index.html
| | |-- style.css
| | |-- family.js
| | |-- config.js (Irin Cloud URL + family bearer)
| |-- clinician/ (doctor inbox, "mock Ascend", doctor.irin-out-of-sleep-at-hackgt.tech,
reads the relay)
| | |-- index.html
| | |-- style.css
| | |-- inbox.js
| | |-- config.js (relay URL)

```

```

| |-- watch/                                (Night Buddy watcher: buddy alerts + hub, watch.irin-out-of-sleep-at-
hackgt.tech)
|     |-- index.html
|     |-- style.css
|     |-- watch.js
|     |-- config.js          (relay URL)
|
|-- ml/  -- OWNER: GEORGE --
|     |-- requirements.txt
|     |-- clean_clarity.py    (clean the export)
|     |-- build_dataset.py    (features + labels)
|     |-- train.py
|     |-- evaluate.py         (accuracy + low-catch metrics)
|     |-- events.py          (the event replay shared by evaluate.py and label_history.py)
|     |-- label_history.py    (inferred reason codes and events over the 22 months, Rounds)
|     |-- evaluate_rounds.py  (lead time, false-alarm rate, detection lag; metrics.md)
|     |-- data/              (gitignored)
|     |   |-- README.md
|     |   |-- raw/           (the 8 Clarity exports, runtime only)
|     |   |   |-- Clarity_Export_*.csv (the 8 Dexcom exports; contain name and DOB)
|     |   |   |-- clean.csv      (runtime only)
|     |   |   |-- therapy_changes.txt (Chris's therapy-change dates, runtime only)
|     |   |   |-- nights_labeled.csv (inferred per-night labels, runtime only)
|     |-- notebooks/
|     |   |-- exploration.ipynb
|     |-- models/
|     |   |-- forecast_v1.json (the trained model)
|     |   |-- features.py      (shared feature code)
|     |   |-- predict.py       (prediction function)
|     |   |-- nights.py        (night classification + metrics, Rounds; backend imports it)
|     |   |-- metrics.md       (eval numbers)
|
|-- hardware/  -- OWNER: SLAVIK --
|     |-- requirements.txt
|     |-- hal.py              (real or mock switch)
|     |-- leds.py             (LED effects)
|     |-- sound.py            (speaker control)
|     |-- presence.py         (LD2410 radar reader)
|     |-- mock.py             (laptop no-ops)
|     |-- scripts/
|     |   |-- first_light.py
|     |   |-- full_frame_test.py
|     |   |-- speaker_test.sh
|     |   |-- radar_watch.py  (prints GPIO17 presence live)
|     |-- docs/
|     |   |-- wiring.md       (pins + columns)
|     |   |-- figures/        (diagram images)
|     |       |-- fig01_pi5_landmarks.png
|     |       |-- fig02_pi_on_display_standoffs.png
|     |       |-- fig03_dsi_ribbon_seating.png
|     |       |-- fig04_display_power_kit_alternative.png
|     |       |-- fig05_cutting_the_strip.png
|     |       |-- fig06_frame_corner_connector.png
|     |       |-- fig07_finding_gpio_pins.png
|     |       |-- fig08_circuit_at_a_glance.png
|     |       |-- fig09_breadboard_hole_by_hole.png
|     |       |-- fig10_screw_terminal.png
|     |       |-- radar_wiring_*.jpg (Slavik's photos of the adapter wire colors)
|
|-- relay/  -- OWNER: CHRIS --

```

```

| |-- requirements.txt
| |-- main.py          (blind courier: cards, doctor messages, pairing, buddy alerts)
| |-- hub.py           (Night Buddy hub: listings, claims, treating, audit)
| |-- notify.py        (WhatsApp Cloud API channel: the buddy text + the audio clip; reads
WHATSAPP_*)
| |-- README.md        (Clinical Signal Card v0 schema + versioned API)
| |-- tests/
| | |-- test_relay.py
| | |-- test_hub.py
|
|-- cloud/ -- OWNER: CHRIS (sql/ George) --
| |-- requirements.txt
| |-- README.md        (what Irin Cloud is: ingest, dashboards API, family rollup, audio)
| |-- main.py          (Irin Cloud: ingest, dashboards API, family rollup, audio)
| |-- migrate.py       (applies cloud/sql/ once at cloud boot)
| |-- ingest.py        (POST /v1/ingest, idempotent on (device_id, timestamp))
| |-- dash.py          (GET /v1/dash/* from the continuous aggregates)
| |-- audio.py         (ElevenLabs render + cache; buddy alert, script, story)
| |-- audio_cache/     (rendered clips, created at runtime, gitignored)
| |-- sql/             (George: hypertables, continuous aggregates, policies)
| | |-- 001_hypertables.sql
| | |-- 002_aggregates.sql
| | |-- 003_policies.sql
| |-- tests/
| | |-- test_ingest.py
| | |-- test_dash.py
|
|-- demo/ -- SHARED, MERGES VIA CHRIS --
| |-- make_scenarios.py
| |-- DEMO_SCRIPT.md   (run of show)
| |-- scenarios/
| | |-- the_save.csv
| | |-- normal_night.csv
| | |-- failure.csv
| | |-- meal_context.csv
| | |-- rearm_low.csv
| | |-- high_spike.csv
| | |-- basal_change_*.csv + .json (14 real nights either side of a real basal increase)
| | |-- titration_synthetic.csv + .json (SYNTHETIC ~3-week GLP-1 step scenario)
| | |-- README.md      (SYNTHETIC notes + the companion JSON schema)
|
|-- docs/ -- SHARED, MERGES VIA CHRIS --
| |-- Irin_Spec.pdf
| |-- Irin_Rounds.pdf  (Impiricus challenge companion spec)
| |-- Irin_Night_Buddy.pdf (Meta challenge companion spec)
| |-- plans/           (per-owner Claude briefs)
| | |-- chris.md
| | |-- justin.md
| | |-- george.md
| | |-- slavik.md
| |-- PITCH.md
| |-- FAMILY_STORY.md  (Family Story one-page definition)
| |-- META_WRITEUP.md  (the Meta write-up, team effort)
| |-- OPEN_ITEMS.md    (decisions log + remaining to-dos)
|
|-- deploy/ -- SHARED, MERGES VIA CHRIS --
| |-- install.sh       (one-time Pi setup)
| |-- irin.service     (backend on boot)
| |-- kiosk.service    (Chromium on boot)
| |-- cloudflared-config.yml (the tunnel)

```

```

|-- nightscout-railway.md    (cloud Nightscout setup)
|-- atlas.md                (Atlas cluster, Nightscout MONGODB_URI, relay collections)
|-- public_mirror.sh        (submission-day public mirror: squash, exclude, trim, grep, push)
|-- kiosk.sh                (kiosk by hand: --now shows it, Alt+F4 closes; autostart stays disabled
until the final audit)
|-- sync_rules.sh           (regenerates .cursor/rules/irin.mdc and AGENTS.md from CLAUDE.md)
|-- docker-compose.yml      (caddy + relay + cloud + local mongo/timescaledb for the laptop)
|-- Caddyfile               (irin-out-of-sleep-at-hackgt.tech, api., cloud., doctor., watch.,
family.)
|-- vultr.md                (VPS setup: Docker, DNS records, compose up, env)
|-- pi_setup.md

```

Initial Setup, Per Person

Everyone does the common steps once, in order, then their role's extra steps. Chris's day-one skeleton commit must exist before anyone else starts.

Common Steps (everyone)

1. Install prerequisites. Check git (git --version) and Python 3.14 (python3 --version; install it with uv or pyenv if the system has another). Install whichever is missing from git-scm.com and python.org.
2. Install Claude Code and sign in with YOUR OWN subscription. Install per the current instructions at docs.claude.com (npm install -g @anthropic-ai/claude-code for Justin, who has Node, or the native installer for everyone else). Run claude once and complete /login with your own account. Nobody uses anyone else's login.
3. Accept the GitHub invite from Chris, then clone: git clone <repo-url> and cd irin.
4. Create a virtual environment and install dependencies: python3 -m venv .venv, activate it, then pip install -r backend/requirements.txt for Chris and Justin, ml/requirements.txt for George, hardware/requirements.txt for Slavik. Note for Justin: you also install Node 22 LTS and run npm install from frontend/app/ (decision 28: the app is Vite + React + TypeScript + Tailwind); the display and role pages still have no dependencies. You install the backend's because running it in mock mode is what the Device tab talks to while you edit. Python 3.14 on every laptop (uv or pyenv); Docker Desktop on Chris's laptop only, for the compose fallback.
5. Copy .env.example to .env and fill the values Chris hands you in person or by direct message, never through the repo. The .env file stays uncommitted forever. George sets NARRATIVE_BACKEND=template on his laptop and hands back only the Backboard key and the routing line. The decision-29 names are in .env.example too: META_MODEL_API_KEY (the Pi and the server), WHATSAPP_TOKEN and WHATSAPP_PHONE_NUMBER_ID (the server only), and VITE_FB_APP_ID in frontend/app/.env.example.
6. Start Claude Code in the repo root (claude) and confirm it absorbed the constitution: ask it "what directories do I own and what are the journal rules?" It should answer from CLAUDE.md. If it cannot, you are in the wrong directory.
7. Smoke-test the stack: from backend/, run IRIN_HW=mock uvicorn app.main:app --reload and open http://localhost:8000 (display) in a browser; Justin additionally runs npm run dev from frontend/app/ and opens http://localhost:5173 (the app). Everyone runs this once, even non-backend roles; it proves your clone works end to end.

8. Write your first journal entry (journal/<you>.md, the five headers), cut your first branch from dev (<yourname>/setup-test), commit the journal entry on it, push, and have Chris merge it into dev. This proves push rights and teaches the loop in one shot.

Chris, extra steps

1. Before anyone else starts: create the private repo, clone it, move the prepared documents in at their repo paths (CLAUDE.md at root; audit.md, review.md, and skeleton.md into .claude/commands/; the four briefs into docs/plans/; the three spec PDFs into docs/), then start Claude Code in the repo root and run /skeleton. The command builds everything below, verifies the backend boots in mock mode, and stops WITHOUT committing; review the diff, make the initial commit yourself, create dev from main and push both, turn on GitHub branch protection for main (require pull request, no direct pushes), and send invites. The skeleton must contain: CLAUDE.md; .claude/commands/ with review.md and audit.md; docs/plans/ with all four role briefs and the three spec PDFs in docs/; contracts.py with REAL content (models and message types, including the alarm state's trigger_type field), not a stub, since four Claudes build against it from hour one; a backend that actually BOOTS in mock mode (main.py, clock.py, config with the datasource switch, mock hal), because the main-always-boots rule applies to commit number one; journal/ with four seeded files already in the correct five-header format, so the first thing every Claude reads has the right shape to imitate; .gitignore covering .env, the SQLite db, ml/data/, __pycache__; .env.example with placeholders only; README with the quickstart; and at least one date-shifted scenario CSV in demo/scenarios/ so the replay engine has something to play on day one.
2. Hand George the 8 Clarity export CSVs (already exported, about 22 months) on a USB stick or by direct message. They never touch the repo.
3. Set up the seven cloud accounts before the event and keep every key in one private note that becomes the team's .env: the .tech domain, irin-out-of-sleep-at-hackgt.tech (nameservers to a free Cloudflare account, which the Pi's named tunnel needs anyway), Vultr (one Ubuntu VPS with Docker, its IP in DNS as irin-out-of-sleep-at-hackgt.tech, api., cloud., doctor., watch., family., all DNS-only so Caddy can issue certificates), MongoDB Atlas (an M0 cluster, a database user, network access open for the weekend; set Nightscout's MONGODB_URI on Railway to it, then recreate the irin read token or restore the old data, since Nightscout's access tokens live in the database), Tiger Cloud (a free service and its connection string), ElevenLabs (an API key and two voice ids), Backboard (an API key from app.backboard.io and the Anthropic model names from GET /api/models; check whether its OpenRouter provider carries Muse Spark), and the Meta developer app (the Model API key as META_MODEL_API_KEY, on the Pi and the server; the WhatsApp Cloud API test number with up to five registered recipients as WHATSAPP_TOKEN and WHATSAPP_PHONE_NUMBER_ID, server only; optional Facebook Login as VITE_FB_APP_ID in frontend/app/.env.production). Plus the ones you already hold: Railway for Nightscout, Cloudflare for the tunnel, the Anthropic key, SMTP, the PIN. The Pi's .env also carries DEVICE_URL (https://device.irin-out-of-sleep-at-hackgt.tech; http://localhost:8000 on a laptop).

Justin, extra steps

1. With the backend running in mock mode, edit frontend/display/style.css (change a color), refresh the browser, and see it change. That refresh loop is the whole toolchain for the display and the role pages. The app adds one step: npm run dev gives the same loop with hot reload, and npm run build plus a committed dist/ goes with every merge.

2. Open the app on your phone: `npm run dev -- --host`, then `http://<laptop-ip>:5173` on the same Wi-Fi, so you know the phone-testing loop from day one.
3. Generate the message types and set up the dev config: `npm run types` from `frontend/app/` (openapi-typescript against the mock backend's `http://localhost:8000/openapi.json` and the relay's), then create `frontend/app/.env.development.local` (gitignored) with `VITE_RELAY_URL` and `VITE_CLOUD_URL` for your dev targets and `VITE_DEVICE_URL=http://localhost:8000`, the development-only override: when it is set and the backend's `/api/health` reports `hw: mock`, the Device tab uses it directly, skips pairing, and shows a small DEV badge. It never applies to a real Pi; there the Device tab uses the `device_url` it got from pairing.

George, extra steps

1. Receive the 8 Clarity export CSVs from Chris (USB stick or direct message) and place them in `ml/data/raw/`. Confirm git status does NOT list them; if it does, stop and fix `.gitignore` before anything else. The files contain Chris's name and date of birth, so they never enter a Claude conversation either: scripts print summaries only.
2. Have your Claude run `clean_clarity.py` and review the printed summary it shows you (date range, reading count, gaps, spike-filter drops). That is your first checkpoint in the Claude-drives, George-steers SOP below; approve it before moving to the dataset build.

Slavik, extra steps

1. On your laptop, hardware runs as mocks; nothing extra to install. The real `hardware/requirements.txt` (Pi5Neo etc.) installs on the Pi only, during build Phase 6.
2. Read `hardware/docs/wiring.md` and the figures folder end to end before the parts arrive; flag anything unclear in your journal so it gets fixed before build day.

Device

The bedside unit. It displays glucose, runs the alarms, and drives the LEDs and speaker.

Hardware

- Raspberry Pi 5 with the Raspberry Pi Active Cooler, 256GB microSDXC card with Raspberry Pi OS
- Raspberry Pi Touch Display 2 (7 inch, DSI), connected by a 15-pin to 22-pin FFC ribbon cable, with the Pi mounted on its back
- SmartiPi Touch Pro 3 case, Large, holding the display, Pi, and the LED circuit in its 45mm back cavity
- Power: RasTech 27W GaN USB-C PD supply (5.1V 5A) directly into the Pi's USB-C port (full 5A mode). The display powers from the Pi through its included GPIO power cable, 5V on pin 2 and ground on pin 6
- LED U-frame: BTF-LIGHTING WS2812B ECO strip segments (about 28 LEDs cut from the 16.4FT reel) inside Muzata aluminum channels with frosted diffusers, mounted on the case's left, top, and right edges, corners joined with BTF L-shape solderless connectors. Powered by the ALITOVE 5V 5A supply, sharing ground with the Pi. Data from GPIO10 through a SN74AHCT125N level shifter and a 330 ohm resistor (the Pi outputs 3.3V, the strip wants 5V data), with a Rubycon 1000uF 16V capacitor in parallel across 5V and ground at the screw terminal (the same point the strip's power leads connect to)

- Bed-presence radar: HLK-LD2410B-P 24GHz mmWave module, mounted with its antenna face (the side with the flat patch-antenna pattern) pointed at the bed and nothing metal in front of it: not the aluminum LED channels, not the display's metal backplate. Preferred spot: the SmartiPi's camera mount beneath the display, which sits in the open bottom of the LED U-frame, if the 7 x 35mm board fits the cradle (it is about 10mm longer than a camera board); otherwise a 3D-printed LD2410B housing (plain PLA or PETG, never carbon-fiber or metal-filled filament) taped to the front of the case below the screen. Per the datasheet, either leave an open window over the antenna or keep 12.4 to 18.6mm between the antenna and any cover. It must be rigidly fixed, since a radar that wobbles reads its own motion as a person, and kept a few centimeters from the Active Cooler's fan. Powered from the Pi (5V on physical pin 4, ground on pin 9), presence output to GPIO17 (physical pin 11) at 3.3V logic, no level shifter needed. Connected with a 1.25mm-to-2.54mm 5-pin adapter cable; distance calibrated with the HLK Bluetooth app. Range is set in distance gates 0.75m (about 2.5 ft) deep, and the radar stores the setting itself, so it survives power cycles. Two profiles: HOME, the smallest gate that covers the sleeper's chest (usually gate 2 or 3, about 7.5 to 10 ft), and DEMO, gate 1 (about 5 ft) so a crowded room does not keep the device awake
- LIELONGREN USB soundbar speaker, plugged into a Pi USB-A port (audio over USB; the Pi 5 has no headphone jack)
- Mini breadboard and jumper wires for the LED circuit

What Rounds and Night Buddy ask of the hardware. Nothing mandatory; the sponsor tiers read the device, they do not rewire it. Four things to know while building:

- The radar's raw reading is sampled every 30 s during an alarm episode (`hal.get_presence()`) for the AlarmEvent's `presence_during` verdict, so the radar has to detect a motionless sleeper, not just a person walking in; Phase 7's sit-still check is the test that matters.
- The radar DEMO profile (gate 1, about 5 ft) must cover a judge standing at the table during the pairing beat and the Night Buddy beat, since the buddy rung fires only with presence confirmed. The demo panel's "trigger buddy rung" control drives the mock radar on a laptop; on the real device it is the radar.
- `buddy_chime.wav` is a distinct sound for the brokered call, never one of the alarm tones, generated by `make_tones.py` with the other three.
- Optional LED states: `message_waiting` (a soft daytime color when a doctor message or a recall card is waiting; never at night, never during an alarm) and `buddy_alert` (a night pattern by definition, since the buddy rung fires during an alarm; concept-only for the two-device demo). The MockHAL returns None for presence until the demo panel drives it.

Hardware Setup, Step by Step

The complete build, in order, for whoever owns the hardware. Nothing gets glued, taped, or closed up until the step that says so, and nothing gets powered until its checks pass. Phases 1 through 4 make the screen unit work, 5 through 7 build the light frame, 8 and 9 close it up and prove it.

Phase 1: Flash the microSD card

1. On the laptop, install Raspberry Pi Imager and insert the 256GB microSDXC card (through the USB card reader if the laptop has no slot).
2. In Imager choose device Raspberry Pi 5, OS Raspberry Pi OS (64-bit, Trixie / Debian 13), and the card as storage.

3. In the Imager settings before writing: set the hostname (irin), enable SSH, enter the Wi-Fi network name and password, and set a username and password. Write these credentials down.
4. Write, let it verify, then move the card into the microSD slot on the underside of the Pi.

Phase 2: Bench-boot the bare Pi

1. Clip the Raspberry Pi Active Cooler onto the Pi 5 (push pins into the two holes) and plug its small cable into the fan header next to the GPIO pins.
2. Power the Pi with the RasTech 27W supply, nothing else attached. The green activity LED should flicker as it boots.
3. From the laptop on the same Wi-Fi, connect with SSH to irin.local using the username from Phase 1. Run `sudo apt update` to confirm internet works. Note the Pi's IP address. Then run `uname -m`: it must print `aarch64` (the 64-bit image is mandatory because XGBoost, PyNaCl, and pydantic ship prebuilt wheels only for `aarch64`; a 32-bit card fails hours later, the way last year's local model did). The project runs Python 3.14 everywhere: laptops natively, the Pi through `uv` (`curl -LsSf https://astral.sh/uv/install.sh | sh`, then `uv python install 3.14`), because Raspberry Pi OS ships 3.13 and never 3.14 from `apt`.
4. Prove the whole Python stack on the Pi the same day the skeleton commit lands, before any feature exists: `git clone the repo`, `uv venv --python 3.14 .venv`, `sudo apt install liblgpio-dev swig build-essential ffmpeg` (the C library `lgpio` builds against; the `apt python3-* GPIO` packages are for the system 3.13 and are not used), `uv pip install -r backend/requirements.txt -r hardware/requirements.txt` (`gpiozero`, `lgpio`, `spidev`, and `Pi5Neo` or `Blinka` build or install into the 3.14 `venv`), `pytest` from `backend/`, then `IRIN_HW=mock DATASOURCE=replay uvicorn app.main:app --host 0.0.0.0` and open the display page from the laptop. Anything that fails here is a day-one problem instead of a demo-night one; the one planned escape hatch is that the code is written 3.13-compatible (no 3.14-only syntax), so if `lgpio` will not build in fifteen minutes the Pi's backend runs on the system 3.13 with `python3 -m venv --system-site-packages` and `apt's python3-gpiozero`, `python3-lgpio`, `python3-spidev`, and nothing else changes. From then on, every promotion of `dev` to `main` includes `git pull` and `pytest` ON THE PI before the service restarts; a green laptop is not a green Pi.
5. Shut down cleanly (`sudo shutdown now`), wait for the LED to stop, unplug. Never rewire anything while powered.

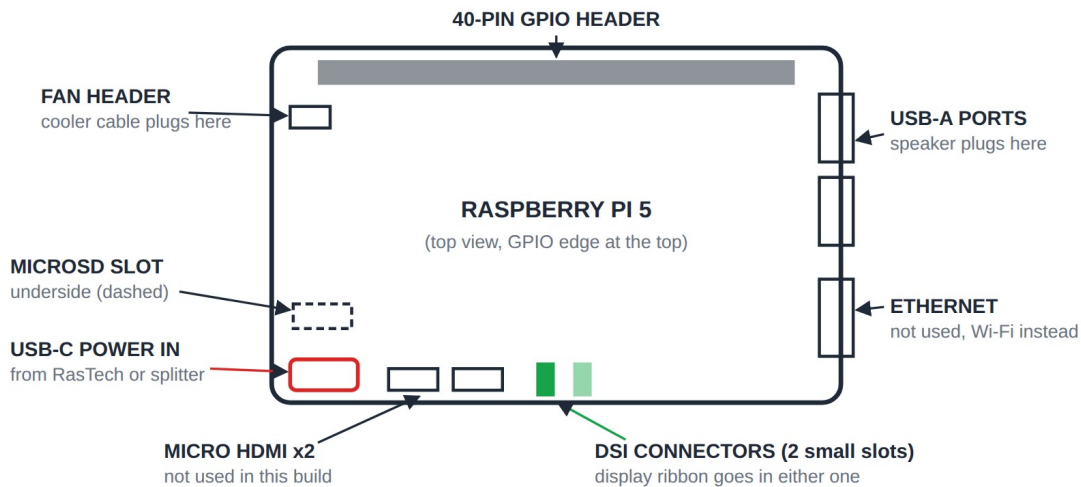


Figure 1. The Pi 5's landmarks: GPIO header, fan header, USB-C power, DSI connectors, microSD slot, USB-A ports.

Phase 3: Mount the Pi on the Touch Display 2

1. Lay the display face-down on something soft. Set the Pi over the display's four standoffs with the Pi's ports facing outward, and screw it down with four M2.5 screws.
2. Connect the DSI ribbon: gently lift the latch on the Pi 5's DSI connector (the small one beside the micro HDMI ports), slide in the ribbon's narrow 22-pin end with the contacts facing as the cable's leaflet shows, and press the latch closed. Repeat at the display's 15-pin connector. Do not force anything; the latch moves a millimeter or two.
3. Display power: plug the included GPIO power cable onto the Pi's header, 5V at physical pin 2 and ground at physical pin 6, and click its keyed plug into the display's power socket.
4. Power up once to verify: the panel lights and shows the desktop, and touch responds. A rotated image is normal. Fix it in Trixie's Screen Configuration tool (Preferences → Screen Configuration), which rotates the image AND the touch matrix together; if touch still lands rotated afterwards, that is the one known time sink here: set a libinput calibration matrix for the touch device (a udev rule with `ENV{LIBINPUT_CALIBRATION_MATRIX}`) rather than rotating inside Chromium. Shut down and unplug.

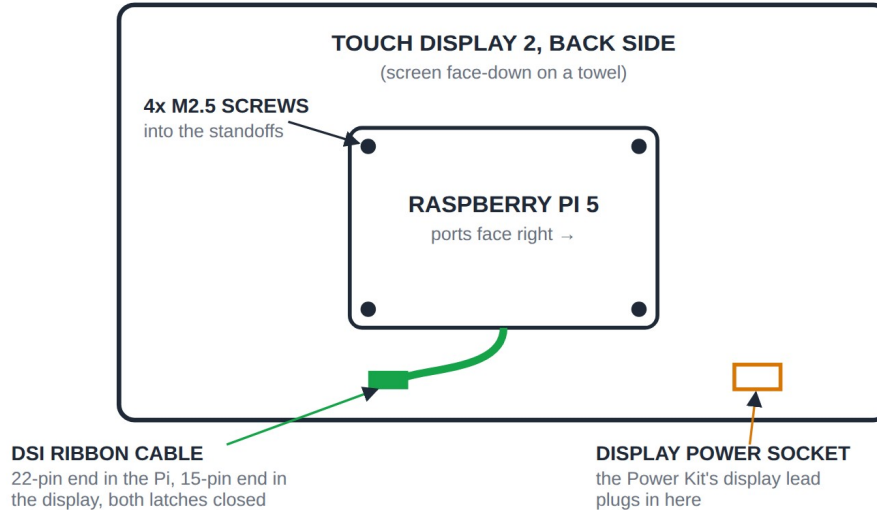


Figure 2. The Pi mounted on the display's standoffs with the DSI ribbon and the display's power socket.

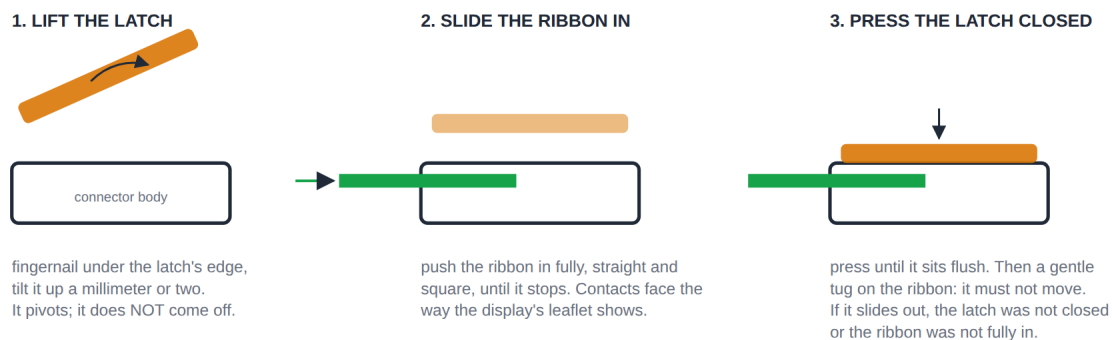


Figure 3. Seating a DSI ribbon in three moves: lift the latch, slide the ribbon in square, press the latch closed, then tug-test.

Phase 4: Install into the SmartiPi Touch Pro 3 case

1. Follow the Smarticase instructions to seat the display-plus-Pi assembly into the front shell and attach the stand.
2. Route the RasTech 27W supply's USB-C cable in through the case opening, directly into the Pi's power port. With no splitter in the path, the Pi negotiates its full 5A mode and shows no low-power notice.
3. Boot once to confirm the Pi and the screen both power up; the screen draws its power through the GPIO cable connected in Phase 3. (Optional alternative: the SmartiPi Display Power Kit splitter can feed both from one panel-mounted input, but it is not needed since the display's own power cable is in hand.)
4. Speaker: plug the LIELONGREN soundbar's USB cable into one of the Pi's USB-A ports. It carries both power and audio; the Pi 5 has no headphone jack, so USB is the only audio path.

5. Make the Pi use it: run `aplay -l` and confirm the LIELONGREN shows up as a USB Audio device, then pick it as the output in the desktop's volume menu (right-click the speaker icon) or set it as the default ALSA/PipeWire device over SSH.
6. Test it: `speaker-test -t wav -c 2` should play voice samples through the soundbar. Set the volume around 80 percent with the soundbar's own buttons and control loudness from software; the alarm code ramps volume itself.
7. Leave the Large back cover off. It goes on in Phase 8, after the wiring.



Figure 4. Optional alternative: the Display Power Kit splitter feeds Pi and display from one panel input. The build's primary path is the display's included GPIO power cable with the RasTech direct into the Pi.

Phase 5: Build the LED U-frame

1. Measure the case's left, top, and right edges with a ruler and write the three lengths down. Cut nothing before this.
2. Cut three Muzata channel pieces to those lengths: channel in the Hanpex miter box, KATA hacksaw in the 90 degree slot (45 degree slots if mitering the corners). Smooth the cut ends with the ValueMax files.
3. Cut the strip: from the 16.4FT reel, keep the factory JST connector end as the start of the left segment. Cut only at the printed cut lines, sizing each segment to the most LEDs that fit its channel, and keep the direction arrows flowing up the left, across the top, down the right.
4. Join the two corners with the BTF L-shape corner sets: clip each strip end into its connector so one segment's DOUT side feeds the next segment's DIN side.
5. Stop. Do not mount anything yet. The frame gets electrically tested in Phase 7 before any adhesive is used.

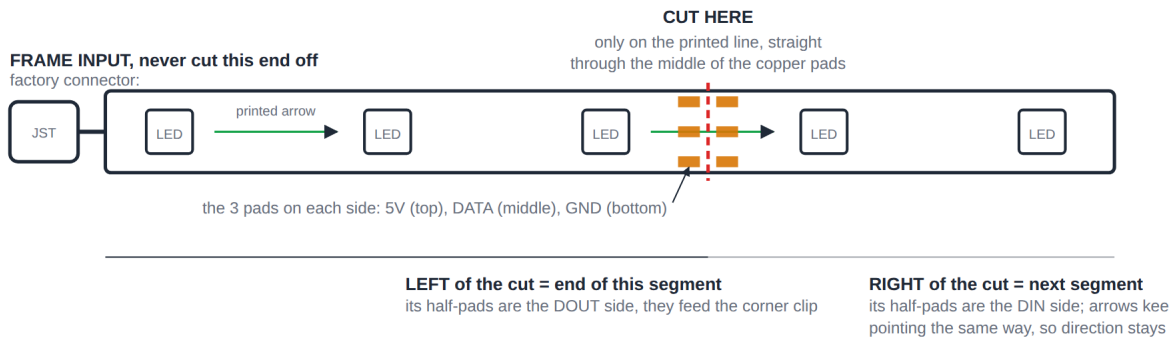


Figure 5. Cutting the strip: cut only on the printed line through the copper pads. The left side's pads become the segment's DOUT, the right side's become the next segment's DIN.

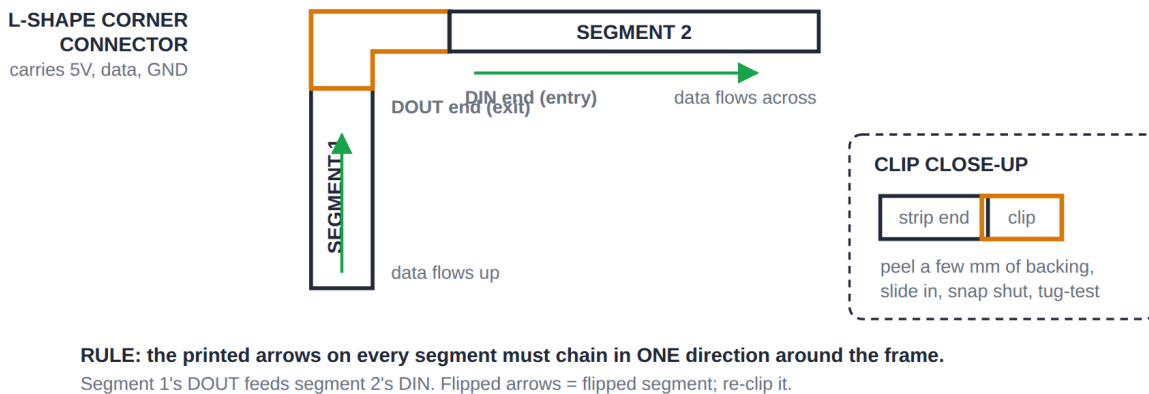


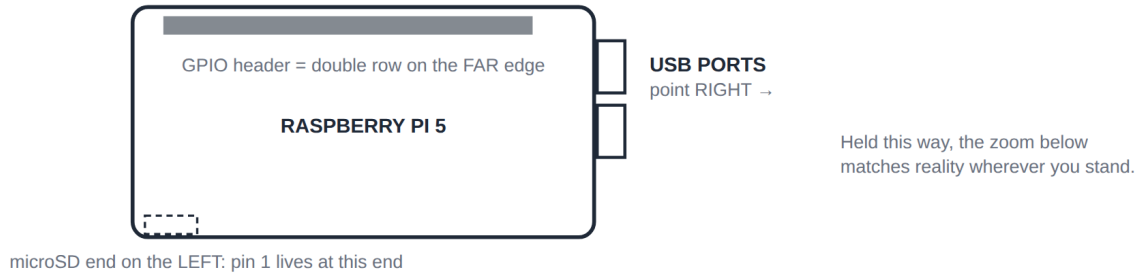
Figure 6. A frame corner: one L-shape connector carries 5V, data, and ground; segment 1's DOUT feeds segment 2's DIN, and the printed arrows must chain one way around the frame.

Phase 6: Wire the circuit on the mini breadboard, pin by pin

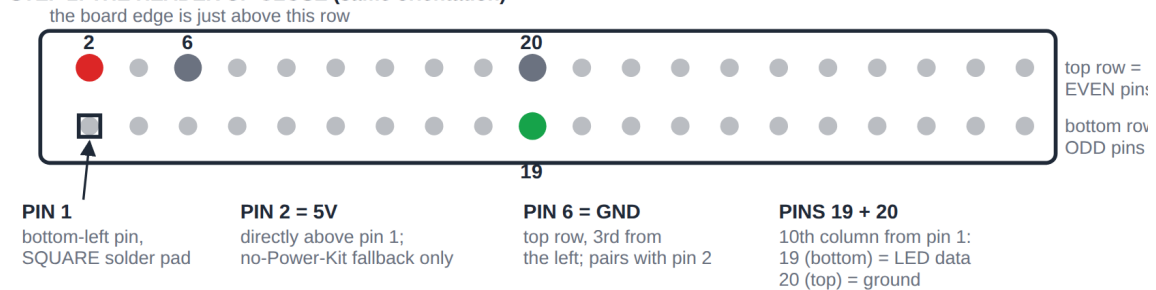
1. Seat one SN74AHCT125N chip across the breadboard's center gap, notch to the left. Pins number counterclockwise from the notch: pin 1 is lower-left, pin 14 upper-left.
2. Chip pin 14 (VCC) to the breadboard's 5V rail. Chip pin 7 (GND) to the ground rail.
3. Chip pin 1 (1OE) to the ground rail. This enables the output.
4. Tie the chip's six unused inputs and enables to ground, or they float, and a floating CMOS input oscillates and runs the chip warm: 2OE (pin 4, col 9 bottom), 2A (pin 5, col 10 bottom), 3A (pin 9, col 11 top), 3OE (pin 10, col 10 top), 4A (pin 12, col 8 top), 4OE (pin 13, col 7 top). The two bottom pins get a jumper each to the ground node (col 16 bottom, or col 12 bottom, which pin 7 already grounds). The four top pins sit on the half whose col 16 is the 5V node, so bridge ground up first: one jumper col 16 bottom → col 15 top, then one jumper from col 15 top to each of cols 11, 10, 8, and 7 top. Never touch col 6 top (pin 14, 5V). Leave the unused outputs (pins 6, 8, 11) open. Seven jumpers in all.
5. Pi data: a female-to-male jumper from the Pi's GPIO10 / SPI MOSI, physical pin 19, to chip pin 2 (1A).

6. Chip pin 3 (1Y) through the 330 ohm resistor to the frame's data lead. The strip's data lead is stranded wire and is the one joint in the build that lives in a breadboard hole. If the pigtail's ends are factory-tinned (BTF's usually are), push the tinned end straight in and tug-test it; that is enough. If the end is bare or frays, do not solder: join it to a solid-core jumper with a Wago 221 lever connector (in a pinch, a tight twist wrapped in electrical tape) and put the jumper in the hole. No soldering iron is needed anywhere in this build: the corners clip and the power leads screw in.
7. Power: the ALITOVE's barrel plug into the green screw terminal. Terminal plus to the 5V rail and the frame's 5V lead; terminal minus to the ground rail and the frame's GND lead. Check the barrel fit: the ALITOVE plug is 5.5 x 2.5 mm and most screw-terminal jacks are 5.5 x 2.1 mm, so the plug can seat loosely and cut out under vibration. If it wobbles, cut the plug off and screw the ALITOVE's bare leads straight into the terminal, after confirming with the multimeter which lead is + (the center conductor).
8. The Rubycon 1000uF capacitor across the terminal's plus and minus. The leg on the can's stripe side is negative and goes to minus. Check this twice.
9. Common ground: a female-to-male jumper from the Pi's ground, physical pin 20 (beside pin 19), to the ground rail.
10. Presence radar: press the adapter cable's small 1.25mm connector onto the LD2410B-P's five header pins (it is keyed and fits one way), then take the three wires that matter to the Pi header: VCC to 5V physical pin 4, GND to ground physical pin 9, OUT to GPIO17 physical pin 11. The OUT pin is 3.3V logic, so it goes straight to the Pi with no level shifter. Identify each wire by the labels printed on the radar board beside the pins (VCC, GND, TX, RX, OUT), never by color, since the adapter's wire colors are not standardized. Tape off the two unused wires (UART TX/RX) and photograph which color went where for wiring.md. If a wire is short, extend it with a male-to-female Dupont jumper and tape the joint. Two checks: the display's GPIO power plug sits on pins 2 and 6, so look at its housing before assuming pin 4 is reachable; if the housing covers pin 4, take the radar's VCC from the breadboard's 5V node (col 16 top) instead, which is fine because grounds are already common. And in presence.py configure GPIO17 with the internal pull-down, so an unplugged or dead radar reads low (no presence) instead of floating.
11. On the Pi, enable SPI (sudo raspi-config, Interface Options, SPI) and install a Pi 5 compatible driver: Pi5Neo or Adafruit Blinka's SPI NeoPixel. The classic rpi_ws281x library does not work on the Pi 5. Same rule for the radar pin: read GPIO17 with gpiozero (its lgpio backend) or libgpiod; the old RPi.GPIO library does not work on the Pi 5 either. Put the service user in the spi and gpio groups so neither driver needs root.

STEP 1: ORIENT THE BOARD. Chips up, big USB ports pointing RIGHT.



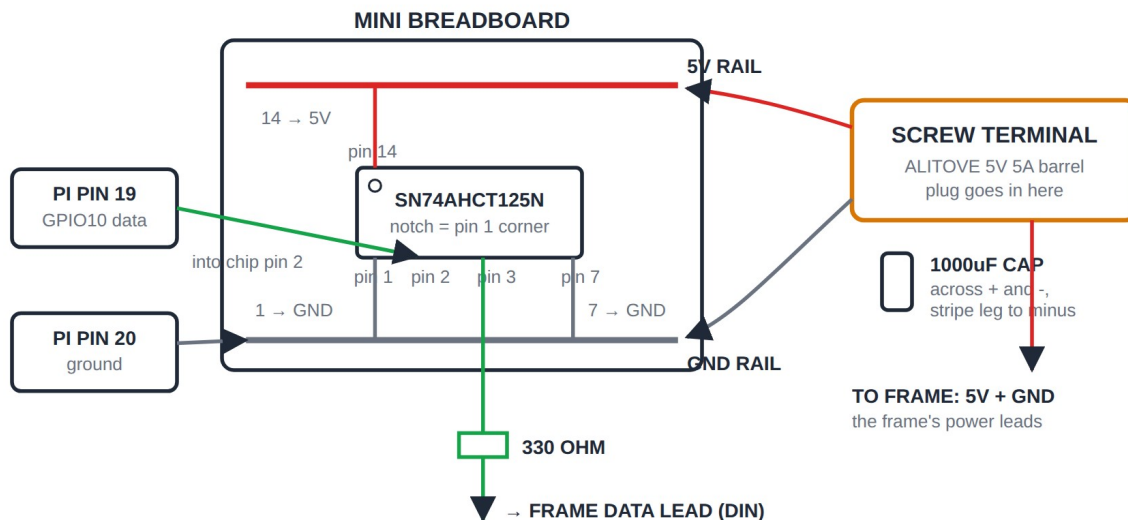
STEP 2: THE HEADER UP CLOSE (same orientation)



DOUBLE-CHECK, WORKS FROM ANY ANGLE:

the one SQUARE solder pad on the underside is pin 1, always. Pin 2 is its neighbor toward the board edge, and pins 19/20 are the 10th column of pins away from pin 1.

Figure 7. Finding the GPIO pins: orient the board with the USB ports pointing right, then pin 1 is the bottom-left pin with the square solder pad, and pins 19/20 are the 10th column from it.

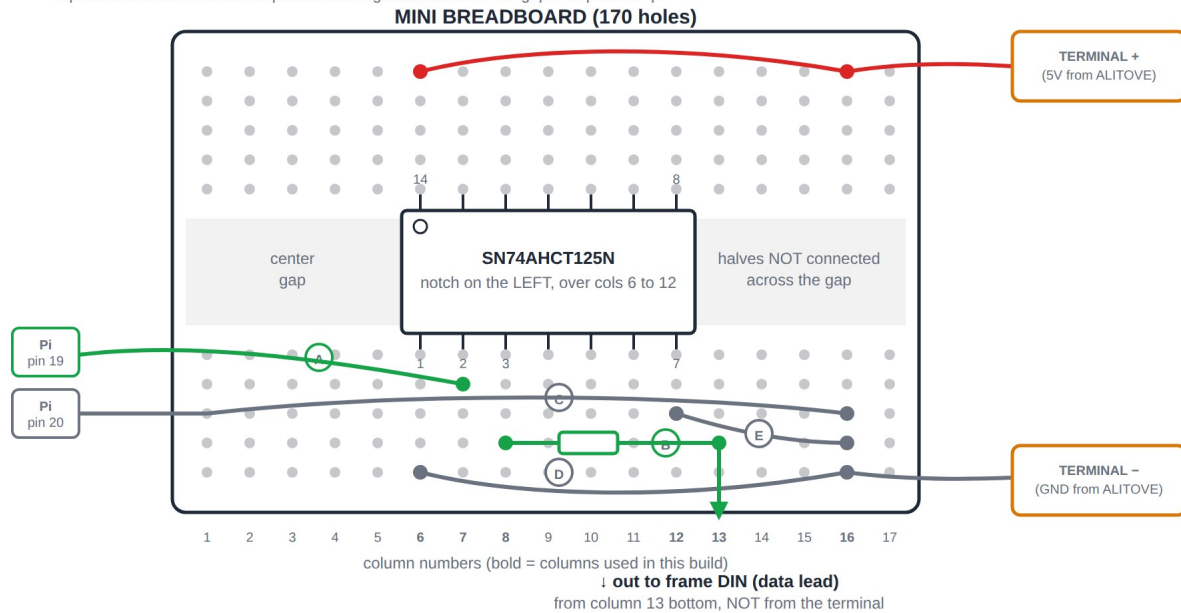


The rails join the terminal's + and -, so the chip runs on the same 5V as the strip. The frame's 5V never touches the Pi.

Figure 8. The circuit at a glance: five chip connections, rails fed by the screw terminal, capacitor across them, Pi pin 20 tying the common ground.

HOW THE BOARD CONNECTS: the 5 holes in one column half are wired together inside.

Top half and bottom half are separate. Nothing crosses the center gap except the chip.



WHERE EACH WIRE GOES (solid dots mark the exact holes):

RED: terminal + → col 16 top, then a jumper col 16 top → col 6 top (chip pin 14). GREEN: Pi pin 19 → col 7 bottom; resistor col 8 bottom → col 13 bottom; frame DIN also taps col 13 bottom. GRAY: terminal - → col 16 bottom; jumpers col 16 bottom → col 6 bottom (pin 1) and col 16 bottom → col 12 bottom (pin 7); Pi pin 20 → col 16 bottom.

A wire never shares a hole with a chip pin: same column, different hole. Any free hole in the right column half works. Lines that cross WITHOUT a dot are not connected; only the solid dots are real hole connections.

WIRE TAGS: A = Pi pin 19 data in · B = 330 ohm resistor run · C = Pi pin 20 ground · D = ground jumper to chip pin 1 · E = ground jumper to chip pin 7

Figure 9. The same circuit hole by hole on the mini breadboard. Solid dots mark exact holes; the wire tags A to E are named in the legend; lines crossing without a dot are not connected.

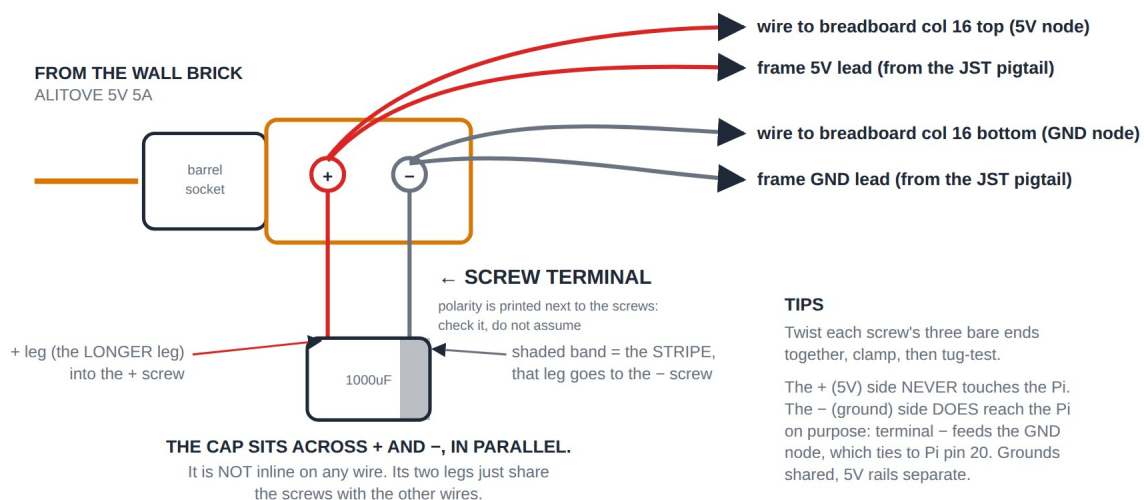


Figure 10. The screw terminal: three conductors per screw, capacitor in parallel across + and -, stripe leg to minus.

Phase 7: First light

1. Pre-power checks: the frame's 5V connects only to the screw terminal, never to the Pi; capacitor stripe to minus; chip notch matches the pin numbering used. Then, with the ALITOVE plugged in and nothing else on the terminal, put the multimeter across the two screws: about +5.0 V on the screw marked +. A negative reading means the adapter or plug polarity is wrong; fix it before a single LED or chip pin sees power. No multimeter? Build the kit's polarity tester: an LED with a 330 ohm resistor in series across the two screws, long leg on the + screw; it lights only if + is really +. Borrow a meter anyway for the rest of the build: it is the only way to see 5 V at the far end of the strip or to find a bad corner connector.
2. Plug in the ALITOVE, boot the Pi, and run a test script lighting the first 3 LEDs at 10 percent brightness.
3. Run a full-frame color cycle at 30 percent. Wiggle both corner connectors while it runs; any flicker means reseal that connector now.
4. Only after a clean wiggle test: stick each segment into its channel, snap on the diffusers, cap the outer ends (notch the bottom-left cap for the input wire), leave the corner ends open, and mount the channels to the case's left, top, and right edges with 3M 5925 tape. Dry-fit and pencil-mark first; the tape is one-shot.
5. Radar check: with the radar wired, watch GPIO17 with `watch -n 0.5 pinctrl get 17`` (the Pi 5 uses `pinctrl`, not `raspi-gpio`) while someone sits still in front of the device, then leaves the room. The pin should hold high with a person present (even motionless) and drop low within the configured hold time once the room is empty. Then pair the HLK Bluetooth app and set the distance gates for the HOME profile: measure from the nightstand to where your chest is lying down, and pick the smallest max gate that covers it (gate n reaches $(n + 1) \times 0.75\text{m}$; gate 2 = 2.25m, about 7.4 ft; gate 3 = 3.0m, about 9.8 ft). Stopping at the bed rather than the hallway keeps traffic outside the door from counting as presence. Set the no-one duration (how long after the last detection it reports empty) to a few seconds; the backend applies its own longer Away rule on top.

Phase 8: Close it up

1. Stick the mini breadboard and screw terminal inside the Large cover's 45mm cavity with tape or adhesive pads.
2. Mount the radar with its antenna face (the side with the flat patch-antenna pattern) pointed at the bed and nothing metal in front of it: not the aluminum LED channels, not the display's metal backplate. Preferred spot: the SmartiPi's camera mount beneath the display, which sits in the open bottom of the LED U-frame, if the 7 x 35mm board fits the cradle (it is about 10mm longer than a camera board); otherwise a 3D-printed LD2410B housing (plain PLA or PETG, never carbon-fiber or metal-filled filament) taped to the front of the case below the screen. Per the datasheet, either leave an open window over the antenna or keep 12.4 to 18.6mm between the antenna and any cover. It must be rigidly fixed, since a radar that wobbles reads its own motion as a person, and kept a few centimeters from the Active Cooler's fan. Before anything is permanent, stick it down temporarily and rerun the Phase 7 sit-still-then-leave check at the display tilt you will actually use; also check that the case's steel base does not reflect into the lower part of the radar's ± 60 degree cone.
3. Route the frame's input wire around the case's bottom-left edge into the cavity through the removable side panel, or through one drilled 6 to 8mm hole in the plastic back cover with a dab of hot glue as a grommet. Never drill anywhere but the plastic back cover.
4. Bring the ALITOVE barrel cable and the RasTech cable in through the panel opening, and plug the speaker's USB into a Pi USB-A port.

5. Fit the Large cover, checking no wire is pinched and the cooler's fan is unobstructed. Zip-tie the three cords together where they exit.

Phase 9: Full system test

1. On wall power: boot to the desktop, start the kiosk by hand with `deploy/kiosk.sh --now` (autostart is enabled only after the final audit), play a speaker test tone, confirm the frame shows ambient color. Also confirm three things a fresh Pi OS gets wrong: the screen is still lit after 15 minutes idle (blanking off), `timedatectl` reports the clock synchronized (there is no RTC battery, so the Pi only knows the time after NTP), and the alarm audio played from the service itself, not just from a terminal.
2. Trigger a fake low from the demo control panel: the predicted-low alarm fires, the frame ramps to bright red, sound escalates, and acknowledging from the phone silences it within a second.
3. Optional, only if the battery path is ever used: repeat the same test on the UGreen power bank: bank USB-C into the splitter, bank USB-A through the barrel cable into the screw terminal, LED brightness capped in config.
4. Before the hackathon, pack directly from this working state.

Common failure modes

- LEDs flicker or show random colors: almost always a missing common ground or a data wire that is too long
- First LED wrong, rest fine: data voltage marginal, check the level shifter wiring
- Pi reboots when LEDs go bright: the strip is drawing power through the Pi somewhere, or the Pi's own supply is weak
- Nothing lights: data plugged into the DOUT end of the strip instead of DIN, or the wrong GPIO pin for the library in use
- Chip warm or LEDs erratic with a clean data path: the unused 74AHCT125 inputs are floating; ground pins 4, 5, 9, 10, 12, 13 (Phase 6)
- Kiosk black at night: screen blanking was left on. Silent alarms or "device busy" from the service: the backend is running as root outside the kiosk user's PipeWire session (see Software on the Pi)

Software on the Pi

- FastAPI backend started as a systemd service on boot
- Chromium in kiosk mode on boot, pointed at localhost, showing the display UI. Autostart stays DISABLED until the whole project is complete and the final audit has passed: the Pi boots to the normal desktop for the entire build, the display page is opened in a normal Chromium window, and `deploy/kiosk.sh --now` shows the real kiosk on demand (Alt+F4 closes it). Chris enables `kiosk.service` by hand as the last step before judging, never a Claude and never `install.sh`. The unit cannot trap anyone: it refuses to start while `/boot/firmware/NO_KIOSK` exists (touch the flag from a text console with Ctrl+Alt+F2, over SSH, or from any computer with the SD card in a reader, then reboot), it waits 20 seconds before Chromium appears, it never respawns (`Restart=no`), and SSH from the laptop works underneath it at all times.
- SQLite database for the readings cache, logged treatments, settings, alarm history, and stored reports
- LED control through Pi5Neo or Adafruit Blinky SPI NeoPixel (`rpi_ws281x` does not work on the Pi 5), sound through `aplay` or `pw-play` (no `pygame`)

- The trained ML model file, loaded at startup
- Screen blanking OFF: `sudo raspi-config nonint do_blanking 1` (Display Options → Screen Blanking → No in the menu). Night mode dims through the backlight, never through blanking; the kiosk must be lit at 3 AM. Verify by leaving it idle for 15 minutes.
- Architecture: the Pi is aarch64 and the laptops are not, and nothing compiled crosses that line. Python 3.14 everywhere (the Pi through uv; Phase 2), with the code kept 3.13-compatible as the escape hatch. There is no local LLM (the narrative chain is Backboard → Meta → Claude → template, the first three over HTTPS, with pre-generated fallbacks); the forecaster is XGBoost loaded from JSON with the same pinned version as training, never a pickle; every backend dependency has a prebuilt aarch64 wheel for cp314 or abi3, or builds from an sdist in the day-one proof (backend/requirements.txt includes anthropic, backboard-sdk, openai, elevenlabs, httpx, and matplotlib); and the stack is installed and pytest-ed on the Pi on day one (Phase 2).
- Backlight for Night mode: hal writes `/sys/class/backlight/<name>/brightness`. Install a udev rule that makes that file group-writable by video (`SUBSYSTEM=="backlight", RUN+="/bin/chgrp video /sys/class/backlight/%k/brightness", RUN+="/bin/chmod g+w /sys/class/backlight/%k/brightness"`) so the service never needs root.
- The backend runs as the kiosk user, not root: `irin.service` is a `systemd --user` unit for that user with `loginctl enable-linger on`, so alarm audio shares the user session's PipeWire with Chromium. A root system service gets "device busy" or silence. The user is in the `spi`, `gpio`, `audio`, and `video` groups.
- GPIO on the Pi 5: `gpiozero` (lgpio backend) for GPIO17 and Pi5Neo or `Blinka` for the SPI NeoPixels. `RPi.GPIO` and `rpi_ws281x` both fail on the Pi 5.
- No RTC battery is fitted, so after a power cycle the Pi 5 only knows the time once NTP syncs. ALWAYS boot with the network up (hotspot on before the Pi). Staleness is measured on the monotonic clock since the last new reading, never as a wall-clock difference, and the scheduler holds the night window, basal nudge, ledger, and morning report until `timedatectl` reports synchronized, while the display shows a "clock not set" badge.
- Sound files are generated, not sourced: `backend/sounds/make_tones.py` writes `alarm_soft.wav`, `alarm_urgent.wav`, `chirp.wav`, and `buddy_chime.wav` as distinct tones, so the driver has real audio from day one.

Functional Requirements

1. Poll the data source every 60 seconds. The source is swappable, Nightscout (live) or the replay engine (demo), and nothing downstream knows which is active.
2. Cache every reading in SQLite. On data source failure, serve the cache and mark it stale.
3. **Mode switch.** Live (Nightscout) and demo (replay) are runtime modes behind the one datasource interface, switched by a PIN-gated button on the demo panel. Switching resets alarm state and the clock multiplier; demo mode shows a DEMO badge on every screen; injection endpoints exist only in demo mode; and treatments logged in demo mode are never posted to the real Nightscout.
4. Display states, chosen by time of day and alarm state:

State	Shows	When
Detail	Big number, trend arrow, 3-hour graph with target band and dotted forecast line, IOB, minutes since last dose,	Daytime default

State	Shows	When
	today's TIR	
Night	Dim clock with a smaller number	Configured quiet window
Morning	Overnight low and high with times, time below and above range, TIR rings	2 hours after the night window ends
Alarm	Full-screen takeover with the value and instructions	Any active alarm

5. **Alarm engine.** A state machine with two distinct tiers, because an alarm's intensity should carry information: the WARNING tier (predicted low) says you have about 30 minutes, the FULL tier (actual low) says you have none. If both sounded identical, waking up would tell you nothing about urgency, and the brain would learn that the red alarm sometimes means nothing is wrong yet, which is how alarm fatigue starts. Keeping the full tier's meaning pure means that when it fires, it is never a drill.
 - Triggers: actual low below threshold (FULL tier), predicted low with $N = 2$ consecutive agreeing forecasts (WARNING tier), actual high (one-shot notification, see below), stale data (distinct chime plus on-screen banner)
 - **Warning tier (predicted low):** amber LEDs, a softer distinct tone, volume starting low and ramping patiently. A gentler START with the same CEILING, never a weaker alarm: unacknowledged, it escalates to the full tier after 5 minutes, or instantly the moment the actual value crosses the threshold, whichever comes first, so it cannot be slept through indefinitely and a prediction that comes true always ends at full intensity.
 - **Full tier (actual low):** red LEDs at full bright pulse, the main alarm tone. Unacknowledged: louder plus LED strobe after 5 minutes. Acknowledged: silenced now, but re-arms if still below threshold 15 minutes later, and every 15 minutes until back above.
 - **Acknowledgment rules (the asymmetry is deliberate):** acknowledging a WARNING silences it for that episode with no re-arm and no escalation, because a prediction is advisory: once you are awake and informed, nagging about a forecast adds nothing, and the actual-low alarm remains fully armed as the backstop. Acknowledging a FULL alarm silences the noise but keeps the re-arm, because ack proves you are awake, not that you are safe; the person who acks at 54 and passes back out is exactly who the device exists for. Acknowledging a warning NEVER suppresses the actual-low alarm: the two tiers are separate alarms in the state machine, not one alarm at two volumes.
 - **Episode:** a warning acknowledgment covers one episode, not the night. The episode ends once the forecast has been back above the threshold for 2 consecutive readings; a later fresh crossing is a new episode and warns again. Without this, a 10 PM ack could silently eat a 3 AM warning.
 - **Transitions:** Idle $\rightarrow$ Pending on the $N = 2$ predicted crossing. Pending $\rightarrow$ Active on actual threshold crossing or 5 unacknowledged minutes. Pending $\rightarrow$ Idle on warning ack, or when the forecast recovers for 2 consecutive readings. Active $\rightarrow$ Acknowledged on ack. Acknowledged $\rightarrow$ Re-armed if still below threshold after 15 minutes; Re-armed behaves as Active. Acknowledged/Re-armed $\rightarrow$ Idle once the actual value is back above threshold. No transition skips acknowledgment on the full tier.

- **Highs (deliberately not an alarm):** a high is measured in hours and treated with a correction dose, not juice in 90 seconds, so a continuously escalating high alarm would mostly train the user to mute the device. Crossing the high threshold produces a ONE-SHOT: single chime, amber ambient tint, and an on-screen indicator that persists until the value recrosses. No acknowledgment required, nothing re-arms. High behavior is a user setting, including an optional “remind me if still above X after N hours” repeat, OFF by default, which covers the one case a one-shot fails: a missed bedtime bolus leaving an 8-hour high behind a single slept-through chime. Highs respect quiet hours; only lows override them.
 - **Concurrency and priority:** actual low outranks predicted low outranks stale outranks high. A higher-priority trigger replaces a lower one immediately and does not inherit its acknowledgment (an acked prediction does not pre-ack the real low). Stale data during any low alarm keeps the low alarm sounding and adds the stale banner; it never silences it. Stale ack silences the chime until data returns, then clears on its own.
 - **Named test:** in replay, acknowledge a predicted-low warning, let the scenario cross the actual threshold anyway, and assert the full alarm fires. That single test in `backend/tests/test_alarm.py` guards the whole tiered design.
 - **Context rules:** a meal logged in the last 30 minutes softens a predicted low. Rapid insulin without carbs raises drop sensitivity for 2 to 3 hours. Evening exercise tightens overnight vigilance. Basal unlogged past its usual time is flagged on screen and in the report.
6. **Basal reminder.** A nudge appears on the display and web app at the configured time. The visual escalates after 1 hour, an email notification goes out after 90 minutes, and it resets daily.
 7. **IOB.** Computed from logged rapid-acting doses with a duration-of-action decay curve, 4 hours by default, configurable.
 8. **LED behaviors.** Ambient range color at low brightness by day, off at night, amber ramp-up on a predicted-low warning, red full bright pulse on an actual-low alarm, strobe on escalation. Colors are user configurable, but warning and full tiers must remain visually distinct.
 9. **Sound.** Configurable alarm tone with volume that starts moderate and ramps up. Low alarms always sound, regardless of quiet-hour settings.
 10. **Morning report.** Compute overnight stats, render the graph to PNG with matplotlib, get the LLM summary, send the email, store the full report in SQLite, and show the morning screen.
 11. **Presence mode.** The PRIMARY input is the bed-presence radar: any detection sets Home instantly; Away requires sustained no-presence (about 15 minutes, timed on `clock.py` like the alarm timers, so demo replay speed shortens it to seconds) and only outside the night window. The FALLBACK and override is the manual Home/Away toggle in the phone app, which always wins over every automatic input. Optional supplements (an iOS Shortcuts geofence calling home/away endpoints, Wi-Fi or router presence detection) may add evidence but are not required. When Away, the device suppresses room outputs only: LEDs off, speaker silent, display dimmed to a clock. Alarm logic keeps running and everything still records; the phone app (via the tunnel) and the phone's own CGM alarms are the away path. Returning re-enables outputs immediately, so a still-active alarm sounds at full volume the moment presence is detected. Evidence stays asymmetric because false Away is dangerous while false Home is merely annoying: strong evidence to leave, any evidence to return, and radar absence alone NEVER sets Away during the night window — at night the device assumes Home unless the user set Away manually.
 12. **Recovery.** On boot, restore state from SQLite, resume the kiosk, and never re-fire acknowledged alarms. On boot with no network, show cached data marked stale rather than a blank screen. There is no RTC battery: on a boot with no network the clock is whatever was saved at shutdown,

so the scheduler treats time as unknown until NTP syncs, staleness is judged on the monotonic clock, and the demo rule is simply to bring the hotspot up before the Pi.

13. **Rounds on the device (Impiricus).** The phone app carries all input; the display glances and alarms and carries no forms. Rounds reads the alarm engine through one hook and adds a handful of glance surfaces; it never edits alarm.py.
 - **The AlarmEvent recorder** observes alarm.py through its one on_transition hook and writes one event per episode: tier (predicted_low / actual_low / stale / high), started_at, acknowledged_at, ack_source (device / app), escalated, rearm_count, crossed_actual (a predicted-low episode that never crossed is a near-miss), presence_during, is_demo. presence_during comes from the raw radar sampled every 30 s of clock time: home if a majority of samples show detection or any detection falls within 2 minutes of alarm start; away if zero detections; unknown otherwise (undriven mock, hal error, brain_only). Never the Home/Away state, which the night rule pins to Home. The recorder never calls into alarm.py.
 - **Physical acknowledge.** A phone across the room at 2 AM is the failure case the device exists to prevent, so the kiosk gains one big Acknowledge during an alarm, the one verb that may use the PIN cached in localStorage; the event records which surface acknowledged. Every other mutating verb on the kiosk (pairing confirm, doctor-message confirm and decline) is in FRESH_PIN_ENDPOINTS and re-prompts the touch keypad every time.
 - **Glance surfaces.** A passive chip ("2 questions waiting in the app"); the step timeline strip during a watch; the doctor-message confirm screen ("Dr. Patel: basal 22 units from tomorrow. Confirm?", the units spoken out loud) with Confirm and Decline, which never auto-confirms and never dismisses on its own; the ketone-prompt banner when a watch has ketone prompts on; and the "Share with my doctor" QR screen reached from settings, showing the QR, then the four digits, then a fresh-PIN Confirm. No forms anywhere.
 - **Timing and transport.** The ledger runs at the night-window end on clock.py and waits for clock_synced like the morning report; the programs evaluate at 07:05 and red rules run on every new reading; catch-up derives what should have run from plan dates, night dates, and stored card IDs, so a reboot or a replay seek never duplicates a card; the device posts sealed cards outbound and polls for doctor messages (60 s live, 5 s demo); no new port opens on the Pi.
 - **Step-week vigilance (last, optional).** For 7 days after each step-up the predicted-low threshold rises by 10 mg/dL (70 → 80) through the settings path only; it never touches the actual-low alarm and expires on its own. It is the only Rounds code that reaches the alarm threshold path.
14. **Night Buddy on the device (Meta).** One more observer on the same hook, one independent timer, one new sound.
 - **The buddy rung** fires when a full alarm (Active or Re-armed) has been unacknowledged for 10 minutes after the actual-low crossing (config, 10 to 12) AND the recorder's running presence verdict for the episode is home; it never fires when the radar says away. On fire it seals a BuddyAlert to the buddy's key, posts it to the relay, and opens a hub listing if hub_watchable is on. After an acknowledge that is still low 15 minutes later, the rung re-climbs with the re-arm. Resolution is two consecutive readings above the low threshold or an acknowledge.
 - **The emergency script** has its own timer on clock.py: T+20 after the crossing it fires regardless of claims, calls, or statuses (demo tier: a stored event, a WebSocket line, and the banner "emergency contact alerted (simulated)"). Nothing on the buddy or hub path can delay it, quiet a local alarm, or gate anything; the named test in tests/test_buddy.py asserts all three.

- **Chime, opt-ins, treating.** The Pi's relay poll also carries brokered "call" events, which play `buddy_chime` through `hal`, a sound used for nothing else; the four opt-ins (`have_buddy`, `be_watcher`, `hub_watchable`, `hub_volunteer`) are separate Settings, all off by default and PIN-gated; "I'm treating" is one POST from the phone app's one-press button, relayed as `treating_set`.
 - **Brain-only and demo.** `brain_only` has no radar, so its condition is stricter: a confirmed level 2 low sustained 10 minutes with zero phone interaction, and the alert and listing are labeled unconfirmed. In demo mode alerts and listings are `is_demo` and reach only demo pairings and the demo hub.
15. **Family Story on the device (Meta).** A hook on the morning-report job and one chip on the morning screen.
- **The send hook** builds one `FamilyStory` per active recipient: `pending_approval` for a first story or an approve-each recipient (`family_story_pending` puts Send and Skip on the morning screen and the phone app, PIN), otherwise sent over SMTP with an unsubscribe line and, for Level 2, the family-view link.
 - **The morning screen** shows "Sent to Mom," one chip per recipient sent, with one-tap Pause; paused and revoked recipients are skipped by the hook. Demo-mode stories are badged DEMO, rendered on the morning screen only, and never reach SMTP.

Machine Learning

Predicts glucose 30 minutes ahead, so alarms fire before the low happens rather than during it. Also writes the morning report in plain language.

Forecast Model

- **Framing.** Regression on the 30-minute-ahead value, because the display needs the dotted forecast line anyway and the alarm falls out for free when the prediction crosses the low threshold. The model predicts the CHANGE ($\Delta = \text{glucose at } t+30 - \text{glucose now}$) and the current value is added back for display; predicting absolutes lets a model score well by echoing the current number, while predicting the delta forces it to learn dynamics, which is the entire job.
- **Features.** About 15, all computable from the last 60 minutes: current value; lagged values at 5, 10, 15, 30, and 60 minutes back; rate of change over the last 5, 15, and 30 minutes; acceleration (change in the 5-minute rate between now and 15 minutes ago, the signature of falling-and-falling-faster); rolling mean, standard deviation, and minimum of the window (the std lets the model discount jittery end-of-sensor-life stretches on its own); time of day as $\sin/\cos$ of the hour plus a night-window flag. No insulin or carb features in v1: the historical export has almost no logged treatments, and at runtime the meal and IOB context is applied as backend rules on top of the raw prediction, not inside the model.
- **Model.** XGBoost or scikit-learn GradientBoostingRegressor, compared against the baselines below. Both are reported, and if the model does not beat the trend baseline at the event level, the baseline ships. Whichever wins, the artifact that ships is XGBoost JSON (or the linear baseline as plain coefficients in JSON); a scikit-learn pickle never ships, because pickles break across Python and library versions and the Pi is a different architecture from every laptop that trains.
- **Training data.** Chris's Dexcom Clarity export: 8 files covering 2024-12-05 to 2026-09-20 (about 22 months), 169,202 readings after dedup, roughly 132 low events (58 overnight). Two multi-week

holes (mid-Feb to late Mar 2025 and two weeks in mid-Jun 2025) are periods on a different CGM brand; treat them as ordinary gaps and do not import that data (different cadence and sensor bias for a handful of extra events).

ML Method (George's checklist, specific enough to hand to a Claude)

The order below is the build order. Baselines are evaluated BEFORE the model exists; that is what makes every later number mean something. Each step names its check.

Step 1: Cleaning (ml/clean_clarity.py).

1. Parse only rows with Event Type EGV; discard the metadata header rows (they contain name and date of birth, one more reason the raw files live only in the gitignored ml/data/ and never touch git).
2. Concatenate all 8 files and deduplicate on timestamp FIRST. The exports overlap (files 2/3 share two months, 4/5 share a day); without dedup those months are silently double-weighted.
3. Map the literal strings "Low" to 39 and "High" to 401. Dexcom caps at the sensor's range, so "Low" means below 40; these rows are the deepest lows in the set, keep them.
4. Spike filter, the ONLY value-based cleaning: drop reading $x(t)$ when $|x(t) - x(t-1)| > 30$ AND $|x(t+1) - x(t)| > 30$ AND the two moves have opposite signs. In words: the value jumped more than 30 mg/dL away from its neighbor and immediately jumped more than 30 back. Real glucose tops out near 4 mg/dL/min (20 per reading), so one 30+ move is suspicious and two opposite ones in 10 minutes is electronics, not biology. The opposite-signs clause is what protects real lows: in a genuine crash the second move continues down, so it never fires. Worked example: 118, 121, 74, 119 drops the 74 ($74-121 = -47$, $119-74 = +45$, opposite signs); 118, 90, 58, 52 keeps the 58 ($52-58 = -6$, same direction, fails). The dropped reading becomes a HOLE; never interpolate a replacement, that would invent a reading the sensor never produced.
5. Do NOT auto-delete suspicious lows (e.g. nighttime compression lows, the steep V from lying on the sensor). They are not reliably distinguishable from real fast lows, real low events are the scarcest thing in the data, and the live sensor will produce compression lows too, so the model should have seen them. The label is what the sensor will say, not true blood glucose, and what the sensor says is also exactly what the deployed device alarms on.
6. A gap is any interval over 30 minutes between consecutive readings (including the two multi-week CGM-brand holes). Check: cleaned reading count within a few hundred of 169,000, and a printed list of gaps that matches the known sensor-change pattern.

Step 2: Dataset builder.

1. One row per reading, but only when the full 60-minute feature history AND the $t+30$ label both sit inside a single gap-free stretch. Anything spanning a gap is dropped, the training-side mirror of safety invariant 1.
2. Label = $\text{glucose}(t+30) - \text{glucose}(t)$. Features as listed under Forecast Model. No normalization needed for tree models; keep raw mg/dL.
3. Check: row count in the low hundred-thousands, no row whose window crosses a printed gap, and feature columns free of NaN.

Step 3: Evaluation design.

1. Split by TIME, never randomly: train on everything up to 8 weeks before the last reading, hold out the final 8 weeks (about 11 low events) untouched. A random split puts readings 15 minutes apart

on opposite sides and the model scores brilliantly by memorizing nights it half-saw; this is the single most common way this eval lies.

2. Evaluate two baselines BEFORE training anything: (A) persistence, predict no change; (B) linear extrapolation of the last 15-minute trend. B is honestly decent at 30 minutes. If XGBoost does not clearly beat B at the event level, fix features before touching hyperparameters. This also pre-answers the judge question of why ML beats a simple trend rule, with a measured number.
3. Point metrics (sanity only): MAE overall, and MAE on the subset where the true value is under 100, because a model can have a lovely overall MAE while being useless exactly in the range that matters.
4. Event metrics (the real eval): replay the held-out weeks through the actual alarm rule (prediction crosses threshold, $N = 2$ consecutive agreeing predictions) and score EVENTS: detection rate (of the held-out lows, how many got a warning at least 10 minutes before crossing 70), median lead time on detected ones, and false alarms per night. These three numbers are the demo claim.
5. Sweep the alarm threshold (predicted 70 / 75 / 80) and plot detection rate against false alarms per night. Choose the operating point keeping false alarms under about one per two nights: Chris goes low 1.4 times a week, and a device that false-alarms nightly gets unplugged within a week, at which point its detection rate is zero. Alarm fatigue is the failure mode that kills real CGM alarms; being able to say which point on the curve was chosen and why is worth more than any accuracy number.
6. Human review of eval events only: generate a one-page plot per held-out low event, and Chris (who wore the sensor) marks any obvious artifact, a steep midnight V with instant full recovery and no treatment. Flagged events leave the detection-rate denominator and are noted in metrics.md. Training data stays untouched; only the number told to judges is verified against lows Chris will vouch for.
7. ml/models/metrics.md records exactly: the split dates, the three event-level numbers, the baseline comparison, the chosen threshold with its position on the sweep curve, the feature list, and any events excluded by review. That file is the committable artifact and the Q&A sheet at judging.

Guardrails for the Claude running this: no future information in any feature; dedup before everything; never interpolate; never train or forecast across a gap; raw CSVs never leave ml/data/, and raw rows never enter Claude's context: scripts print summaries only. Features are computed by ONE shared function (ml/models/features.py) imported by both the dataset build and predict.py, so training and the Pi can never compute a feature differently (train/serve skew). The whole pipeline runs in about two minutes, so Claude runs it in-loop and stops at each checkpoint for George's approval; only a step slower than a minute (a hyperparameter search) is run by George in a terminal.

Rounds Classification and Metrics (ml/models/nights.py)

The Rounds equivalent of features.py: every reason code and every per-night and per-window number on a card is computed in ONE place, ml/models/nights.py, imported by Chris's ledger, Standing Cards, and Step Watch and by George's validation scripts, so no metric is ever implemented twice and none can drift between validation and serving. Pure functions, numpy only (every metric is arithmetic over readings; the Pi's backend gains nothing), no I/O. Five functions, and ml/tests/test_nights.py reproduces the worked examples by hand.

- **classify_night(readings, treatments, alarm_events, presence, night_window).** One or more reason codes per night and their source: late_meal (carbs within 3 h before night start or during the night), late_correction (bolus within 3 h before night start, so IOB above zero at night start), basal_late /

basal_missed (basal logged more than 60 min after the usual time, or not logged), exercise (logged after 17:00), treated_low (any low alarm followed by a rise above about 60 mg/dL within 2 h), stale (coverage under 85%: $9 \text{ h} \times 12 = 108$ expected, $0.85 \times 108 = 91.8$, so 92 readings or more is adequate), away (the presence toggle set to Away), clean (none of the above). Logged on a patient who logs; on history and on a Brain-only patient late_meal is inferred (a rise faster than about 2 mg/dL/min in the first 2 h of the night), treated_low is inferred (a reading under 70 followed by a rise above 60 within 2 h), stale is fully available, and basal_late, exercise, and away are “context unavailable”, never assumed clean.

- **night_metrics(readings, alarm_events, night_window).** Overnight rise (glucose at 07:00 minus glucose at night start + 2 h, each a 15-minute median), dawn rise (03:00 to 07:00), nightly low point (minimum of the 15-minute rolling median), TBR (under 70, full day and overnight), nocturnal burden (minutes under 70 and the area under 70 in mg/dL-minutes), level 2 lows (under 54 confirmed by 2 consecutive readings), near-misses (predicted-low warnings acknowledged or recovered with no crossing within 60 min), ketone-risk episodes (at or above 200 for at least 120 min), coverage. Stale-flagged readings never count toward coverage.
- **low_events(readings, treatments, alarm_events, night_window).** One LowEvent per nocturnal low (under 70 confirmed by 2 readings): nadir, minutes below 70, area, recovery slope over the 30 min after the nadir, carbs logged within 30 min, and inferred_unfelt, true for a run of at least 20 min whose recovery slope is under 1.0 mg/dL/min with nothing logged (0.99 fires, 1.0 does not; a treated low never does).
- **standing_window(night_records, low_events, recalls).** Over 14 nights: clean nights, the median rise on clean nights and the share in the same direction (8 clean of 14 with 6 rising $\rightarrow 6 \div 8 = 75\%$, median +42), near-misses, escalated warnings, re-arms, median acknowledgment time counted only on events whose presence_during is home, and from the recall answers alone: nocturnal lows, answered, unfelt (woke but felt nothing plus don't remember), no answer, and unfelt-low rate = unfelt $\div$ answered, no answer never in the denominator and reported beside it (4 nocturnal lows, 3 answered, 2 unfelt $\rightarrow 2 \div 3 = 67\%$, 1 no answer). Every value carries its confidence label; a value that cannot be computed is None with a label, never zero and never a blank row.
- **step_window_metrics(window_records, baseline_records, symptom_checks, injections).** low_point_shift = median window low point – median baseline low point (the demo: $76 - 98 = -22$); TBR over the window (58 of $5 \times 288 = 1,440$ expected readings $\rightarrow 58 \div 1,440 = 4.03\%$); near-misses; coverage ($1,390 \div 1,440 = 96.5\%$, against a floor of $0.70 \times 1,440 = 1,008$); baseline nights (fewer than 5 clean flags “baseline thin”); tolerance counts of fine, rough, can't eat, and missing days; adherence (logged weekly injections against expected, plus a dose-mismatch flag); ketone-risk episodes.
- **The rules that read them (backend/app/rounds/standing.py and step_watch.py, Chris; starting values, tuned here, never presented as researched facts).** Basal Check: in the last 14 nights, at least 5 clean nights, a median rise beyond ± 30 , and at least 70% of clean nights in the same direction; the “possibly too high” direction when near-misses reach 3 in 14 days; never from stale nights. Hypo Response: in 14 days, at least 2 escalated warnings, or at least 1 re-arm, or a median acknowledgment above 5 minutes, or any unfelt low reported; adds glucagon status. Follow-up: the 14 nights before a confirmed dose change against days 1 to 7 and 1 to 14 after, at least 3 clean nights each side, otherwise “not enough data yet”. Step Watch, in order: RED at any time on a level 2 low, a re-armed low alarm, or a ketone-risk episode on a can't-eat day (immediate, deduped per event, one per type per 12 hours); INSUFFICIENT under 70% window coverage (no conclusions, red still sent); AMBER lows on a shift of -15 or more, near-misses ≥ 2 , or TBR over 4%; AMBER awareness on one nocturnal low reported unfelt or not remembered, or two inferred suspected

unfelt lows with no answers; AMBER tolerance on rough or can't eat on 3 of 5 days; AMBER highs on 2 ketone-risk episodes; GREEN otherwise, into the weekly digest. Boundary tests: -15 fires and -14 does not; 4.03% fires and 4.00% does not; 92 readings pass and 91 fail; +30 does not fire and +31 does; 70% same direction passes and 69% does not.

- **Labeling the real history (ml/label_history.py).** The cleaned 22 months run through `classify_night`, `night_metrics`, and `low_events` with the shipped forecaster and the actual alarm rule to produce INFERRED reason codes and events per night. Those are the glucose-side fields of `NightRecord`, `AlarmEvent`, and `LowEvent`, and they are real. What the export cannot give (acknowledge times, escalation, presence, morning answers, basal timing, exercise) is never fabricated into the history; it enters only as a labeled overlay in the demo scenarios, and reason codes on history are always marked inferred, never presented as logged. Output: `ml/data/nights_labeled.csv`, `gitignored`.
- **The three measured numbers (ml/evaluate_rounds.py).** All from the real 22 months, never from the synthetic scenario. Retrospective lead time for Basal Check: for each real basal increase Chris lists (2 to 3, with confidence notes, in `gitignored ml/data/therapy_changes.txt`, away from the two Libre holes), run the rule daily over the prior history and record the first trigger; lag = change date - first trigger, so the claim becomes "Rounds would have flagged this N weeks earlier", retrospective, n of 1. False-alarm rate: run the Standing Card rules and the Step Watch amber rules over stable-therapy stretches (excluding gaps and any window within 30 days of a change) and report how often they fire; 22 months is about $22 \times 30.4 = 669$ days, so roughly $669 \div 5 \approx 133$ five-day windows and about 47 fortnights before removals. Detection lag: days from a real therapy change until the low-point-shift rule first flags 15 mg/dL or more in either direction. Wake resistance, response latency, and tolerance cannot be tuned on history (no acks, no answers); `metrics.md` says so instead of inventing a number. Leakage guard: any model-based number on a demo window (a near-miss, a warning) comes from the held-out 8 weeks or a retrained copy excluding that window, and `metrics.md` says which.
- **Scenarios for the cards (demo/make_scenarios.py).** Basal-change windows for Standing Cards: for each of the 2 to 3 real basal increases, 14 nights before and 14 after, date-shifted, with a companion JSON carrying the inferred reason codes (labeled inferred), the inferred events, a labeled synthetic overlay for what history cannot give (acknowledge times and surface, escalation, presence, one recall answer per low with at least one left as no answer), and the confirmed dose change on its real date so the Follow-up can be built after the seek; chosen so Basal Check fires before the change on the real data, with the headline computed by code, so the numbers flow into `DEMO_SCRIPT.md`, never the other way round. The titration scenario is SYNTHETIC with obviously fake timestamps: a 14-night baseline, step 1, and step 2 days 3 to 7 (tirzepatide 2.5 → 5 mg, 4-week steps), with the plan, symptom checks, weekly injection logs, recall answers, and an alarm-event overlay, built so the seek yields step 1 green cards and the step 2 amber card with exactly the worked example. Optional for Night Buddy: one night tuned so the ladder plays cleanly at 60x.
- **metrics.md, the Q&A sheet.** A Rounds section stating what is real (glucose, inferred reason codes and events, near-misses, the three measured numbers), what is a labeled overlay (acks, escalation, presence, morning answers), what is synthetic (the titration scenario), the thresholds and any changes from the starting values, the leakage guard's answer per demo window, and the caveats in the spec's own words: n = 1, observational, retrospective, "since the change" never "the change caused." The Basal Check demo is described with this qualifier, verbatim, every time: "glucose real; reason codes inferred and labeled; acknowledge, presence, and recall data are a labeled overlay." Night Buddy adds nothing here.
- **Tiger Cloud aggregates and the agreement test (George, C2).** The dashboards' SQL lives in `cloud/sql/`: hypertables for readings, alarm events, low events, and treatments; continuous

aggregates daily_stats (time_bucket 1 day: count, min, max, share under 70, in range, over 180), overnight_profile (30-minute buckets across the last 14 days with percentile_agg for the 10th, 50th, and 90th), nightly (1-day buckets with a 22:00 origin: coverage, low point, minutes under 70), hourly_heatmap (hour × weekday share under 70); time_bucket_gapfill for sensor gaps; a compression policy after 7 days. The 22 months load once from the cleaned CSV with the same device_id. ml/tests/test_agreement.py loads one scenario night into both paths and asserts the nightly aggregate and nights.py agree on coverage, low point, and minutes under 70 to the reading; the dashboards never feed a card.

Functional Requirements

1. Every new reading triggers a forecast. The predicted value and trajectory go to the display and the alarm engine.
2. The predictive alarm fires when the forecast crosses the low threshold with at least N consecutive agreeing predictions, starting at N = 2 to damp flicker.
3. If the last hour has data gaps, forecasting is suspended and the UI says so, rather than predicting on bad input.
4. The model file, thresholds, and N are config, not code.
5. Every reason code and every number on a card comes from nights.py; the ledger and the rules never re-implement one, and the same functions run over the real history for validation and for the three measured numbers.
6. Every narrative (morning report, card, buddy line, family story) is validated against the computed numbers before it ships, and the deterministic template is the fallback for a failed check, an error, or no network.
7. No conclusion is drawn from a stale night (coverage under 85%) or from a window under 70% coverage (INSUFFICIENT, red still sent); Basal Check reads clean nights only; and every metric carries its confidence label, with a value that cannot be computed shown as such, never as zero and never as a blank row.

LLM Narratives

- **Inputs.** Overnight stats (low and high with timestamps, time in, below, and above range, alarm events and acknowledgment times), logged treatments, and the prior evening's exercise and meal logs.
- **Generation.** One narrative.py call (the chain above: Backboard → Meta → Claude → template) with a fixed prompt demanding two short paragraphs, plain language, no medical advice, and mention of anything unacknowledged.
- **Output.** Goes into the morning email alongside the graph PNG and into the stored report. A fallback report is pre-generated for the demo in case of no connectivity.
- **One module, four prompts.** narrative.py serves every Irin narrative: the morning report (reports.py becomes a client), the clinician card narrative (two to three sentences, plain language, no dose numbers, no advice), the Night Buddy lines (the morning “all quiet for both of you” and the event close-out, “Sam’s okay. Your call at 3:12 woke him; he treated and recovered.”), and the Family Story (two to three sentences in family register, the patient’s agency, never alarm, never advice, optionally one conversation starter not about glucose).
- **The validator.** It extracts every numeric token from the generated text (integers, decimals, percentages, clock times) and requires each to exist in the computed metrics, including allowed

forms such as “4 of 6”; on any miss, on error, or with no network, a deterministic template renders instead. For a Level 1 Family Story the rule is inverted: zero glucose values allowed (clock times fine), and any mg/dL number sends it to the family template. `tests/test_narrative.py` feeds a text with an invented number and asserts the template is used.

- **Fallbacks.** A morning report is pre-generated for the demo in case of no connectivity, and every card, buddy line, and story has its template, so the demo never depends on the API being reachable.

Web App

One web app at `irin-out-of-sleep-at-hackgt.tech`, hosted on Vultr behind Caddy, with four tabs across the top: Irin Device, My Irin, Irin Buddy, Irin Rounds. There is no login; the PIN (the “1234” code) is entered once, kept in `sessionStorage`, and sent as a header on every mutating call, to the Pi and to the relay alike. The Device tab reaches your Pi through its Cloudflare tunnel at `device.irin-out-of-sleep-at-hackgt.tech` after pairing, so it works from anywhere, including over cellular. Runs as a pinned tab or PWA on the phone. The bullets below are the Device tab; My Irin and Irin Buddy have their own sections after Family View.

- **Pair your Irin.** The first button on the Device tab. The kiosk shows a QR and a 6-digit code; scan or type it; the app posts `{code, username}` to the relay (`POST /v0/device/pair`), stores the `device_url` and token it gets back, and shows the live number within a second; the Pi confirms the pairing on its next relay poll. Unpair from settings (`DELETE /v0/device/pair` with the token) or from the kiosk settings on the Pi. The code is single use and expires in 10 minutes; it is shown only while the radar sees someone in the room. In development, `VITE_DEVICE_URL` (mock backend only, DEV badge) skips pairing.
- **Live view.** Current value, trend, mini graph, and connection status, mirroring the device over the same WebSocket. Each morning, a “last night” shortcut opens the newest report.
- **Connection behavior (the one hard client rule).** On EVERY WebSocket connect, the backend sends a full state snapshot: latest reading, forecast, alarm state, settings, presence, and mode. The client renders only from snapshot plus subsequent updates and never assumes it saw missed messages, so a phone that was locked in an elevator comes back correct, not confidently stale. If the socket is down more than about 15 seconds, the app shows a disconnected banner, the same honesty rule as stale data. This is what makes the acknowledge-after-unlock requirement actually true.
- **PIN handling.** Entered once per session, held in memory or `sessionStorage`, and sent as a request header on every mutating call. Never in a URL: URLs leak into history, server logs, and screenshots of the demo panel.
- **Acknowledge button.** Full width at the top of the screen, visible only while an alarm is active. Silences within 1 second over a WebSocket message, and works after the phone locks and unlocks (the PWA reconnects on focus).
- **Settings.** Low and high thresholds, predictive alarm on/off and lead time, high-alert behavior (one-shot by default, with an optional still-high reminder after a configurable number of hours), night window hours, LED colors per state, alarm sound picker with preview, volume, report email address, basal reminder time, and the Home/Away presence override (the radar drives presence automatically; the toggle is the manual fallback that always wins). All persisted to SQLite and applied without a restart.

- **Logging.** Basal taken with timestamp, meal entries of carbs plus units, and quick notes such as exercise or a site change. Entries post to the backend, become Nightscout treatments, and feed IOB.
- **Voice logging.** A mic button using the browser's Web Speech API. Recognized text goes to the backend parser, and parsed values are echoed back on screen and aloud ("45 carbs, 5 units, save?") with explicit confirmation before anything involving insulin is stored. Partial parses ask for the missing piece, and a 10-second timeout discards the entry.
- **The Device tab's Reports screen.** Stored morning reports listed by date, tap to open, same content as the email. The email becomes a notification channel rather than the only copy.
- **Demo control panel.** Reached by a small, low-key button inside the app (an icon at the nav's edge or on the settings screen), one tap, never by typing a URL, and kept out of the main tab row: it is stagecraft, not a daily screen. On the panel: the LIVE/DEMO mode switch, scenario picker (the save, normal night, failure), replay speed slider, pause feed, and a "trigger low now" injection.
- **Rounds in the phone app.** The app carries all input; the device carries none. Morning recall cards driven by recall_due: one per nocturnal low with a small curve around the nadir, the question ("At 2:47 AM you were 58 for about 25 minutes. Do you remember that?"), and four one-tap answers; when carbs were logged within 30 minutes of the nadir the card says "you logged carbs at 3:05" and asks only whether symptoms were felt. Nothing is ever pre-selected; an unanswered card disappears at noon and the record says no answer. During a watch: the daily stomach check-in (fine / rough / can't eat; missing is no data, never fine) and the weekly "I took my shot" with its dose label echoed and confirmed like every insulin entry. A Rounds tab: pairing status and Revoke, "What my doctor sees" (every card the device sent; the inbox's card renderer is plain JS and the app's Rounds tab is a React component built from the same generated card type, so patient and doctor read the same fields), the glucagon-on-hand toggle with its expiry, today's check-in status, and the step timeline with holds shown as shifted dates and the next gate. The doctor-message confirm mirrors the kiosk takeover with the sessionStorage PIN, an insulin change echoing its units. Every tap is a PIN-gated POST (recall, stomach, injection, pair/start, revoke, confirm, decline).
- **Night Buddy in the phone app.** A buddy card (paired first name, twin or mirror, this morning's line, revoke), the four opt-in toggles, the emergency-script editor, and the TREATING button: one giant "I'm treating" on the app home while an alert is open and as the action on the alert notification, one thumb press with no confirmation step, because hypoglycemia impairs exactly the faculties a fiddly UI demands (invariant 17; checked on a real phone from the lock screen). Buddy alerts arrive over the WebSocket while the app is open and loop on the wake-locked watcher page; the WhatsApp message is the second channel; native Critical Alerts or a call are production, not built.
- **Family Story in the phone app.** A Family section in Settings: add a recipient by name and email, pick the level (Story only / Story + view) and the send mode (automatic / approve each morning), Pause and Revoke per recipient, the default stated plainly (first story needs approval, then automatic). On the home screen, "Sent to Mom" chips from family_story_status, and when a story is pending, its text with Send and Skip (PIN). Demo-mode stories carry the DEMO badge and say "not sent (demo)."

Family View

The one piece of the system with no Pi in the loop: a single static page in frontend/family/ (index.html, style.css, family.js, no build step) served at family.irin-out-of-sleep-at-hackgt.tech, for the people who worry about you.

- **Data path.** Dexcom phone app → Dexcom Share → Nightscout → the Pi → the forwarder → Irin Cloud's nightly rollup → this page, which calls GET /v1/family/last_night with the family bearer. The Pi is not required for the page to load (the rollup lives in the cloud), but it is the only source of new readings, so the page says stale when the Pi is off.
- **What it shows.** The current value with trend arrow, minutes since that reading, and a "last night" strip from Irin Cloud's nightly rollup (GET /v1/family/last_night): overnight low and high with times, and rough time below range. Nothing else — it is a window, not a dashboard.
- **Refresh and staleness.** Re-fetches every 60 seconds. If the newest entry is older than 15 minutes, the page says stale and greys the number, the same honesty rule as the device: a frozen number that looks fresh is worse than no number.
- **Access.** The page authenticates with a family bearer the patient issues per recipient from the Family section (Level 2 recipients), kept in config.js only for the demo page and revocable with one tap; it grants one rollup and the current value, nothing else. No Nightscout token ever sits in page source, and no PIN gate, because the page has no verbs.
- **Setup.** Issue a family bearer, put it and the Irin Cloud URL in config.js, and the page is served from family.irin-out-of-sleep-at-hackgt.tech by the same Caddy as the app. Minutes.
- **Demo value.** A URL judges open on their own phones while the device does something else entirely — the "check on your kid from another state" story made tangible.
- **Family Story in front of it.** Level 2 recipients get this page's link at the bottom of each morning's story; Level 1 recipients get the story alone, never the token. The page itself is unchanged. It is never shown in the Impiricus slot: a family bearer in page source thirty seconds after the encryption demo invites the wrong question, so it belongs to the main slot and the Meta video. Stretch, only after the relay exists: a story card at the top of the page, fetched from the relay with a bearer, and the page still works with the Pi off, with or without the card.

My Irin (dashboards)

The second tab: history and insight, the Device tab being the present tense. Every chart is one continuous aggregate or hyperfunction query on Tiger Cloud over the readings and events your Pi forwards, plus the one-time load of your 22 months of history, drawn client-side from GET /v1/dash/<name> with the owner bearer. Two rules: dashboards draw pictures and nights.py decides cards (where both compute coverage, low point, or time below range, a test asserts they agree), and the relay never touches this data (it lives in the separate Irin Cloud service).

- **Nights strip.** Last 14, 30, or 90 nights, one tile each: low point, high point, minutes under 70, coverage, colored by reason code. The patient's view of what the doctor's Basal Check sees.
- **Time in range trend.** Daily TIR, TBR, and TAR from the 1-day aggregate with 7- and 30-day rolling lines.
- **Overnight profile.** Median and 10th to 90th percentile glucose by time of day over 14 days, percentile_agg over 30-minute buckets: the ambulatory glucose profile clinicians actually read.
- **Where my lows live.** Hour-by-weekday heatmap of the share of readings under 70; the 3 AM cluster in one picture.
- **Alarm response.** Alarms per night by tier, escalations, median minutes to acknowledge, trend over weeks. Device rows, labeled measured.
- **Near-misses.** Warnings that never crossed, per week: the number that makes your chart look cleaner than reality.

- **Basal timing.** Logged basal time against your usual time, late and missed counts, and the overnight rise on clean nights, patient-side.
- **Sensor health.** Coverage per day and gap minutes from time_bucket_gapfill: when sensors died and how long you were blind.
- **Step Watch.** During a watch: nightly low point against the baseline band, the tolerance strip, injection adherence.
- **Buddy.** Nights covered, all-quiet streak, alerts sent and received. No glucose.
- **Under the hood.** Readings stored, compressed size, compression ratio, last forwarder sync. Costs nothing and judges ask about it.

Irin Buddy (Find a Buddy)

The third tab holds the buddy card, the treating button, and the Find a Buddy flow. The flow is one screen per step, all input in the app, all of it stored in the relay's directory (MongoDB Atlas) under your username; a username is just a handle, and the membership card is still a CGM feed behind the account.

- **Opt in.** Do you want a personal buddy? Then: pick someone you know by username (a search box over the directory) or be matched by the algorithm.
- **The globe.** A d3-geo orthographic globe you spin with a finger; the political timezone under your finger highlights as it turns and shows its current time; tap to select or unselect, as many zones as you want; your own zone starts highlighted. Zone shapes from the timezone-boundary-builder GeoJSON, simplified; the clock from Intl.DateTimeFormat with the zone's IANA name. Fallback if the shapes fight back: 24 offset bands with the same interaction.
- **Languages.** The languages you speak, so your buddy can speak one of them.
- **Availability.** The hours you are available, on a week grid; 22:00 to 08:00 is locked as asleep and cannot be unchecked. Free text works too ("weeknights after nine, weekends any time"): Muse Spark parses it into the week-grid rows (the availability_parse task, structured output), which you confirm on the grid before they are stored; the deterministic scorer reads the rows, never the text. This is what lets someone who is asleep during your free hours be paired with you: their sleep is your watch, yours is theirs.
- **Emergency contact.** The number of the person to call if you cannot be woken and the instructions for them. Rough notes become a calm, ordered script (the emergency_script task, Muse Spark, JSON fields) that you confirm. The number is encrypted at rest and never shown to anyone; the relay places the call. The instructions are shown only to the live claim-holder.
- **Matching.** One aggregation pipeline in the relay: candidates opted in as watchers, language overlap, timezone in your set, availability covering your sleep window (converted to UTC), scored as hours of your sleep covered plus a bonus for a mirror 10 to 14 hours away plus one per shared language; the top three are offered; either side can decline or re-match, and declined never return. The introduction line and the why-this-match line are written by Muse Spark through narrative.py from the score parts (the match_explanation task), first names only, never a glucose value, and arrive from the relay as text; the app never calls Meta directly, and an LLM never picks the buddy.
- **Then.** Pairing (the same four-digit ceremony), the buddy card with this morning's line, and, when it matters, the alert: the watcher page requests a wake lock when you tap On watch and loops the voice until you tap Call.

Clinician Inbox (mock Ascend)

The doctor's side of Rounds: frontend/clinician/ (index.html, style.css, inbox.js, config.js), served at doctor.irin-out-of-sleep-at-hackgt.tech by the same Caddy as the app, styled to read as a screen inside Impiricus Ascend with a clear "mock" label in the footer. Like the family view it has no Pi in the loop; unlike it, it holds a key. One card model, two layouts: the program field decides which sections render.

- **Data path.** Pi → relay (sealed cards, outbound only) → this page, which polls GET /v0/inbox every 5 seconds with the bearer it was issued at pairing and decrypts each card with the private key its browser generated. Actions go the other way as sealed DoctorMessages, and the Pi polls for them. The Pi never hears from this page directly.
- **Pairing page.** Reads the token and device public key from the URL fragment the QR encoded, generates the doctor keypair with tweetnacl-js into localStorage, posts the public key with a display name, shows the four-digit code, waits for the device's confirmation, then stores the bearer.
- **Inbox list.** Ordered red, then amber, then one digest line for greens, with program, source (Irin Bedside / Irin Brain), and DEMO badges, and Spark offers pinned at the top.
- **Standing card detail.** A strip of the 14 nights colored by reason code, logged and inferred visually distinguished; the rise chart on clean nights only; the excluded nights with their reasons; the metrics table with a confidence label on every row and never a blank one; the narrative; the actions the card's allowed_actions lists (adjust basal, schedule visit, ask patient, dismiss on a Basal Check; message patient, schedule visit, access and resources, dismiss on a Hypo Response).
- **Watch card detail.** The step timeline, the low-point chart against the baseline band, metrics with confidence labels, the tolerance strip, the awareness row, the narrative, and the actions: proceed, hold 2 / 4 / 8 weeks, adjust insulin with typed units and a start date, message, schedule visit, access and resources, end watch.
- **Offer page and action modals.** The Spark offer opens the Step Watch plan setup form (drug class and schedule pre-filled from a small JSON of label schedules and editable, start date, on insulin, the three options), whose Accept sends a plan_create message; every action is a sealed DoctorMessage, and "Patient confirmed 07:14" shows from the message resolution.
- **Resources and the panel.** Category first, then brand, then the mock Ascend handoff screen with the banner "No patient data shared with any manufacturer," then POST /v0/resources/request; hidden when the card's resource categories are empty (off-label use). The "what Impiricus sees" panel: GET /v0/log, IDs, sizes, timestamps, ciphertext prefixes, never cut from the demo. Optional, last: tap a night to replay it, about 20 seconds, rendered from the card's own nights list.
- **Access and setup.** The bearer is the access control and the private key never leaves the browser; revoke from either side ends it. Put the relay URL in config.js, frontend/clinician/ is served by the compose file's Caddy at doctor.irin-out-of-sleep-at-hackgt.tech, pair once with a team phone. Minutes, like the family view.
- **Demo value.** A judge scans the QR with their own phone and becomes the doctor: the four digits match, the Basal Check lands on their phone with the excluded nights listed, they adjust the basal and watch the device ask Chris to confirm, seek to the Follow-up, then to step 2 of the watch, hold it, and flip to the panel showing nothing but ciphertext.

Watcher Page (Night Buddy)

The buddy's and the volunteer's side of Night Buddy: frontend/watch/ (index.html, style.css, watch.js, config.js), served at watch.irin-out-of-sleep-at-hackgt.tech beside the app. It pairs as a browser peer

with the same pairing code as the inbox (the file is copied), so a watcher needs nothing but a phone and this page, plus a CGM feed behind the account.

- **Data path.** Buddy alerts and the morning lines arrive sealed through the relay and decrypt here; the hub list comes from GET /v0/hub/list with a volunteer bearer; claims, calls, and script reads go to the hub endpoints; the patient's device learns about a call from its next relay poll and chimes. No phone number crosses the network in either direction. The same alert text and the ElevenLabs clip also reach the buddy's WhatsApp through the relay (relay/notify.py, the Cloud API), the second channel, never with a glucose value; a web page cannot ring through a silent phone, so the buddy keeps this page open (wake lock) while on watch, and production is native Critical Alerts or a call.
- **The alert screen.** Huge and single-purpose: "Sam's alarm has been unacknowledged for 12 min, presence confirmed" (Bedside) or "no response to phone alerts, unconfirmed" (Brain-only), with Call (brokered) and, after that, Execute script, which shows the patient's own instructions only while the claim is live.
- **The hub list.** Pseudonymous listings (first name, alarm state, elapsed minutes) with the confidence label always visible and visually distinct, ordered by urgency then confidence; "I've got this" claims a 3-minute lease and everyone else sees "claimed, in progress"; an expired lease reopens the listing; "marked treating 2 min ago" when the patient taps treating; TOP urgency when treating expires unrecovered. Never a glucose value, a location, or a number.
- **The buddy card.** The morning line ("All quiet for both of you last night") and, after an event, the close-out, both written by the narrative pipeline and sealed to this page's key.
- **Access.** The list is visible only to verified volunteer accounts (a CGM feed behind the account, in standing); the demo pre-provisions one on a teammate's phone.
- **Setup.** Relay and Irin Cloud URLs in config.js, served at watch.irin-out-of-sleep-at-hackgt.tech, pair once. In the demo the pre-matched pair is the Pi-side patient and this page on a teammate's phone.
- **Demo value.** The device escalating on the table, this page lighting up across it, a claim, a brokered call, the device chiming, the acknowledge, and the morning message, all real code paths at 60x with DEMO badges everywhere.

Interface Contracts

Shared object definitions so each part can be built in parallel and meet only at these boundaries.

Object	Fields
Reading	timestamp, glucose mg/dL, trend, source (nightscout or replay), is_stale
Forecast	timestamp, predicted mg/dL at +30 min, trajectory points, suspended flag
Alarm state	state (idle / pending / active / acknowledged / re-armed), trigger type, started_at, acknowledged_at
Treatment	timestamp, type (bolus / basal / carbs / note), carbs, units, note text
Settings	thresholds, predictive on/off and lead time, night window, LED colors, sound, volume, email, basal time, IOB duration, presence override
WebSocket messages	reading update, forecast update, alarm state change, acknowledge command, settings change, treatment logged, presence change, mode change, state

Object	Fields
	snapshot (sent on every connect)
NightRecord (Rounds)	night_date, window_start, window_end, coverage_pct, reason_codes, code_source (logged / inferred / unknown), rise_mgdl, dawn_rise_mgdl, low_point_mgdl, tbr_pct, minutes_below_70, near_miss_count, level2_count, alarm_event_ids, is_demo
AlarmEvent (Rounds)	event_id, tier (predicted_low / actual_low / stale / high), started_at, acknowledged_at, ack_source (device / app), escalated, rearm_count, crossed_actual, presence_during (the raw-radar verdict for the episode: home / away / unknown, never the Home/Away state), is_demo
LowEvent, LowEventRecall, SymptomCheck	low_event_id, night_date, started_at, nadir_mgdl, nadir_at, minutes_below_70, auc_below_70, recovery_slope, carbs_logged_within_30min, inferred_unfelt, alarm_event_id; low_event_id, asked_at, answered_at, answer (felt_and_treated / woke_no_symptoms / dont_remember / was_aware; unanswered after the morning window is reported as no answer, never fine); date, gi (fine / rough / cant_eat), is_demo
SignalCard (Clinical Signal Card v0)	card_id, schema_version, program (standing / step_watch), kind (basal_check / hypo_response / follow_up / early_check / step_check / step_gate / safety / graduation / baseline_note), status (green / amber / red / insufficient), flags, confidence (metric → measured / reported / inferred), source (irin_bedside / irin_brain), patient_pseudonym, plan_id, step_index, period_start, period_end, headline, metrics, nights (date, reason_code, code_source, rise; readings and events for the replay), excluded_counts, tolerance_days, narrative, allowed_actions, resource_categories, is_demo, generated_at
TitrationPlan, Pairing, DoctorMessage	plan_id, drug_class (glp1 / gip_glp1 / weekly_basal), drug_label, steps (index, dose_label, planned_start), started_at, on_insulin, options (ketone_prompts, step_week_vigilance, vigilance_offset_mgdl), status (pending_confirm / active / graduated / ended), is_demo; device_id, doctor_id, doctor_display_name, doctor_pk, status (pending / paired / revoked), confirmed_at, peer_kind (doctor / buddy), is_demo; message_id, plan_id, kind (plan_create / plan_update / proceed / hold_step / insulin_change / note / schedule_request / end_watch / dismiss), hold_weeks, insulin (basal / bolus), new_units, start_date, plan, text, created_at, status (pending / confirmed / declined / expired)
Night Buddy models	BuddyLink, BuddyAlert, HubListing (status open / claimed / treating / resolved; confidence device_confirmed / unconfirmed; claim_expires_at), HubClaim, EmergencyScript. Settings gains basal_units, glucagon_on_hand, glucagon_expiry, four night_buddy opt-ins, emergency_script; Treatment.kind gains glp1_dose and therapy_change, and Treatment gains dose_label
Family Story models	FamilyRecipient (recipient_id, name, email, level story_only / story_and_view, send_mode automatic / approve_each, state active / paused / revoked, first_story_approved); FamilyStory (story_id, night_date, recipient_id, level, text, status pending_approval / sent / skipped / failed / demo, sent_at, is_demo); Settings gains family_recipients
FRESH_PIN_ENDPOINTS	the one list of routes that require a freshly entered PIN (pairing confirm, doctor-message confirm and decline), read by auth.py, by display.js, and by the app's usePin hook
WebSocket additions	card_sent, doctor_message_received, doctor_message_resolved, recall_due, symptom_check_due, pairing_state, plan_state, buddy_alert, hub_update,

Object	Fields
	treating_set, family_story_pending, family_story_sent; state_snapshot gains the active plan, pending doctor messages, today's check-in status, pairing state, buddy state, this morning's family story status, clock_synced (false until NTP syncs; there is no RTC battery)
UserProfile, BuddyMatch (relay directory)	user_id, username, first_name, languages (list), timezones (list of IANA names), availability (list of {weekday, start, end} in local time), sleep_window (22:00 to 08:00, locked), cgm_feed (nightscout_url, read token; the membership card), emergency_contact (ciphertext; the number is never returned), emergency_script, optins (have_buddy, be_watcher, hub_watchable, hub_volunteer), created_at; match_id, user_id, candidate_id, score, hours_covered, mirror (bool), shared_languages, status (offered / accepted / declined)
DevicePairing (owner)	device_id, username, device_url (the Pi's DEVICE_URL: https://device.irin-out-of-sleep-at-hackgt.tech on the Pi, http://localhost:8000 on a laptop), token, code, expires_at, kind (owner), paired_at, state; the separate owner-pairing model, never a Pairing row (Pairing.peer_kind stays doctor buddy): the Pi registers the kiosk code with the relay, the app redeems it with {code, username}, and the Pi confirms on its next poll; the code is single use and expires in 10 minutes
Irin Cloud ingest and dashboards	POST /v1/ingest {device_id, readings[], alarm_events[], low_events[], treatments[]} (device token; idempotent on (device_id, timestamp)); GET /v1/dash/{nights tir profile lows_heatmap alarms near_misses basal sensor step_watch buddy under_the_hood}?days= (owner bearer); GET /v1/family/last_night (family bearer); POST /v1/family/bearers and DELETE (PIN, from the app). Hypertables readings(time, device_id, mgdl, trend, source), alarm_events, low_events, treatments; continuous aggregates daily_stats, overnight_profile (30 min), nightly (22:00 origin), hourly_heatmap; compression after 7 days
Audio (ElevenLabs)	POST /v1/audio/render {kind (buddy_alert / script / story / echo), text, voice} → {url, duration} on Irin Cloud (cached by hash of text and voice); the Pi renders its own phrases directly and caches wav in backend/sounds/generated/; FamilyStory gains audio_url; BuddyAlert gains audio_url
Narrative backends	NARRATIVE_BACKEND = backboard anthropic template (the first link to try; the chain is Backboard → Meta direct → Claude → template, the validator after every link); provider meta is selected per task by NARRATIVE_ROUTING, not by NARRATIVE_BACKEND: JSON, task → [provider, model], eight tasks, final: clinician_card and morning_report (anthropic, a Sonnet-class Claude), buddy_line and family_story (openrouter / the Muse Spark name if Backboard's OpenRouter carries it, else meta / muse-spark-1.3), match_explanation, emergency_script, and availability_parse (meta / muse-spark-1.3, structured output), second_opinion (openai, a small model); NarrativeScope (kind patient / doctor / buddy / family, scope_id, backboard_assistant_id, thread_id); SecondOpinion {invented_number, advice} from a different provider; META_MODEL_API_KEY on the Pi and on the server

Relay API v0 (relay/README.md is authoritative; the two are kept identical). The relay stores ciphertext only. Every route is gated by the X-Source-Key header (device-originated calls), by the bearer issued at pairing (inbox and watcher reads), or is public and single-use (pairing completion).

- **Cards and inbox.** POST /v0/cards {recipient_id, sender_id, nonce, ciphertext, source, kind, program, is_demo} (source key; is_demo travels outside the ciphertext, and a card whose is_demo does not match the pairing's is rejected). GET /v0/inbox/{recipient_id}?since= (bearer).
- **Doctor messages.** POST /v0/messages (doctor → device, sealed, bearer). GET /v0/device/{device_id}/messages (source key; the Pi polls, 60 s live, 5 s demo). POST /v0/messages/{id}/resolution {status} (source key; metadata only, so the inbox can show Patient confirmed).
- **Pairing.** POST /v0/pair {token, device_pk, is_demo} (source key). GET /v0/pair/{token} (public; pairing state and device_pk). POST /v0/pair/{token}/complete {doctor_pk, doctor_display_name} (public, single use). POST /v0/pair/{token}/confirm (source key; issues the bearer). POST /v0/pair/{id}/revoke (source key or bearer). Tokens are 128-bit, single use, and expire in 10 minutes on clock.py; after pairing both directions use crypto_box, so the relay can neither read nor forge. Owner pairing, the separate DevicePairing model (Pairing.peer_kind stays doctor | buddy): POST /v0/device/pairings {code, device_id, device_url, token, expires_at} (source key; the Pi registers the kiosk's 6-digit code, single use, 10 minutes). POST /v0/device/pair {code, username} → {device_id, device_url, token} (the app redeems it; the relay writes the DevicePairing document, kind owner, and marks the code used; the Pi learns the username on its next poll). DELETE /v0/device/pair (the device token from the app, or the source key from the kiosk settings; both sides revoke on the next poll).
- **Impiricus stand-ins.** POST /v0/spark/new_rx (demo-only, key-gated; no patient identity, just “a new start in your panel”). POST /v0/resources/request {doctor_id, category, brand} (bearer): the only thing a pharma-side system would ever see; the schema has no patient field. GET /v0/log: the “what Impiricus sees” view, sender and recipient IDs, timestamps, sizes, ciphertext prefixes, never a plaintext field.
- **Hub (Night Buddy).** POST /v0/hub/listing (source key; first name, alarm state, elapsed minutes, confidence, is_demo; any glucose, location, or phone field is rejected). GET /v0/hub/list (volunteer bearer; ordered by urgency then confidence; demo listings only to demo volunteers). POST /v0/hub/claim (exclusive 3-minute lease; a second simultaneous tap gets 409 with the holder's expiry). GET /v0/hub/claim/{id}/script (the script, only to the live claim-holder, only while the lease is live; in the demo tier stored on the relay encrypted at rest, in production encrypted by the device to the claim-holder's key on demand). POST /v0/hub/call (brokered; the demo tier triggers the device chime through its poll, no telephony). POST /v0/hub/treating (clears the listing for 20 minutes; on expiry without recovery it returns at top urgency). POST /v0/hub/resolve (device-originated, on recovery or ack). GET /v0/hub/audit (every claim: who, when, actions, outcome).
- **Buddy directory (Night Buddy).** POST /v0/users (PIN; creates or updates a UserProfile; the emergency number is encrypted before storage and never returned), GET /v0/users/search?username= (bearer; first names and usernames only), POST /v0/match (bearer; runs the aggregation pipeline and returns the top three BuddyMatch offers), POST /v0/match/{id}/accept | decline (bearer). The directory holds no glucose and no readings; the CGM feed check is a one-time verification, not a stored stream.
- **Everywhere.** CORS limited to the irin-out-of-sleep-at-hackgt.tech origins and localhost, rate limits per source key, documents on MongoDB Atlas (the local mongo container in the laptop compose), and relay/tests/test_relay.py plus test_hub.py green before any deploy: key gating, bearer gating, single-use pairing, is_demo mismatch rejected, lease conflict, reopen on expiry, forbidden fields rejected, /v0/log shows no plaintext.

Claude Code Workflow

Each teammate runs Claude Code on their own laptop with their own subscription, in their own clone of the repo, on their own branch. No logins are shared.

- **One constitution for four Claudes.** CLAUDE.md at the repo root loads automatically into every Claude Code session, so all four copies follow the same rules without configuration: stay in your owner's directories, never touch contracts.py unasked, never commit secrets or medical data, run pytest before backend commits, and main must always boot.
- **Usage-limit hand-off.** When a teammate hits their usage limit mid-task, the work moves, not the account: they commit their state to their branch (broken is fine), push, and ping Chris. Chris pulls the branch and continues on his machine, opening the session with "I'm stepping in on ml/ for George" so the ownership rule yields for that session. When the teammate's window resets, Chris pushes and they pull. Two sessions on one machine are fine, with four rules. The stepped-in session never runs in the directory the taker's own session uses: `git worktree add ../irin-<lane> <branch>` gives the branch its own directory (copy .env in; gitignored files are not in a worktree). It runs on its own ports (8001, relay 8101) so both lanes run at once. Journals belong to lanes, not people: the taker writes the lane's journal file with a first line saying who stepped in, so the owner's next session reads its own file. And hand-off branches get plain commits only, never amend, rebase, or force-push, so the owner's git pull fast-forwards on return. If Claude was cut off before journaling, the human writes the five-line snapshot by hand. Ping Chris, who takes it over.
- **The two quality gates, /review and /audit.** Both are custom commands living in .claude/commands/, so every clone has them and any teammate's Claude runs them identically. Both share one principle: the session that wrote code is the wrong session to judge it, because it inherits all the author's assumptions. So both gates run in a FRESH session with no memory of writing the code, told explicitly to assume the code is faulty until the diff proves otherwise.
- **/review, the per-branch gate.** Used throughout the hackathon: when a big branch is ready to merge, its owner opens a fresh Claude session and runs /review. It diffs the branch against main, reads every changed file in full, and attacks along five axes in order: correctness (does it do what the branch name and journal claim; trace the main path and one failure path), the project safety invariants (only if the diff touches alarms, logging, data sources, or the frontends), security (new endpoints gated, no secrets, no real CGM data, input validated), bugs (off-by-ones on thresholds, unhandled None, WebSocket races, unawaited coroutines), and regressions (what existing behavior could break, and the NAME of the test that would catch it; a missing test is itself a finding). It runs pytest if backend code changed, then reports findings ranked by severity with file and line. It diagnoses only; the human decides what blocks the merge. Small single-file branches skip it so the gate never slows the loop. For sponsor code the invariants axis has three more lists, matching review.md: rounds/, relay/, and clinician/ (no dose recommendation anywhere, every number computed from nights.py with a confidence label and no blank row, the narrative validated with the template intact, no conclusion from stale nights or windows under 70%, the noise budget enforced in noise.py alone, a doctor message applies only after patient confirm and only from the paired key with an insulin change echoing its units, silence changes nothing, ciphertext only on the relay, is_demo everywhere, the recorder observes only, a missing morning answer is no answer, the unfelt-low rate divides by answered, the inferred-unfelt rule never fires on a treated low); buddy/, the hub, and watch/ (rungs additive only, the T+20 clock never delayed, the script visible only to the live claim-holder, no glucose or contact data in any listing, confidence labels distinct, the four opt-ins separate, treating one press, demo events never real, the WhatsApp payload in relay/notify.py and the buddy prompts grepped for mgdl, the narrative axis naming all four links of the chain, and

the Meta rows never touching the scorer); and family_story.py (no glucose value in a Level 1 story, no demo story to SMTP, paused and revoked recipients receive nothing, a no-data night never “fine,” the first story waits for approval).

- **/audit, the one-time full audit.** Runs exactly once, the night before judging: Chris, fresh session, fresh pull, whole codebase. Where /review trusts main and inspects one diff, /audit trusts nothing and walks everything: each safety invariant traced through the actual code (can a frozen reading ever render without a stale mark; can any path store insulin without a confirm; can settings silence a low; can an acked warning suppress the real low; does presence gating live only in the hal layer), contracts consistency across display.js, the app’s src/ (App.tsx and the usePin hook), ws.py, and contracts.py, PIN-gate coverage on every mutating route, no secrets or medical data in git, IOB math, every alarm transition, replay timing, test health, README accuracy, and every scenario CSV (and its companion JSON) playing clean at speed, plus the Rounds and Night Buddy invariants and the relay (the narrative axis names all four links of the chain, relay/notify.py and the buddy prompts are grepped for mgdl, and the Meta rows never touch the scorer). Findings go to AUDIT.md ordered most severe first, split into fix-before-judging and mention-if-asked. That triage is the point: the night before, knowing which bugs to leave alone matters as much as knowing which to fix.
- **What to know about both gates.** They report, never fix, unless the human asks, so a gate can never quietly rewrite code nobody reviews. They are told not to invent findings to look thorough; a clean pass is one line. And their findings feed the journal: anything /review flags that is not fixed before merging goes under Broken or Notes for other models, so the next Claude inherits the warning instead of rediscovering the bug.
- **The journal, the cross-Claude bulletin.** journal/ holds one file per owner (journal/chris.md and so on), overwritten entirely each time so it always reads as current state; git preserves every past version, so history costs nothing and the final audit can replay it. Each file opens with an Updated timestamp line and holds a complete snapshot, max 20 lines, under five fixed headers: Done, In progress, Broken, Interface changes, Notes for other models. Write triggers: end of every work session, immediately before any merge (feature into dev, or dev into main), and before any context compaction. Read triggers: session start (pull dev, read the other three files), again whenever dev is pulled mid-session, and every 2 to 3 hours during the hackathon itself. Journals belong to lanes, not people: only the human driving a lane edits its file (during a hand-off that is the person stepping in, who says so on the first line), so conflicts are structurally impossible. git log remains the code history, so commit messages say what changed and why. One content rule from day one: journal entries never contain health details beyond what is already in the public pitch, and the working repo stays private; the Meta submission is a fresh public mirror built on submission day with squashed history, journal/ and AUDIT.md excluded, and metrics.md trimmed to the headline numbers.
- **Merge etiquette for agent-written code.** Claude works on the feature branch; the human skims the diff before it merges into dev. Small, frequent merges: agent-generated code diverges fast, so a day-old branch is treated as stale. The dev-to-main promotion is a human act by design: branch protection on main means no Claude, and no accidental push, can reach the branch the Pi pulls.
- **Speed rules, binding during the event.** Nothing slower than one minute runs inside Claude's loop: no CI, and the only build step is the app's npm run build (seconds; allowed in-loop for frontend/app), verification is pytest and a browser refresh, the ML pipeline runs in-loop because it finishes in seconds (only a step slower than a minute, like a hyperparameter search, runs in a plain terminal outside Claude), and tests use clock.py's accelerated time rather than sleeping. One deliverable per prompt, with a stated plan approved first for non-trivial tasks. New feature means new session; CLAUDE.md and the journal re-orient a fresh one instantly. Tasks spanning two areas

split at the contract boundary and run in parallel. Model selection is delegated to the team's agent-master tooling. And humans interrupt early: thirty seconds of redirecting beats twenty minutes of confident wrong work.

Standard Operating Procedures

Everyone: Session Rituals

- **Session start:** pull dev, read the other three journal files, read your own, then start Claude with the standard opener: "Read docs/plans/<your-name>.md and the journal files, then: <today's single deliverable>." The plan file is the Claude-facing spec; nobody pastes the spec document into a session.
- **Session end:** overwrite your journal file, commit and push your branch even if broken (say so under Broken), so the hand-off SOP is always possible.
- **Running low on context:** compaction tends to hit around 50% context, so past the halfway mark the journal gets checkpointed at every natural pause and completed step, and always before any compaction or /clear, with In progress naming the exact next action (file, function, failing test and its error). After a compaction, Claude re-reads its own journal and plan file before continuing: the compacted summary is lossy, the journal is ground truth.
- **Merging:** features merge into dev: pytest green, human skims the diff, big branches get /review in a fresh session first, shared files merge only via Chris. dev is promoted to main only by a human (Chris), only when dev is verified working; Claude never merges into main.
- **Stuck:** 15 minutes or three failed attempts on the same fix means stop, journal it, and ask a human or hand off. Circling is the only true waste.
- **Usage limit:** commit, push, ping Chris; Chris pulls the branch and continues, opening his session with "I'm stepping in on <area> for <name>." The taker works in a separate git worktree on its own ports, writes the lane's journal, and makes plain commits only.

George: Claude Drives, George Steers

The whole pipeline (clean, build the dataset, train, evaluate) runs in about two minutes total, so George's Claude writes AND runs every script inside its own loop, reads the output, and fixes what breaks. George's job is judgment, not relaying runs. At five checkpoints Claude stops, shows the result, and waits for approval: (1) after cleaning, the reading count, gap list, and spike drops look sane; (2) after the dataset build, NO feature uses future information, the most important review in the pipeline, since a leak makes the model look brilliant in evaluation and useless on the device; (3) after baselines and the first model, the model beats the trend baseline on events, not just average error; (4) George and Chris pick the alarm threshold from the sweep plot, a product decision; (5) Chris reviews the held-out lows for sensor artifacts. Only a step slower than a minute (a hyperparameter search, kept small) runs in George's own terminal with the output pasted back. Privacy: running a script keeps the data on George's laptop and only what it prints reaches the conversation, so scripts print summaries only and Claude never opens or prints raw CSV rows, which carry Chris's name, date of birth, and full history. When a run finishes: commit only the small artifacts (forecast_v1.json, metrics.md), update the journal's Interface changes line if predict.py's signature moved, and never commit anything from ml/data/. Two more checkpoints for Rounds: (6) the lead-time, false-alarm, and detection-lag table over the real stable stretches, thresholds moved or kept with Chris; (7) the basal-change windows and their overlay, with

Chris vouching for the dates and answering the recall question for any low he remembers, at least one left as no answer, and every historical reason code marked inferred.

Slavik: Hands on Hardware, Claude on Diagnosis

All physical work is manual and follows the spec's phases: power off before rewiring, the pre-power checks in Phase 7, the wiggle test before any adhesive. Claude's role is diagnosis and scripts: paste it the error, describe the symptom ("first LED wrong, rest fine"), or photograph the wiring for your own reference; it reasons, you touch. Test scripts (`first_light.py`, `full_frame_test.py`) run by hand on the Pi over SSH, never through a Claude session on the Pi. Any wiring deviation from `wiring.md` gets written back into `wiring.md` the same day, or the doc becomes fiction. The Meta video is yours regardless of which Night Buddy tier ships: the dark-room device scene filmed early enough for a re-shoot, the split screen with a teammate's phone, and the 20-second Family Story shot; the second "device" in the video is the mock HAL on a laptop.

Justin: Edit, Refresh, Screenshot

The loop is edit, save, refresh the browser for the display and the role pages (no build, no install); for the app it is `npm run dev` with hot reload, `npx tsc --noEmit clean` and `npm run build green` before a merge, dist/ committed with the feature. Iterate with Claude by screenshot: paste what the screen looks like and say what is wrong with it, which beats describing CSS in words. Changes to how things LOOK need nobody's sign-off; changes to what messages the frontend SENDS need Chris's sign-off before merging. Test the phone app on a real phone (laptop IP, same Wi-Fi) at least once per feature, because the phone is what judges hold. The clinician inbox and the watcher page are static folders the compose file's Caddy serves at their .tech names (`config.js` only, no server code); the treating button gets the lock-screen check on a real phone, one press to `treating_set`, before the Night Buddy beat is promised.

Chris: Merge Gate and Deploy

Contracts changes: tiny standalone commit, announced in the group chat before merging, everyone pulls immediately after. Shared-file merges: skim, merge, same day, so nobody's branch waits on the gate. Promotions: you alone merge dev into main, by hand, once dev boots and the demo scenarios play; batch promotions at natural milestones rather than per-feature. Pi deploy: `git pull` plus `systemctl restart irin`, and only ever from main. The final audit: fresh pull, fresh session, `/audit`, the night before judging, findings triaged into fix-now versus mention-if-asked. When stepping in for a teammate, announce it to Claude at session start and journal under the LANE's file (journals belong to lanes, not people), in a separate git worktree on its own ports so your own session keeps running. Kiosk autostart: enable it yourself, by hand, only after `/audit`'s fix-now list is empty, and verify the `NO_KIOSK` escape hatch (touch, reboot, no Chromium; remove, reboot, kiosk) before packing. Pi deploy is `git pull`, `pytest` on the Pi, then `systemctl restart`; never skip the middle step. Cloud deploy: `docker compose up` on the Vultr box (Caddy, relay, cloud) with the Atlas and Tiger connection strings in its `.env`, `pytest` from relay/ and cloud/ first, and the laptop fallback (the same compose with local mongo and timescaledb) rehearsed once at home before judging.

Demo Plan

Eleven scenarios: three scripted replays always shown, four live interactions woven between them, and four held in reserve for questions. All scripted scenarios are date-shifted CSVs in `demo/scenarios/`, played through the replay engine at accelerated speed.

The demo panel and the mode switch. The Demo section of the Device tab (an in-app route, `/demo`), opened by a one-tap button inside the app, never by typing the URL, and PIN-gated like every mutating surface (the PIN is entered once during table setup, so at the table the whole gesture is two taps: panel, then switch). Its first control is the LIVE/DEMO switch: one tap swaps the backend between the Nightscout poller and the replay engine at runtime, both living behind the same datasource interface. Switching resets alarm state (no alarm carries across worlds) and the clock multiplier, and switching back to live serves the cached real reading marked stale until a fresh poll lands. While demo mode is active, every screen shows an unmistakable DEMO badge, the same honesty rule as stale data: the device never renders fake data that looks live. The panel's other controls (scenario selector, speed, pause feed, inject low, basal-time button) work ONLY in demo mode and return 404 in live mode, so fake readings can never be injected into the real stream. Injections are an overlay on the playing scenario, never written into the CSV, so a scenario replays clean every time.

Core Three (always shown)

- **1. The Save (`the_save.csv`).** A real night sliding toward a low, told in two acts. Act one: the dotted forecast line crosses the threshold before the actual value does and the amber warning ramps up, and the narrator says “nothing is wrong yet, it's alarming for a low that hasn't happened.” Act two: let it sit unacknowledged and it escalates to the full red alarm on its own, then one tap on the phone silences it. The tier change IS the pitch: the device knows the difference between 30 minutes of warning and none.
- **2. Normal night into morning (`normal_night.csv`).** A quiet in-range night fast-forwarded to the morning boundary: morning screen, TIR rings, and the LLM report arriving on the Device tab's Reports screen. Then one tap to the My Irin tab: the nights strip and the overnight profile over 22 months of real history, live from Tiger Cloud, the one screen no other bedside device can show.
- **3. The Failure (`failure.csv`).** The feed is paused mid-demo; the display flags stale data and the stale-data alarm fires. Demoing the failure mode unprompted answers the sensor-death question before judges ask it.

Live Interactions (no files, performed against whatever is playing)

- **4. Voice logging.** “log 45 carbs and 5 units” on the phone, echo and confirm, IOB updates on the display.
- **5. Trigger low now.** A judge is handed the phone and presses the inject button themselves.
- **6. Acknowledge and escalation.** One alarm is deliberately left unacknowledged through the escalation window (louder, strobe), then acknowledged.
- **7. Presence dormancy.** No CSV; live hardware. Everyone steps back from the table, the radar loses presence, and the display dims to the clock with the LEDs off. A judge walks up and the device wakes instantly. Answers “what happens when nobody's home?” and shows the manual toggle overriding the radar from the phone. At table setup, switch the radar to the DEMO profile in the HLK app (max gate 1, about 5 ft) so people behind the judges do not count, then walk the boundary with the real crowd present: step in until presence switches on, step back until it clears. After the

event, switch back to the HOME profile, since the radar keeps whatever was last set. Two backend rules shape this demo: the Away timer reads clock.py, so at demo replay speed the 15-minute rule takes seconds, and radar absence never sets Away during the night window, so run it while the playing scenario is in daytime. If the room still defeats the radar, the manual Home/Away toggle is the guaranteed path.

Held in Reserve (Q&A ammunition)

- **8. Suppressed false alarm (meal_context.csv).** The same falling curve twice: bare, the predicted-low fires; with a meal logged, the context rule softens it. Answers “won't it cry wolf at every dinner?”
- **9. Re-arm (rearm_low.csv).** A low acknowledged but still low 15 minutes later alarms again. Answers “what if you silence it and pass back out?”
- **10. High notification (high_spike.csv).** A post-meal spike crossing the high threshold: one chime, amber tint, persistent indicator, nothing to acknowledge. Shows the device treats highs with proportion instead of panic.
- **11. Missed basal.** No CSV; clock-driven. The demo panel gets a button that sets the basal reminder time to two minutes from now, so the nudge and escalation can be shown on command.

Run of Show (3-minute slot)

Open LIVE: the device sits tracking Chris's actual Dexcom in real time, worn on his arm, while he gives 30 seconds of problem framing with his own T1D story. Then one tap on the demo panel switches to demo mode (“now let me show you a bad night”), the DEMO badge appears, and the scenarios run: Scenario 1, Scenario 4 while things are calm, Scenario 3, and Scenario 2 as the closer, handing the phone over for Scenario 5 if a judge lingers. One person drives, one narrates, one watches the hardware. Live mode needs the Pi online (venue Wi-Fi or a phone hotspot); if connectivity dies, start in demo mode and say so, the switch is a bonus, never load-bearing. The full script lives in demo/DEMO_SCRIPT.md. Boot order at the table: hotspot up first, then the Pi, so the clock syncs (no RTC battery).

Engineering note. The alarm engine's timers (15-minute re-arm, 5-minute escalation) read time from clock.py, which accelerates under replay. Without this, Scenario 9 stalls in real time during the demo.

Irin Rounds Run of Show (Impiricus, about 2 minutes plus Q&A)

Doctor-first, one narrative arc: detect, decide, verify, then the pharma moment. Beats 2 to 4 alone tell a complete story, and so do beats 5 to 7; rehearse both halves standalone in case the room runs short. Everything runs in demo mode with the DEMO badge on every screen, on a team phone or the judge's own; the full script is in demo/DEMO_SCRIPT.md.

- **1. Pairing.** The device shows a QR code, a judge scans it with their own phone, both screens show the same four digits, Chris confirms with the PIN. The judge is now the doctor.
- **2. Detect.** Replay Chris's real, date-shifted two weeks before a real basal increase. The Basal Check card arrives: the rise on the clean nights, the excluded nights listed with their reasons, every number labeled measured, reported, or inferred. Introduced with this qualifier verbatim, every time: “glucose real; reason codes inferred and labeled; acknowledge, presence, and recall data are a labeled overlay.” Line: “Same CGM line, opposite prescriptions. Rounds knows which.”
- **3. Decide.** The judge taps adjust basal and types the units; the device asks Chris to confirm by voice, echoing the units; the inbox shows Patient confirmed.

- **4. Verify.** Jump forward through the real post-change data; the Follow-up card shows the rise flattening. “Rounds would have flagged this N weeks earlier,” with N from metrics.md, retrospective and n of 1.
- **5. The therapy start.** The judge’s inbox shows a Spark offer: new GLP-1 start in your panel. Thirty seconds of plan setup, Accept, the device asks Chris to confirm, the watch is on. Jump to step 2, day 7.
- **6. The amber card.** Low point down 22, two near-misses, time below range 4.0%, one low not remembered, rough stomach 3 of 5, labeled SYNTHETIC. The judge holds the next step four weeks and reduces insulin; Chris confirms again; the timeline slides.
- **7. The pharma moment.** On the Hypo Response row, no glucagon on file. The judge taps access and resources, picks the category and a brand, and the mock Ascend handoff appears with the banner “No patient data shared with any manufacturer.” Flip to “what Impiricus sees”: ciphertext only.
- **8. Close.** “Every hold is a patient who didn’t quit, and every one of these cards started from the patient, not from pharma. No glucose value ever left the doctor’s hands.”

Night Buddy and Family Story Beats (Meta)

The Meta entry is judged on the video, the write-up, and the public repository; these are the two live-or-filmed beats behind it.

- **Night Buddy, 45 seconds, only if the demo tier shipped.** The device on the table is the patient; a teammate’s phone with the watcher page is the mirror buddy, both visible to the judge. The replay drives a real unacknowledged low at 60x: warning, full alarm, escalation, and at T+10 with the radar confirming presence the alert lands on the phone (first name, minutes unacknowledged, presence confirmed, no glucose); the phone speaks, in the ElevenLabs voice, “Your buddy Chris is in trouble,” on a loop until the teammate taps I’ve got this, then Call; the device chimes; Chris acknowledges; the listing resolves; the morning close-out renders on both screens. The demo panel’s “trigger buddy rung” control exists for the laptop rehearsal, where it drives the mock radar first; on the real device it is the radar. If the tier did not ship, the beat is in the video only, and that decision was made when R1–R11 had landed with a full day left, never at 3 AM.
- **Family Story, 20 seconds, in the video.** Split shot: a phone showing a raw CGM-follow graph and a worried face, versus Mom’s phone playing the story as a 20-second voice note in the ElevenLabs voice. “Same night. Two ways to find out your son is okay.” Then the patient’s morning screen: “Sent to Mom” and the pause button, to show who is in control.
- **The demo panel’s sponsor controls,** all demo-only, all 404 in live mode, greyed in the app: jump to step N day D (the replay seek), send card (Bedside | Brain-only, which runs the adapter twice so both cards sit side by side, differing only in their confidence labels), brain_only toggle, simulate Spark offer, start watch, trigger buddy rung.

The Meta Video (2 to 3 minutes, Slavik)

The Meta entry’s required deliverable, and the only place Night Buddy is guaranteed to appear. Chris is the subject; the second device is the mock HAL on a laptop; every staged scene is labeled concept.

- **0:00, the problem.** First person: Chris, alone, phone on the nightstand, the fear every T1D adult living alone shares. One line on Dexcom Follow: “the only fix today is giving someone your medical data forever.”

- **0:30, the promise.** Matched with a stranger; the AI introduction; the morning ritual, “All quiet for both of you,” shown as a week of five-second mornings.
- **1:00, the night it matters.** Split screen: left, the device escalating in a dark room, nobody stirring, the radar dot showing someone present; right, a phone across the world lighting up in daylight. The claim. The brokered call. The dark room fills with ringtone and chime. Movement. Juice.
- **1:45, the morning after.** Both screens: “Sam’s okay. Your call at 3:12 woke him; he treated and recovered.” A beat of silence. The 20-second Family Story split shot sits beside this beat, the same morning told to Mom.
- **2:00, the hub, labeled concept.** The watch list, a claim, the treating button, “only the watchable can be watchers,” one line of scale: the sun never sets on the network.
- **2:20, the close.** “AI does not replace the person who checks on you. It finds them, and it knows when to wake them.”

Sponsor Demo Logistics and Fallbacks

What has to be true at the table for the two sponsor beats, and the canned fallback for each thing that can break.

- **Reach.** Judges’ phones must reach `irin-out-of-sleep-at-hackgt.tech`, the role pages, `api.irin-out-of-sleep-at-hackgt.tech`, `cloud.irin-out-of-sleep-at-hackgt.tech`, and `device.irin-out-of-sleep-at-hackgt.tech`; test from the venue and over the hotspot before judging. If Vultr or the venue network fails, the laptop fallback (stated once in `chris.md`, R6 fallback): the same compose file on Chris’s laptop serves everything on one origin, `http://<laptop-ip>` (Caddy on plain HTTP); the app’s dist, the role pages, the relay, the cloud service, local mongo and timescaledb; the Pi’s `APP_ORIGIN` is set to `http://<laptop-ip>`; the app’s dist is rebuilt with the `VITE_*` values pointing at `http://<laptop-ip>` (or `npm run dev -- --host with APP_ORIGIN set to that :5173 origin`); the role pages’ `config.js` point at the laptop; judges’ phones on the hotspot open `http://<laptop-ip>`; the Pi keeps working with no cloud at all. Rehearsed once at home.
- **Pairing.** If the QR will not scan (glare, camera focus), the demo phone is already paired and the beat continues from the four-digit match on that phone. Test the QR with every phone the team owns.
- **The radar.** DEMO profile (gate 1, about 5 ft) at table setup so a crowded room does not hold presence on, boundary walked with the real crowd, back to HOME afterwards. The pairing beat and the buddy beat both need a judge at the table to count as present.
- **Honesty labels, never improvised around.** The qualifier at the detect beat; reason codes on history marked inferred; SYNTHETIC on the titration scenario; DEMO on every screen; a confidence label on every metric and never a blank row; “no patient data shared with any manufacturer” on the handoff; unconfirmed on a Brain-only listing. Each beat has a canned fallback per the cut order; a missing label is never worked around live.
- **Q&A numbers to memorize.** 8.7% of short-acting insulin users filled glucagon in a decade; 17.5% of adults with T1D have impaired awareness, with about 4x the severe lows; the two questionnaires disagree about 60% of the time; insulin needs drop 25 to 35% on a GLP-1 start and should be reassessed within 2 to 3 days and at every 4-week step; over half stop within a year; \$396 per patient per year of possible review billing ($12 \times \$33$), always “may support”; about 6.3 million type 2 insulin users, roughly 3x the type 1 population; 13 clinically driven touchpoints per tirzepatide watch ($5 \times 2 + 3$). The full Rounds Q&A ammunition and the Night Buddy table live in `demo/DEMO_SCRIPT.md`.

Deployment and Hosting

Principle: local-first. The alarm engine, LEDs, and display work with zero internet, because a safety device that dies with the Wi-Fi is a worse product. The cloud only adds reach. Said explicitly in the pitch.

- **Layer 1, the device.** The Pi is the deployment target: git pull plus a systemd restart updates the nightstand. install.sh sets it up once; irin.service and kiosk.service start everything on boot. kiosk.service is installed disabled and enabled by hand after the final audit; until then the Pi boots to the desktop. Once enabled, the NO_KIOSK flag file on the boot partition, a 20-second start delay, and no-respawn keep the terminal reachable.
- **Layer 2, cloud Nightscout.** Nightscout stays on Railway, fed by Dexcom Share through the connect plugin, with its database on MongoDB Atlas (one MONGODB_URI). The Pi just points at its URL. Setup notes in deploy/nightscout-railway.md.
- **Layer 3, Cloudflare Tunnel.** cloudflared on the Pi opens an outbound named tunnel that serves the Pi's API at device.irin-out-of-sleep-at-hackgt.tech (the domain's DNS lives on Cloudflare), no port forwarding. The app reaches the Pi from anywhere, including over cellular, which also sidesteps venue Wi-Fi device isolation. Because the Pi becomes internet-reachable, auth.py gates every mutating route behind the PIN and the pairing token.
- **Layer 4, the family view.** family.irin-out-of-sleep-at-hackgt.tech, a small read-only page (current value, trend, last night) served by the same Caddy, reading one rollup from Irin Cloud with a revocable family bearer. No Nightscout token in page source any more; the token the page carries grants one rollup and nothing else, and the patient revokes it from the Family section. The "check on your kid from another state" story, and a URL judges can open on their own phones. It is never shown in the Impiricus slot.
- **Layer 5, Irin Cloud on Vultr.** One Vultr VPS (Ubuntu 24.04, Docker Compose) is the whole Irin-owned cloud, at irin-out-of-sleep-at-hackgt.tech, api.irin-out-of-sleep-at-hackgt.tech, and cloud.irin-out-of-sleep-at-hackgt.tech behind Caddy, which issues HTTPS certificates itself (which is why every DNS record stays DNS-only, proxy off): the relay (the courier for Rounds and Night Buddy: cards, doctor messages, pairing, buddy alerts, the hub, and now the buddy directory and matching; its documents on MongoDB Atlas; it stores sealed envelopes and profile fields only, never a glucose value, and the "what Impiricus sees" panel is its log) and the cloud service (Irin Cloud: the ingest endpoint the Pi forwards to, Tiger Cloud time series, the dashboards API, the family rollup, audio rendering for buddy alerts). The Pi only ever calls out to both, so no new port opens on the Pi. Fallback if Vultr or the venue network fails (stated once, in chris.md's R6 fallback paragraph, cited here): the same compose file on Chris's laptop serves everything on one origin, http://<laptop-ip> (Caddy on plain HTTP for the LAN): the app's dist, the role pages, the relay, the cloud service, and the local mongo and timescaledb containers the same file declares. The Pi's APP_ORIGIN is set to http://<laptop-ip>, the app's dist is rebuilt with the VITE_* values pointing at http://<laptop-ip> (or Justin runs npm run dev -- --host on the laptop with APP_ORIGIN set to that :5173 origin), the role pages' config.js point at the laptop, and judges' phones on the hotspot open http://<laptop-ip>, so nothing is ever mixed content. Rehearsed once at home.
- **Layer 6, the role pages.** doctor.irin-out-of-sleep-at-hackgt.tech (the inbox, "mock Ascend"), watch.irin-out-of-sleep-at-hackgt.tech (buddy alerts and the hub list), and family.irin-out-of-sleep-at-hackgt.tech are static pages served by the same Caddy beside the app. They talk only to the relay and Irin Cloud, hold their keys in the browser, and decrypt there; the Pi never hears from them. Participants without a device: a watcher (buddy or hub volunteer) needs nothing but a phone and the watcher page, plus a CGM feed behind the account, since only the watchable can be watchers.

A watched patient with no device runs the same backend with the hardware flags off (IRIN_HW=mock, brain_only, DATASOURCE=nightscout pointed at their own Nightscout); that Brain-only backend is not deployed for the hackathon: Brain-only is demonstrated from the Pi's adapter with IRIN_BRAIN_ONLY=1 (or the demo-panel toggle); the design note stands that it could run as its own container in the compose file later. Their listings carry the unconfirmed label under the stricter rule.

Priority if time runs short: the domain and Caddy first (HTTPS on every name is what makes the rest demoable), the Pi's tunnel second (the app cannot reach the device without it), Nightscout on Atlas third (one env var), the relay on Atlas fourth (Rounds and Night Buddy cannot demo without it), Irin Cloud and the My Irin tab fifth (the wow, but the device works without it), the family page last (polish).

Hackathon Packing Checklist

Pack directly from a working system: plug everything in at home, trigger one fake low end to end (LEDs, sound, phone acknowledge), then power down and pack from that state.

The Device

- Assembled unit: SmartiPi Touch Pro 3 case with Pi 5, Active Cooler, and Touch Display 2, ribbon seated
- 256GB microSD flashed and already in the Pi (verify before leaving)
- LED U-frame already mounted on the case: three channel segments with diffusers, corners joined, input wire routed
- Mini breadboard with the built circuit: SN74AHCT125N, 330 ohm resistor, Rubycon 1000uF cap
- LD2410B-P radar already mounted (camera mount or 3D-printed housing below the screen), adapter cable connected to pins 4, 9, and 11, joints taped
- Spare male-to-female Dupont jumpers for extending the radar wires
- Phone with the HLK Bluetooth app installed and paired to the radar, for switching to the DEMO distance profile (gate 1, about 5 ft) at table setup and back to HOME afterward
- Spare 1.25-to-2.54mm adapter cables (the 10-pack; bring at least three)
- Green screw-terminal barrel adapter
- LIELONGREN speaker
- The display's GPIO power cable, connected (5V pin 2, ground pin 6)

Power

- RasTech 27W USB-C PD power supply
- ALITOVE 5V 5A brick
- Power strip with long cord, name written on it
- UGreen power bank, charged the night before, plus a USB-C to USB-C cable
- Both USB-A to barrel jack cables

Cables and Spares

- Second GODIYMODULES ribbon cable from the 2-pack

- Both Inland HDMI to micro-HDMI cables
- All four Anker cables (A-to-C x2, C-to-C, C-to-A)
- Inland F-M jumper set and the ELEGOO Dupont pack
- 2 to 3 spare SN74AHCT125N chips, the resistor assortment, 2 to 3 spare 1000uF caps
- Spare strip length from the 16.4FT reel, spare L-corner connector sets, the 3M 5925 tape roll
- 52Pi HDMI screen and its USB cable, boxed, as the backup display

Tools and Consumables

- Small Phillips precision screwdriver and the display box's spare screws
- KATA hacksaw and ValueMax files, in case frame work continues at the venue
- USB microSD card reader
- Electrical tape, 3M strips or velcro, zip ties, scissors
- Multimeter (borrowed is fine) and a few Wago 221 lever connectors; a soldering iron only if someone already owns one
- Sandwich bags or a parts organizer for screws and chips, and a sharpie

Computing and Demo

- Laptop and its charger
- Phone and its charger (the phone is the web app and the Dexcom receiver)
- Phone hotspot enabled and tested with the Pi before the event (venue Wi-Fi often blocks device-to-device traffic)
- Backup demo video downloaded onto the laptop and phone, not just in the cloud
- Clarity CSV export, trained model file, and repo cloned locally on the laptop
- Pi login credentials and the Nightscout URL and API secret written down somewhere that is not only on the Pi
- The buddy demo phone: a teammate's, the PWA installed, the watcher page pre-paired, on the hotspot; and a second phone or the laptop for the clinician inbox
- Relay URL and source key, and the .tech names (app, api, cloud, device, doctor, watch, family) written down somewhere that is not only on the Pi
- The QR pairing tested on every team phone, and the demo phone pre-paired as the fallback
- Every cloud key in the private note, off the Pi: Atlas and Tiger connection strings, Vultr root access, ElevenLabs, Backboard, Anthropic, the Meta Model API key, the WhatsApp token and phone number id, the relay source key, the device pairing token
- The laptop compose fallback rehearsed once at home (chris.md, R6 fallback): docker compose up, one origin, every page and the app from <http://<laptop-ip>> on a phone on the hotspot, the app's dist rebuilt with laptop VITE_* values (or npm run dev -- --host) and the Pi's APP_ORIGIN set to the laptop

Personal

- Diabetes kit stocked for the full event plus margin: pens, backup sensor, low treatments within reach of the desk

- Water bottle, snacks, any meds
- Hoodie or layer, deodorant, toothbrush if overnight
- ID for registration, headphones

Beyond the Hackathon

Every logged event is stored with full timestamps in the same schema the model trains on. As logging accumulates over months, meals, insulin, and exercise become real model features and the hand-written context rules retire. Version 2 is a retrain, not a redesign.

Rounds and Night Buddy are the same spine's next audiences: Irin Brain runs on any CGM feed as software, finds the patients worth confirming, and the device is the doctor-recommended instrument that confirms them; the buddy relationship is the sales channel for the device. Every card, alert, and morning line comes from the same ledger and the same validated narrative, so the growth loop is a distribution question, not a rewrite.

Rounds, the growth loop and what comes next. A clinic turns Rounds on for every CGM patient it follows, free and with no hardware; cards flag the patients whose nights are ambiguous or risky; the doctor recommends Irin Bedside to those patients, and their cards become full-context, which answers "who buys the device?" without requiring anyone to. The device reaches the patient by purchase, a clinic-owned loaner pool returned at graduation, or a device-maker program; never pharma-funded, because an item of value from a drug maker to a patient is anti-kickback exposure. In production Brain runs in a patient-owned cloud instance the way the community already self-hosts Nightscout. Next: weekly basal insulin watches as Awiqli launches, the same engine on a different therapy start; Rounds for type 2 insulin users as the main commercial push; automated insulin delivery users, where the card becomes about settings rather than doses; other connected devices publishing the same card format (smart pens, pumps, blood pressure cuffs, inhalers); adverse-event drafting as a card type; and real evidence, a small prospective cohort at one clinic measuring time to insulin adjustment, severe lows, glucagon prescribing, and persistence at 6 months against matched patients.

Night Buddy, the full tier. background alerts as native Critical Alerts or a call through the WhatsApp/Twilio calling API (the demo keeps the watcher page open with a wake lock and sends the WhatsApp message; a web page cannot ring through silent), the brokered call as a real telephony call, a matching flow on schedule compatibility and stated preference (AI matching on management style and alarm patterns is roadmap), the hub audit view, and duty-of-care review before anyone is promised a service rather than a friend who happens to be far away. Scale honesty stays in the pitch: a pair works at $n = 2$; the hub needs thousands before the list is ever non-empty (1 in 50 members with a hub-worthy event a month on a 10,000-person network is about 200 listings a month, roughly 7 a night worldwide).

Family Story, next. Save Shared, a patient-initiated action that sends one save as a story to whoever the patient picks, kept out of the automatic channel so the morning email never discloses more than its level promises; and the story card at the top of the family view, fetched from the relay, for Level 2 recipients.